

Tor Vergata University of Rome



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FACULTY OF ENGINEERING

**COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII**

PhD Thesis

**A UNIFIED SOFTWARE ARCHITECTURE FOR
DISTRIBUTED AUTONOMOUS SYSTEMS**

ADVISOR

Prof. Sergio Galeani

CANDIDATE

Roberto Masocco

COORDINATOR

Prof. Francesco Quaglia

A.Y. 2023/2024



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

A UNIFIED SOFTWARE ARCHITECTURE FOR DISTRIBUTED AUTONOMOUS SYSTEMS

PHD THESIS

Advisor:

Prof. Sergio Galeani

Signature:_____

Coordinator:

Prof. Francesco Quaglia

Signature:_____

Candidate:

Roberto Masocco

Signature:_____

A.Y. 2023/2024

Tor Vergata University of Rome, Faculty of Engineering
Via del Politecnico 1, Rome, Italy

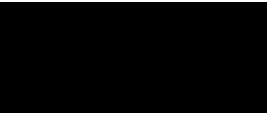
Wings are a constraint that makes
it possible to fly.
— Robert Bringhurst



Acknowledgements

Some day, when this is finished, I will write something here.
This is not that day.

Rome, December 2024



Preface

This is the summary of three years of work spanning multiple areas. From the problem to the solution. It is inevitable to name some tools and concepts before they are explained later on among chapters. The project described herein is under active development and the contents of this document may not be completely up-to-date or they may already include fixes and update that are going to be deployed in the short term.

Contents

Acknowledgements	i
Preface	ii
List of Figures	v
List of Tables	vii
Listings	viii
1 Introduction	1
2 The importance of an architecture for autonomous systems	6
2.1 The design challenges of autonomous systems	8
2.1.1 Defining the tasks	8
2.1.2 Choosing the hardware	9
2.1.3 Organizing the software	11
2.1.4 Handling data	12
2.1.5 Debugging and testing	13
2.2 Basic requirements	14
2.3 Related work	18
2.3.1 RoboStack	19
2.3.2 eProsima Vulcanexus	20
2.3.3 EPICS	21
2.3.4 BlockNet	24
2.3.5 NASA F Prime	26
2.3.6 MARTe2	27
3 Tools for the job	32
3.1 Inter-process communication: the middleware	33
3.1.1 Data Distribution Service	38
3.1.2 Zenoh	45
3.1.3 Robot Operating System 2	52
3.2 Containerization	74
3.2.1 Docker Engine	80
3.3 Version control	85

3.3.1	Git	87
4	The proposed Distributed Unified Architecture	94
4.1	Assumptions	96
4.2	System abstraction layer: dua-foundation	99
4.2.1	Common software dependencies	102
4.2.2	Currently supported hardware platforms	104
4.2.3	Base units	108
4.3	Unit template: dua-template	116
4.3.1	Unit filesystem layout	118
4.3.2	Configuring a new unit	122
4.3.3	Updating dua-template in units and projects	132
4.4	Remarks	133
5	The case of autonomous robots	135
5.1	Software utilities for ROS 2: dua-utils	136
5.1.1	Managing ROS 2 processes: dua_app_management	139
5.1.2	Handling Quality of Service correctly: dua_qos	143
5.1.3	Improving node development: params_manager, dua_node	147
5.1.4	Wrapping clients: simple_serviceclient, simple_actionclient	152
5.1.5	Implementing ROS 2 automata: transitions_ros	158
5.2	Developing robotic software with DUA	163
5.2.1	Autonomous flight: flight_stack	164
5.2.2	Driving a sensor: zed_drivers	176
5.3	Preliminary results	185
5.3.1	A ROS 2 distributed architecture for unmanned aircraft	185
5.3.2	Efficient sensor fusion for autonomous agents	200
6	The case of tokamak devices	207
6.1	Magnetic confinement fusion on TCV	208
6.1.1	The Tokamak à Configuration Variable	212
6.1.2	The Système de Contrôle Distribué	214
6.2	Improving the SCD network with the DDS	219
6.2.1	Design of the DDSDataSource for the MARTe2 framework	222
6.2.2	Features of the DDSDataSource	227
6.2.3	Preliminary benchmarking results	233
A	The dua-template update workflow	239
B	Unit configurations	243
	Bibliography	259

List of Figures

2.1	Organization of the core libraries of the MARTe2 framework [12].	28
3.1	Software organization in a general-purpose computer, as presented at the 1968 NATO conference on software engineering [14].	36
3.2	A network of applications using the DDS, each denoted by its own DomainParticipant [20].	42
3.3	Possible network topologies in Zenoh [21].	50
3.4	Placement of the Zenoh protocol in the network stack [21].	51
3.5	ROS 2 logo [25].	53
3.6	Scheme of the message communication primitive [41].	61
3.7	Scheme of the service communication primitive [42].	61
3.8	Scheme of the action communication primitive [43].	62
3.9	State machine of an action goal [44].	62
3.10	Core ROS 2 API organization [45].	64
3.11	ROS 2 event-based execution model.	66
3.12	Simulation of an autonomous drone with imaging and depth sensors in Gazebo (right) and visualization of the data in RViz (left).	71
3.13	micro-ROS architecture [69].	75
3.14	Docker logo [75].	80
3.15	Git logo [85].	88
4.1	Some of the platforms currently supported by DUA, either with full support or with some limitations as in Table 4.1; from top left to bottom right: a Dell XPS 13 laptop, a Microsoft Surface tablet, a MacBook Pro, an Nvidia Jetson Xavier AGX mounted on a mobile robot prototype, an Up Squared, an Nvidia Jetson Nano, an Nvidia Jetson TX2 development kit, a Raspberry Pi, and an Nvidia Jetson Xavier NX.	110
4.2	Directory tree of a DUA unit based on <code>dua-template</code> [115].	119
5.1	Functional diagram of a hardware architecture for autonomous flight: a Pixhawk 4 flight management unit (FMU) (top), and an Nvidia Jetson Xavier NX SBC (bottom).	168
5.2	The 2021 Leonardo Drone Contest field layout.	186

5.3	Software architecture of the LDC21 autonomous drone: thin arrows indicate communications established with ROS 2 interfaces, thick arrows represent dedicated software interfaces, white boxes represent ROS 2 nodes.	189
5.4	High-level finite-state machine for the Leonardo Drone Contest 2021 mission.	195
5.5	Low-level finite-state machine for the Leonardo Drone Contest 2021 mission.	195
5.6	VSLAM algorithm performance evaluation test.	197
5.7	Laboratory test of the 2021 Leonardo Drone Contest mission; the green path represents the setpoints while the red one represents the estimated trajectory.	198
5.8	2021 Leonardo Drone Contest mission run; the green path represents the setpoints while the red one represents the estimated trajectory. . .	199
5.9	Successful precision landings performed on the ArUco landing pads during the 2021 Leonardo Drone Contest.	201
6.1	Schematic of the TCV device.	210
6.2	TCV poloidal cross-section with an example plasma configuration (lower single-null). The OH, E, F, and G coils are highlighted in different yellow, green, blue, and red, respectively. Removable baffles appear in the cross-section in gray protruding inward in the bottom part of the vessel. . . .	214
6.3	Schematic of the Système de Contrôle Distribué (SCD) architecture. . .	218
6.4	Steady-state cycle time of the real-time thread containing a Publisher DDSDataSource running in Synchronized mode.	235
6.5	Steady-state cycle time of the real-time thread containing a Subscriber DDSDataSource running in Synchronized mode.	237
6.6	Steady-state cycle time of the real-time thread containing a Subscriber DDSDataSource running in NotSynchronized mode.	238

List of Tables

2.1	Comparison of the solutions presented in Section 2.3 with respect to the requirements listed in Section 2.2; “✓” denotes a completely satisfied requirement, “+” indicates that the solutions provided do not fully satisfy a requirement, and “×” marks an unsatisfied requirement. Is DUA, the framework presented later on in this work, going to meet them? . .	31
4.1	Docker features relevant for the DUA framework and their support on different operating systems.	99
4.2	Summary of the technical specifications of some of the single-board computers currently supported by DUA.	109
4.3	Sizes of the compressed Docker images of the base units currently available in dua-foundation.	116
4.4	Comparison of DUA against the solutions presented in Section 2.3, with respect to the requirements listed in Section 2.2; “✓” denotes a completely satisfied requirement, “+” indicates that the solutions provided do not fully satisfy a requirement, and “×” marks an unsatisfied requirement.	134
5.1	Main technical specifications of the SBCs used in the implementation of [67].	188
5.2	RMSE between the ORB-SLAM2 estimated trajectory and the ground truth waypoints.	198
5.3	Laboratory test RMSE (m) along each axis.	199
5.4	2021 Leonardo Drone Contest run RMSE (m) along each axis.	200
5.5	Main technical specifications of the SBCs used in the implementation of [68].	206

Listings

3.1	Definition of the <code>sensor_msgs/Image</code> message.	61
3.2	Definition of the <code>example_interfaces/AddTwoInts</code> service.	62
3.3	Definition of the <code>example_interfaces/Fibonacci</code> action.	62
4.1	Possible invocations of the <code>dua_setup.sh</code> script.	122
4.2	Docker Compose command to open an instance of the Zsh shell inside a DUA container.	124
4.3	Default command of a DUA container: an idle shell process.	127
4.4	Unit configuration step in a Dockerfile as generated by the template.	128
5.1	Example of <code>params_manager</code> YAML configuration file [121].	150
5.2	Signature of the <code>Client::call_sync</code> class method from the C++ action client wrapper library <code>simple_actionclient_cpp</code> [125].	157
5.3	Example of transitions and state tables in the <code>transitions_ros</code> library [131].	162
5.4	Unit configuration of the <code>jetson5</code> target of the <code>flight_stack</code> DUA unit [137] as of November 26, 2024.	175
5.5	Unit configuration of the <code>jetson5</code> target of the <code>zed_drivers</code> DUA unit [142] as of November 26, 2024.	183
5.6	Unit configuration of the <code>x86-cudev</code> target of the <code>zed_drivers</code> DUA unit [142] as of November 26, 2024.	184
6.1	Sample configuration settings of a Publisher <code>DDSDataSource</code>	232
6.2	Sample configuration settings of a Subscriber <code>DDSDataSource</code>	233
A.1	<code>dua-template</code> update GitHub workflow as of November 22, 2024 [115].	239
B.1	Unit configuration of the <code>x86-cudev</code> target of the <code>flight_stack</code> DUA unit [137] as of November 26, 2024.	243

1 Introduction

In the history of humanity, many turning points have been marked by technological advancements. From the invention of the wheel to the steam engine, from the light bulb to the computer, technology has continuously changed how we live, work, and see the world. Its impact on society has always been profound and sometimes positive, sometimes negative. The changes it brought affected our daily habits, introducing new ways of doing things and sometimes even new ways of thinking. With the same spirit that drove our ancestors to develop *tools* made of stones and sticks to make their lives easier while harvesting food, we are now developing ever more powerful machines of all sorts to handle the challenges of the modern world, which can work either together with a human operator or in complete autonomy. Moreover, considering the recent developments of artificial intelligence, these tools have reached such a level of sophistication that they may soon reshape our thought processes.

The technological advancements that brought us these new solutions increased not only their capabilities but also their complexity and, ultimately, their variety. An excellent example is the industrial manipulator robot: a machine that can perform various tasks, from simple pick-and-place operations to more complex ones such as welding, painting, or even surgery. These machines have evolved from mechanical arms composed of multiple actuators, all connected to a single power and computational unit, to a *cyber-physical system*: a network of sensors and microcontrollers, grouped

in subsystems all interconnected to a central unit, which is, in turn, responsible for their coordination, supervision, and control, as well as for interfacing with human operators. In this case, this evolution has been driven by the need to increase the capabilities of the robot making it more versatile and adaptable to different tasks, and by the need to make it more autonomous as well as more reliable, reducing the risk of accidents and downtime due to maintenance. Another suitable example would be the modern airplane: considering even a small civilian aircraft, originally equipped with an engine, control surfaces, and a few instruments, it has evolved into a complex system of systems composed of a multitude of sensors, probes, indicators, and avionics. In this scenario, the evolution has been driven by specific needs still related to safety and comfort but can ultimately be summarized in the need to ease the lives of both the passengers and the crew.

This design and development paradigm has recently become commonly denoted by the adjective *distributed*. The definitive characteristic of a distributed system is its decomposition into a network of subsystems. Each one is responsible for a specific task; all of them are interconnected, and they all perform a piece of the job that the entire macroscopic system has been designed to do. This decomposition allows for more efficient use of the resources, better scalability, and higher reliability, as the failure of a single subsystem does not necessarily lead to the failure of the whole system. Moreover, the distribution of the subsystems allows for a better organization of the system's functionalities, as each subsystem can be designed and developed independently from the others, following a modular approach that eases the task of understanding, controlling, and maintaining the system as a whole. This approach has been widely adopted in many fields, from industrial automation to aerospace, automotive to robotics, telecommunications to energy. Another peculiar characteristic of a distributed system is the possibility, in many contexts, to deploy its parts

in different locations: coming back to the manipulator example, it is common to have the sensors and the actuators distributed along the robot's arm and the control unit placed in a separate location, while in the airplane example, the sensors and the avionics are distributed along the fuselage and the wings, while the cockpit is placed in the front of the aircraft; in pure software systems, the subsystems can be deployed in different servers, data centers, or even continents, with the additional advantage of partitioning the workload generated by the various regions among local computational nodes. These degrees of physical distribution allow great flexibility in design and resource allocation, but they also introduce new challenges in communication, synchronization, and fault tolerance.

All of this innovation comes at a price: when the multitude of subsystems, subcomponents, and submodules increases, new problems arise. The system's complexity grows, and so does the difficulty of understanding, controlling, and maintaining it. The interactions between the different parts of the system become intricate. With strong dependencies among modules and no redundancy, the consequences of a failure in one of them can propagate to the others, causing a cascade of failures that can lead to catastrophic events. A deep understanding is needed to prevent events like these, for which a good organization of the system *architecture* is of paramount importance. Moreover, a well-designed architecture may also be incredibly beneficial in two other stages: the design and development of the system and its maintenance. In the former, a clear and well-structured architecture naturally leads to a clear and well-structured design, following a top-down approach that starts from the system's requirements and ends with the definition of the subsystems and their interfaces, each following a common set of standards and best practices that adhere to the structure of the architecture itself. In the latter, a well-structured architecture allows for a clear identification of the subsystems and their interactions, easing diagnosing and fixing a

failure or an error. Finally, a good architecture can also be a powerful resource for the evolution of the system as a whole, allowing for the addition of new functionalities or the replacement of old components with new ones without the need to redesign the whole system from scratch, significantly reducing the time and the costs of such an operation.

The subject of this work is the development of a software architecture for distributed systems intended to control dynamic processes and autonomous agents. This setting may seem specific, but it is pretty broad: all of the above examples fall into these categories, representing topics of high interest in today's research and industrial environments. All the design and development problems listed above still apply to these cases, which also bring their own specific challenges addressed in the following. The main goal of this work is to provide a unified solution to these problems by defining a software architecture that can be used as a reference for the design and development of distributed systems in the contexts mentioned above. The proposed architecture is designed to be modular and flexible, and it is based on a set of standards and best practices that may guide the development of the subsystems and their interactions. Moreover, the architecture is designed to be adaptable to and deployable on different hardware platforms, to provide an abstraction layer for solutions ranging from embedded systems to general-purpose computers, and to easily integrate other software modules like third-party libraries. Finally, the architecture is built to be easily usable by developers, providing a unified set of tools and utilities that ease the task of developing, testing, and deploying new modules for the classes of systems mentioned above. The end goal of this work can be summarized as follows: building a standard development environment suitable for a single developer or an entire team, capable of replicating almost entirely the system configuration of a target platform on a development machine and enabling distributed applications to work and behave

in the same way on all supported platforms, from build to run time. This way, the integration overhead gets dramatically cut, allowing developers to focus on their most important task: research and development, leading quickly to new technological advancements.

Just like our primordial ancestors faced the necessity of building tools to ease their lives, this work follows the same approach: identifying the problem, laying down the requirements, designing the solution, building it, testing it, and finally using it. The requirements that drive the design of the architecture are identified and discussed in the following chapters, and the proposed solutions are based on the state of the art in the field of software engineering and distributed systems, as well as on some practical experience acquired during the years in which this work has been developed.

2 The importance of an architecture for autonomous systems

The development of an autonomous system is a process that involves various stages. Each stage is typically characterized by defining and solving issues related to different aspects of the system, ranging from the tasks it will be required to perform, to its construction and programming, and even to how the data it collects and that affect its operation should be presented to potential operators. Generalizing from a specific application case, most issues can be classified into a few high-level categories. Although it may seem counterintuitive from the division that follows, these categories are not always addressed sequentially, nor are they entirely independent of each other. To address situations like these, various project management techniques have been developed in recent times, aimed at minimizing the risk that an error made in an earlier phase could compromise subsequent stages, potentially undermining finished work and increasing development times. In light of the observations made in Chapter 1, it is clear that, with the increasing complexity of modern automated systems and of the tasks they must perform, these topics are more relevant than ever. In particular, new tools made available to developers and designers will need to reflect these specificities, simplify the management of typical situations where possible, and highlight potential critical issues to aid in problem-solving. Practical experience shows that, despite

the variety of cases and applications, the challenges faced are almost always, if not identical, very similar, as are the strategies to address them.

The aim of this chapter is to briefly present the issues typically encountered when designing an autonomous system, highlighting their unique characteristics without focusing on a particular application area, and then to examine some recent hardware and software solutions that enable the optimization of design, development, and testing activities. The discussion starts at a relatively high level of abstraction but becomes progressively more specific, forming the guiding thread that connects all subsequent chapters, finally highlighting the convenience of a well-structured software architecture to address the issues raised. The requirements that such an architecture must meet are then formalized and discussed in due detail.

Before moving further, it is instrumental to clarify the definition of an autonomous system in the context of this work. First of all, in this work the term *system* identifies a macroscopic entity, which can either be entirely hardware, or entirely software, or a mixture of both, that evolves in time and is characterized by a set of inputs and outputs. Its tasks are defined by the operations that it must perform on these inputs to produce the desired outputs, and by the constraints that these operations must satisfy. The system can either be seen as a unitary entity or as a set of *subsystems*, *subcomponents*, and *submodules*, all terms that are used interchangeably in this work, each of which takes care of a part of the processing, thus contributing to a part of the tasks. For the system to work correctly, all the subsystems must do the same and be interconnected accordingly, both in terms of the data they share and of the resources they use. The term *autonomous* refers to the capability of the system to perform these tasks without human intervention, or with minimal human intervention, once it has been set up and started. This definition is intentionally broad and generic, as it is

meant to encompass a wide range of applications and scenarios, *e.g.*, from mobile robots to drones, from power plants to industrial automation systems. Another class of systems that is included in this definition and that is of interest in this work is that of *dynamic systems*, which are systems whose state evolves in time according to a mathematical model that can be either deterministic or stochastic, and that can be either known or unknown. The evolution of the system's state, and thus that of its output, can be influenced by acting accordingly on its input, an operation that is commonly referred to as *controlling* the system or *applying a control law* to the system. In the context of autonomous systems this can be one of the tasks to perform, which is especially true for those systems that have a physical structure that can be modeled as a dynamic system and that should evolve over time according to a specific desired behavior.

2.1 The design challenges of autonomous systems

2.1.1 Defining the tasks

The initial decisions in the design of an autonomous system naturally concern the specifications of the operations it will be required to perform. These specifications must be fully defined from the onset, down to the smallest detail possible at this stage, as they will guide all subsequent choices, from hardware selection and construction to programming. During the breakdown of operational requirements into *tasks*, intended as discrete units of work to be executed, various types of these will emerge. Without delving, for now, into the specifics of a particular scenario, common categories of tasks include the following ones.

- **Continuous tasks**, which begin at system startup and run uninterrupted until it

is turned off or malfunctions.

- **Periodic tasks**, which are performed at regular intervals.
- **Asynchronous tasks**, which are executed only when a specific event occurs, which may or may not ever occur.

It is easy to see that most functionalities in modern autonomous systems can be broken down into tasks that fall within these categories. These include operations ranging from actuator control, to the collection of data and measurements required by control algorithms, to communication with eventual operators and users. The characteristics of these tasks will influence both the hardware that must execute them and the software that encodes them, which should be organized accordingly to facilitate development and maintenance. To allow this organization, the system architecture must efficiently support the development of software modules, and the integration of hardware solutions, that can handle all these kinds of tasks.

2.1.2 Choosing the hardware

The next decisions to be made concern the hardware to be used. The choices made initially in this direction may very well not be definitive, since they would concern both the construction of the initial prototypes and the final product: there will likely be misjudgments or unforeseen issues to address down the road. Nevertheless, the hardware must be selected carefully based on the operational, structural, and construction requirements defined so far. Decisions taken in the previous step are of critical importance at this stage: establishing the computational complexity of the operations the system will need to perform will dictate the level of power required

from the hardware, which may need to be equipped with discrete coprocessors^I to handle specific types of data or subdivided into multiple interconnected subsystems.

A market survey is essential to assess whether pre-tested and ready-to-use components or SoCs^{II} could be utilized rather than resorting to a proprietary solution, which generally entails significant development, testing, and validation costs. For the reasons mentioned above, it is very challenging, if not impossible, to outline a comprehensive and definitive list of possible hardware solutions at this point; the priority is to understand what is needed and whether the market offers options compatible with those requirements, while also anticipating potential future demands. Following this approach will allow any subsequent modifications to be managed with minimal effort.

The remaining choices concern lower-level hardware, which generally consists of:

- high-speed microcontrollers;
- actuators;
- sensors;
- interfaces and communication devices;
- data storage systems.

Nothing more can be said about these elements without reference to a specific case; however, practical scenarios discussed later in this work will follow this design pattern.

From the multiplicity of devices mentioned up to this point, and that may compose

^IThe term *discrete* here refers to auxiliary hardware installed in a computer that cannot operate independently but is designed to be driven by the primary computer, providing specialized support for specific tasks; classic examples include floating-point math coprocessors, parallel and graphic coprocessors, and sound cards.

^{II}System-on-Chip.

a modern autonomous system, it starts to become clear how the complexity of the system can grow rapidly, as anticipated in Chapter 1. The need for a well-structured architecture to manage this complexity becomes more evident.

2.1.3 Organizing the software

Except for those components that perform crucial and highly specific functions, such as electrical or electronic circuits, nearly all operations carried out by a modern autonomous system are today encoded within an internal computer, which translates these instructions into commands for other components, subsystems, sensors, or actuators. This is not the place to delve into standard software design and development procedures, but it is worth reflecting on one point which has become quite common in present solutions: given the underlying complexity of the system itself, the software that controls it will also have a layered and highly hierarchical organization. In this hierarchy, the lower levels are occupied by modules that must interact with, if not directly control, the smaller and faster subsystems (*e.g.*, firmware), and these modules must be developed with particular attention to their characteristics and purposes. As we move up the hierarchy, we encounter broader modules that encode operations such as supervision, planning, and user communication, making them closer to the notion of *application* or *system software* which end users are more accustomed to as it is the most visible to them.

This context also involves decisions regarding the programming languages to be adopted. As is well known, each language has its unique features, highlighting its strengths as well as weaknesses compared to alternatives. Only a few programming languages are truly suitable for multiple purposes, in the sense of the ease with which they allow different types of tasks to be coded, and none should be considered

universal.

When operating in this context, it is essential to rely on tools that simplify design and development as much as possible at all levels, assisting with the management of issues related to module organization, inter- and intra-process communication, data processing, and storage of various data types. The convenience of a good architecture to handle these new challenges becomes increasingly evident.

2.1.4 Handling data

For an autonomous system to successfully complete its tasks, it requires a nearly constant stream of information. This information can be received in data packets managed by suitable communication interfaces, or it can be acquired directly through measurement devices, commonly known as *sensors*. For both solutions, typical challenges that arise immediately include the integration of these devices with the rest of the apparatus and the processing of the data they produce. These issues are crucial, as the operation of the entire system is almost always contingent upon obtaining information about measurable quantities of interest and the system's own state^{III}.

It is therefore reasonable to expect that the hardware architecture of a modern autonomous system will comprise numerous communication interfaces and sensors, the latter consisting of specific hardware designed to measure particular physical quantities of interest. The challenges mentioned above pertain to the integration of such hardware with the rest of the system and to the development of software modules capable of managing it, receiving and processing the data it generates. Both the hardware platforms and software tools should, as far as possible, be developed or selected with these requirements in mind, to minimize their integration overhead.

^{III}Consider, *e.g.*, feedback control algorithms.

2.1.5 Debugging and testing

As soon as the initial prototypes are finalized, testing should begin to verify their functionality and confirm the validity of the design decisions made up to that point. Bearing in mind that a generic and universal testing procedure cannot be defined for all types of autonomous systems, it can generally be expected that once hardware functionality is confirmed, attention will shift to software validation. Debugging software running on an autonomous system is generally more challenging than debugging common application software due to several specific factors:

- It is more difficult to trace program execution on an autonomous, potentially mobile device: direct access to the system may not be available, and the tools at hand are often limited or not very sophisticated.
- In the event of issues, both the software and hardware in use must be re-checked, as the problem could stem from hardware faults or incorrect use of the hardware.
- Conducting tests is not as simple as running a program and reviewing its results: the safety of the apparatus and its operators must be ensured throughout the entire procedure, which can make even the organization of a test a challenging task.

Given the difficulty and importance of this phase, it is reasonable to employ tools and methodologies that can facilitate these tasks as much as possible, ideally those that have already been adequately validated.

2.2 Basic requirements

The challenges outlined in the previous section are not exhaustive, nor are they entirely independent of each other. They are, however, representative of the issues that typically arise when designing an autonomous system. Summarizing briefly the many aspects previously covered, suppose that the task at hand is that of developing an autonomous system capable of performing a series of operations. It is reasonable, in the context of this work, to focus on situations that meet the following assumptions.

- The system has a physical structure that can be modeled as a dynamic system, thus, one of the tasks consists in stabilizing and controlling this dynamic system, something that is commonly referred to in the following as *low-level control* or simply *control*.
- Digital programmable computers of different kinds are used to both develop and run all the algorithms, ranging from low-level control to task planning and supervision.

Note that the situations hereby described might span from mobile robots to drones and even power plants, all classes of systems that have been subject of this research work. Recalling Section 2.1, a series of choices regarding the following items must be made:

- sensors and actuators to use;
- software tools and packages to employ and integrate;
- hardware platforms, *i.e.*, onboard electronics to adopt.

Note how these items might be strongly interconnected. Low-level hardware must be chosen according to the control objective for the underlying dynamic system, while high-level hardware must be selected usually weighing the software that it will run against the physical capabilities, as well as the costs, of the system that is going to host it. Finally, software tools and packages can be divided in two groups: that which has to be developed from scratch, possibly very specific to the system at hand, and that which can be reused from existing libraries or frameworks. The latter category, for example, includes solutions for data processing, communication, and visualization, as well as remote access and monitoring tools; when one wants to develop the prototype of an autonomous system, especially for research purposes, these things are often overlooked, but they are crucial for the system to be usable and maintainable and the overhead that they might introduce can easily become significant if not properly managed early on.

There is a principle in computer science [1], dating back to the very early days of the discipline, stating that to do a proper job *no problem should ever be solved twice* or, in a historical fashion, *one must never reinvent the wheel* with the only exception of building a better wheel. One can see how this fits perfectly in the context of the development of autonomous systems: there are a wide range of complex foundational and integration problems to solve which concern the very base of any such system, and thus present themselves each time a new one is developed. Because the author has spent a significant amount of time and effort solving these problems while working on projects whose ultimate goal was that of advancing research in the field of autonomous dynamic systems, the scope of this work is to provide a solution that is *reusable*, *modular*, and *maintainable*, and that can be used as a foundation for the development of innovative prototypes and products. With this in mind, the following requirements are formally identified.

- (M) Modularity** The architecture should constitute both a development and deployment framework for solutions ranging from single modules, meant to be easily integrated and reused, to entire autonomous systems prototypes. To guarantee this, the structure of the framework should suit both these use cases: single modules should carry with them all their dependencies, and it should be possible to easily *merge* single modules to build a “superproject” out of many “subprojects”.
- (D) Development** The architecture should support the development of single modules both as standalone packages, and as parts of the software suite of a prototype, *i.e.*, while working on a superproject that integrates a module, it should be possible to modify and update the module easily propagating the updates to every other superproject which integrates that module, unless such superproject has been specifically configured to use a different version of the module. The architecture should also ease testing and debugging activities, independently of the environment in which a module is developed.
- (A) Abstraction** The architecture should offer a *system abstraction layer*, *i.e.*, it should make it possible to carry on all the development activities without having to care too much about the underlying hardware on which a module or prototype software suite will be deployed, apart from unavoidable specific differences, as well as to the target OS of such a platform. The architecture should ease deployment by offering a common framework across multiple different hardware platforms. In the end, the architecture shall offer very similar, if not equal, development and execution environments across different hardware platforms, requiring minimal effort to be replicated.
- (O) Overhead** The architecture should be as lightweight as possible, in terms of both computational and storage resources. This is to ensure that the overhead intro-

duced by the framework itself is minimal, and that the resources of the hardware platforms on which the framework is deployed can be used to their full potential for the tasks that the system is meant to perform.

Note that these requirements, if examined by the perspective of subjects such as systems engineering, might not appear well-posed due to, *e.g.*, the lack of a clear and objective way to evaluate them. This is intended though, since they are meant to guide the design of an architecture and not that of a specific system. Moreover, the design stage at which they are introduced is one in which many things still have to be evaluated such as, *e.g.*, hardware platforms to support and preexisting software tools to integrate, and thus the final definition of the architecture will be the result of many trade-offs. They are high-level requirements meant to guide the design process, characterizing the solution from a broad and functional point of view.

Coming back to the generic, although representative, example provided at the beginning of this section, it appears clear how a framework that meets these requirements would be beneficial, cutting down to the minimum all of the overhead that is usually due to integration and basic organization activities in the development of an autonomous system. Such an architecture would, in fact, provide a common base layer that would both allow developers and teams to focus on their project- or research-specific tasks in a common environment, and ensure that all the work done is reusable, maintainable, and easily deployable on a variety of hardware platforms, ideally ranging from the simplest SoC to the most complex general-purpose computer.

2.3 Related work

To further motivate the premises of this work, this section presents some of the most relevant solutions that have been proposed in recent years to address the issues outlined in Section 2.1. The list, although short, is in fact exhaustive: to the best of the author's knowledge at the time of writing no other such framework has been developed and proposed in the literature. Furthermore, a review of the works presented hereafter would reveal to the interested reader that none of them fully meets the requirements identified in Section 2.2, especially those that deal with the independence from the underlying hardware and the construction of a replicable development environment. Commercial and proprietary solutions have been voluntarily left out of this review: there exist commercial solutions that satisfy many requirements up to a certain point such as, *e.g.*, MathWorks's MATLAB [2] and Simulink [3]. Such solutions, however, limit the capabilities of the user to what their vendors decide to provide, also imposing a mandatory organization to the user works. As the following chapters clarify, among the aims of this work there is also that of providing to users a platform that follows its own set of standards and practices, but at the same time does not enforce them in any way and does not require any proprietary tool, allowing instead the user to modify and expand any of its parts into what best suits their needs. Finally, to discuss the following, it is inevitable to mention solutions and tools which will be the subject of subsequent chapters: they will be presented in due detail in the following, while proper references are provided here. At the end of this section, a brief comparison of the solutions presented hereafter with respect to the requirements listed in Section 2.2 is provided in Table 2.1.

2.3.1 RoboStack

The RoboStack framework [4], published in 2022 and maintained ever since, constitutes a first promising solution to the challenges outlined in Section 2.1. RoboStack is a collection of software packages built on the Conda package management system [5], which has become the de-facto standard in both industry and academia for the management of Python *environments*, a concept that has gained particular interest in the scientific community in recent years thanks to the popularity of Python as a programming language in contexts like big data processing, statistics, machine learning, and artificial intelligence. The Conda package manager allows to specify a set of software modules to be collected and installed in an isolated location, together with their specific versions in case such a level of granularity becomes necessary. Once such a configuration is specified, it can be replicated with ease on any other machine and filesystem path, regardless of the operating system and many other factors, thus providing a way to ensure that the software environment in which a program is developed and run is always the same. RoboStack builds on this concept, providing a wide set of software packages and libraries that are commonly used in the development of robotic systems, such as the Robot Operating System (ROS) and the Gazebo simulator, illustrated in Section 3.1.3, and that are sometimes difficult to install and configure correctly, especially on embedded platforms. The framework is designed to be used in conjunction with the Conda package manager, thus supporting a variety of operating systems ranging from Linux to Windows and macOS, and it is meant to be installed on machines that will be used for the development of robotic systems, deploying environments through the selection of specific sets of packages to install in each. This solution, although promising and already widely applicable, does not completely meet the requirements previously identified, for three reasons. First, it is entirely dependent on Conda which, although a very powerful tool, may not be the

best choice in terms of *isolation* of the software environment: while Conda allows one to install software packages in an isolated location, it does not provide a way to define a specific portion of system resources to offer to that software, a feature nowadays informally called *containerization* and explained later on, which could be useful to further tune the performance of the software running on the system and ensure that the host OS installation stays intact. Second, every piece of software must be added to RoboStack to be installed in one of its environments, which means that the framework is not as flexible as it could be, as that it is not possible to install any software package available from any source out of the box; the addition process requires a variable amount of work, particularly due to the many dependencies a software module may have, as explained in [4]. Last, RoboStack is dependent on Jupyter for development, which is not the best choice in terms of development environments. While Jupyter is a very powerful tool, its main purpose remains that for which it has been originally developed: offering a way to write and run Python code multiple times in interactive notebooks displayed in a web browser, which is not really adherent to the scenario outlined at the beginning of this chapter which involves developing and testing in real time the software that will run on an autonomous system. Moreover, many plugins existing for other visualizers and IDEs, some of which are presented in the following sections, have to be reimplemented to be used in a Jupyter environment, which is not always possible or convenient. These limitations, although not critical, make RoboStack not the best choice for the development of autonomous systems, especially when the requirements identified in Section 2.2 are taken into account.

2.3.2 eProsima Vulcanexus

The Vulcanexus software suite [6] developed by eProsima, originally released in early 2023, poses itself as a simple framework for the development of autonomous robotic

systems. The suite is composed of a set of software packages that are meant to be installed either on a host operating system or inside a *container* managed by, *e.g.*, the Docker Engine presented in Section 3.2.1. The suite is fairly lightweight but nonetheless comprehensive: it provides the Robot Operating System 2 (ROS 2) middleware, together with the Fast DDS by eProsima [7] implementation of the Data Distribution Service (DDS) communication middleware presented in Section 3.1.1, plus some more software tools aimed at more specific scenarios like, *e.g.*, that of microcontrollers. Moreover, the fact that it is shipped also as a Docker image, compatible with different hardware platforms, adds some flexibility to it in terms of isolation of both the development and execution environments with respect to, *e.g.*, the aforementioned RoboStack framework. However, the major limitation of this framework lies in the fact that its features more or less end here. With respect to the requirements listed in Section 2.2, no effort is made towards the construction of a modular development and deployment environment, nor towards the support to a wide range of hardware platforms, especially in the case of SoCs aimed at robotic systems which, as will be explained later on, are gaining an increasing interest by the scientific community but require a fair amount of effort to be correctly set up. Furthermore, the restriction to the Fast DDS communication middleware for ROS 2, although evidently motivated by the fact that it is also developed by eProsima, hinders the flexibility of the framework in terms of communication protocols, an aspect that becomes critical in some operational scenarios where, *e.g.*, the network is highly subject to packet loss as explained in Section 3.1.2.

2.3.3 EPICS

EPICS (Experimental Physics and Industrial Control System) [8, 9] is a distributed software framework designed to meet the demanding requirements of control systems

in experimental physics, particularly for large-scale accelerators and other complex installations. Originally developed at the Los Alamos National Laboratory, U.S.A., and first published in 1991, the aim of EPICS is to address the challenges of managing data acquisition, control processes, and supervisory management in high-performance environments, where existing industry solutions at the time were inadequate. EPICS is built on a distributed architecture that utilizes a software communication bus, allowing for the seamless integration of multiple functional subsystems. These subsystems include the Input/Output Controllers (IOCs), Display Manager, Alarm Manager, Archiver, and the Sequencer. Each subsystem is designed to operate in a distributed manner, ensuring that tasks can be easily distributed across nodes in a network, thereby scaling from small, localized test setups to expansive, highly distributed networks. The idea is that nodes in the network can represent either actual subsystems of the plant, or users connected to workstations that act as terminals, on which EPICS also runs to let them and their software interact with the control system. The Channel Access protocol serves as the communication backbone of EPICS, providing a "software bus" for communication between various components of the system. Channel Access supports operations like connecting, getting, putting, and monitoring data fields across distributed databases, using both TCP and UDP protocols. This protocol enables subsystems, including user-extended components, to communicate efficiently. One of the core features of EPICS is its distributed database, which is hosted on IOCs and provides functionalities such as data acquisition, data conversion, alarm detection, and closed-loop control. The IOCs run on hardware platforms such as VME or VXI with a real-time vxWorks kernel and are connected via Ethernet. This configuration allows efficient execution of control tasks and the ability to extend the system easily by adding more controllers. Other main components share data to and from the database, in a distributed fashion. Among those there are multiple

Managers, such as the Display Manager which provides a graphical interface to the current user connected to the workstation, and the Alarm Manager which keeps track of the state of all the alarms in the system and presents them in a hierarchical fashion to the rest of the architecture and to the users. EPICS also features a Sequencer, which uses a state notation language to manage the system through different operational states and handle exceptions. This sequencer allows for modal transitions of the IOCs, such as switching between operational modes or handling unexpected system events. EPICS has demonstrated considerable extensibility, allowing new hardware and software modules to be integrated with minimal modifications. This extensibility is further bolstered by ongoing upgrades to both hardware and software components to improve the system's performance and ease of configuration. Among the related frameworks considered in this context, EPICS is without any doubt the most relevant: it has been developed in a similar context to that of this work, *i.e.*, that of physics research and development, and evolved into an industry-standard. However, its scale is also what makes it not applicable to the context of this work. EPICS is designed to manage large-scale installations, such as particle accelerators, and is not suitable for the development of small-scale prototypes or products, as it is too complex and resource-intensive, providing functionalities and services like, *e.g.*, slow remote control and supervision. Moreover, its architecture is not modular, and it is not designed to be easily integrated with third-party software modules or libraries, which is a critical aspect in the context of this work. Finally, EPICS is not designed to be used on a wide range of hardware platforms, as it is tightly coupled with the vxWorks kernel and similar proprietary solutions which do not fit the practical contexts at which this work is originally aimed.

2.3.4 BlockNet

BlockNet [10] is presented as a novel multiplatform control framework designed to address the limitations of existing control-oriented frameworks like, *e.g.*, Simulink and EPICS. Unlike these alternatives, BlockNet aims to provide an integrated solution that supports simulations, real-time control, and the integration of complex technologies while maintaining execution order, ensuring dynamism, and achieving portability across different computing environments. The primary goal of BlockNet is to enable the seamless creation of local or distributed control systems by breaking down computations into individual code blocks that can be linked to form a computational network, completely similar to what would be the block diagram or Simulink diagram of a control system: the idea is exactly that of providing a framework for the development and deployment of control architectures starting from their modeling as block diagrams. This approach is highly beneficial in designing general-purpose control algorithms while maximizing code reusability. One of BlockNet's significant features is its capacity to execute these blocks in the correct order based on their input-output dependencies, thus ensuring deterministic behavior even in distributed systems. Moreover, BlockNet attempts to utilize all available computational resources by executing compatible blocks in parallel and incorporating asynchronous programming to improve overall efficiency. BlockNet is developed on top of the .NET framework, using the C# language to ensure compiled-level portability across multiple operating systems and architectures, such as Windows, Linux, macOS, x86, ARM, and others. The use of .NET also enables effective memory management and provides support for modular extension of the framework, allowing users to add new functionality easily. BlockNet introduces communication primitives made available to nodes based on a *publish-subscribe* paradigm, illustrated in Section 3.1.1, but extends it to ensure deterministic execution, making it well-suited for real-time control applications. This paradigm

allows easy distribution of data between blocks in a network or even across multiple machines without complex network configurations being required. BlockNet's distinguishing features include its emphasis on ease of use, dynamic adaptability, and extendibility. These characteristics make it suitable for a wide range of applications, from purely simulated environments to hardware-in-the-loop (HITL) systems. Unlike many existing frameworks, BlockNet allows the runtime modification of blocks and entire algorithms without the need for recompilation or system downtime, thereby enabling continuous integration and deployment scenarios in control engineering. The key architectural components of BlockNet range from timers and timing sources, to schedulers, data serializers, and mathematical libraries and solvers. BlockNet is indeed close to what could satisfy the requirements stated in Section 2.2, although in that section lies also the main reason why it cannot be considered a solution to the problem that this work addresses. Apart from the fact that it starts from the aim of conceptually modeling distributed systems as block diagrams, a choice that may pose its own limitations, BlockNet tries to provide by itself a solution to any problem that, as the rest of this work illustrates, arises during the development of such a framework. This is a critical aspect, as it makes the framework less flexible and more complex than it could be, as well as prone to errors and design flaws, and less open to contributions from the scientific community. In contrast, this work starts from a thorough review of existing solutions that may solve parts of the original problem, requiring only some integration work and the development of what is missing; the next Chapter 3 is entirely devoted to the detailed description of such tools. Moreover, the choice of the .NET framework, although it is a very powerful and flexible tool, is not the best one in terms of portability and integration with third-party software modules and libraries, as it is a proprietary solution that is not widely used in the scientific community compared to other programming languages and toolchains, and that requires a specific develop-

ment environment to be used as well as a certain degree of expertise to be developed with.

2.3.5 NASA F Prime

The F Prime framework [11], developed in the United States by the Jet Propulsion Laboratory (JPL) of the National Aeronautics and Space Administration (NASA) and published in 2018, is currently one of the solutions that better fit the requirements listed in Section 2.2. It is a lightweight, modular, and open-source framework for the development and deployment of software for small atmospheric and orbital vehicles like the CubeSats and the Mars Helicopter prototype. It provides the foundations for a software architecture oriented at flight control in which software modules, written in the C++ and Python programming languages, can be loaded and configured easily as *components* of the architecture itself. The framework, due to its organization, encourages the development of software packages meant to be published and reused among subsequent missions and prototype developments. Moreover, the framework offers ad-hoc communication primitives and services, as well as some thread scheduling capabilities, to ensure that software tasks in the architecture meet their real-time requirements, an aspect that is crucial in flight and space operations. Finally, thanks to its light organization and basic system requirements, it supports a wide variety of general-purpose and real-time operating systems (RTOS) and can easily be compiled on many hardware architectures. This framework is very promising and shows a lot of potential, but still does not fully meet the aforementioned requirements, mainly because of its limited diffusion and specialized scope. Although it is open-source, it is aimed at a very specific scientific sector, that of space missions, thus receiving limited acclaim outside of such area. Moreover, a downside of its lightweight nature is that it offers limited support for the integration of third-party modules and libraries,

a fact which limits the capability of the framework of being used in a wider range of scenarios comprising, *e.g.*, that of swarm aerial robotics, as well as the potential benefit of contributions from the scientific community. Finally, as declared in [11], in its current version it depends on commercial, licensed software for some of its functionalities, which could be a limitation for users outside NASA and JPL.

2.3.6 MARTe2

The MARTe2 framework [12], standing for *Multi-threaded Application Real-Time executor 2* [13], is the second qualified framework that is presented in this section. Originally developed in 2005 by a team of nuclear fusion researchers at the Joint European Torus (JET) UK facility and then registered as a work of the Fusion For Energy (F4E) european organization, it has been under active development ever since and is currently employed in many nuclear fusion research facilities around the world. As its name suggests, it is a framework aimed at the development of real-time software control architectures, similarly to the EPICS framework described in Section 2.3.3 but on a smaller scale. It is written in C++ and supports only this programming language. Its organization is actually the closest to what is requested in Section 2.2 and thus deserves a little deeper analysis. It is composed by a core set of libraries plus a set of optional "components" that add specific functionalities or support for particular hardware devices, algorithms, communication protocols, etc. As illustrated in Figure 2.1, the core libraries are divided in three distinct groups:

- **BareMetal**: a set of libraries that implement basic functionalities ranging from data types and hardware abstraction, to communication primitives and logging, and provide the definition of base classes that can be used to develop new components of an architecture.

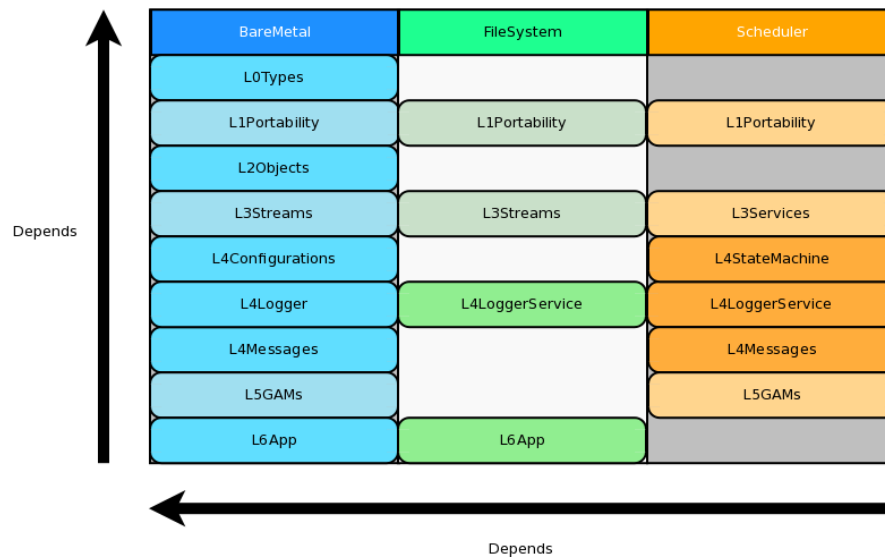


Figure 2.1: Organization of the core libraries of the MARTe2 framework [12].

- **FileSystem:** these libraries provide functionalities to read and write data from and to files, as well as to manage the configuration of the system and the logging of the data produced by the components.
- **Scheduler:** libraries in this group implement real-time thread schedulers and their algorithms, as well as models such as state machines that can be useful to keep track of the state of a software component or algorithm.

Moreover, note that libraries in each group follow a hierarchical organization, with those on the lower levels providing basic functionalities and those on the higher levels providing more complex and specialized ones. All of the framework and its components can be built using the GNU^{IV} make tool, as every MARTe2 software package comes with its own Makefile and the entire framework relies on a system of Makefiles that define environment variables and search paths for the compiler, then finally invoke it. The vision guiding the structure of the framework, although it has now reached a maturity that could make it applicable also in contexts that are

^{IV}GNU is Not Unix, <https://www.gnu.org/>.

completely different from nuclear fusion, *e.g.* again, that of robotics, is still that of its early days: implement real-time software control architectures for physics research plants. Thus, its scope has remained the same ever since: managing software units that run *algorithms* and exchange *data* organized in *signals*, which are collections of data of a specific type and dimension, modeling algebraic objects ranging from scalars to vectors and matrices. This idea reflects the main classes of software components that can be found and implemented in any MARTe2-based architecture, and that are the following:

- DataSource: a component that exchanges data organized in signals with the external world, *e.g.*, a hardware device or another software component.
- Broker: a component that manages the exchange of signals between higher-level components and DataSources, implementing memory synchronization schemes if required.
- GAM: short for *Generic Application Module*, a component that implements an algorithm that processes input signals and produces output ones, *e.g.*, a control algorithm or a data filtering algorithm.

Finally, a MARTe2-based architecture is usually divided in one or more applications, intended as processes running on the host machine, which contain a number of components specified in a configuration file together with their configuration parameters. Such configuration files, informally called *cfgs*, have a specific syntax, allowing an application to be built by literally "assembling" portions of a configuration file that define the components and their parameters, and then running the application with a specific configuration file that specifies the components to be loaded at run time, plus the settings for the OS scheduler. The reason why this work exists and MARTe2,

although being the most promising solution, is still found in this section, is that as every other work hereby discussed it has its own downsides that set it a bit too much apart from the requirements stated earlier in this chapter. The first ones descend from its original scope, and recall the ones commonly found in previously examined works: it has been developed to aid in a very specific scientific research field, that of nuclear fusion in this case, and thus found very little to no adoption in the years outside of it; this translates to limited improvements over the original organization, and lack of support for modern technologies, third-party libraries, and use cases. The code organization and build system described early on is functional and efficient but follows old practices, like the focus on signals rather than software components, and relies on tools like `make` which, although standard and widely adopted, have nowadays been superseded by more modern solutions which are more suited to the management of large and modular codebases. Due to its strict real-time nature, MARTe2 also imposes some restrictions on the development of software modules, which must be written in C++ and must adhere to old language standards, lacking modern features and erecting a barrier against the integration of many open-source libraries and tools. These facts, together with the effective but cryptic syntax of the `cfgs`, make MARTe2 a very powerful but also clunky framework which often scares away newcomers and makes it difficult to maintain and evolve the software architectures built with it. Finally, although its compilation requires very little features from the host system, it is not officially supported on architectures other than x86 and requires a tailored OS kernel to fully exploit its real-time capabilities, limiting its application in practice to Linux-based systems if one wants the smoothest possible usage experience.

	Modularity	Development	Abstraction	Overhead
RoboStack	✓	×	+	✓
Vulcanexus	×	+	+	✓
EPICS	✓	✓	×	✓
BlockNet	✓	+	+	×
F Prime	✓	+	+	✓
MARTe2	✓	+	+	✓
DUA				

Table 2.1: Comparison of the solutions presented in Section 2.3 with respect to the requirements listed in Section 2.2; “✓” denotes a completely satisfied requirement, “+” indicates that the solutions provided do not fully satisfy a requirement, and “×” marks an unsatisfied requirement. Is DUA, the framework presented later on in this work, going to meet them?

3 Tools for the job

The previous chapters highlighted the need for a new software architecture that may constitute the foundational layer of distributed autonomous systems, and provided a solid set of requirements that can drive its design. Existing frameworks have been reviewed, some of which provided valuable insights and ideas about how to address some of the design challenges, but none ultimately offering the complete set of features required. Before presenting the design of the solution that this work proposes, it is important to delve further into a point that the previous discussion shed light onto: there are solutions available in the literature, in the industry, and in the open-source software community, that can be employed to address some of the key issues that the requirements identify. Following the computer science principle recalled in Section 2.2, that is, to not reinvent the wheel, it would be wise to review these tools and see how they can be integrated in the proposed solution. This chapter provides such an overview: for each tool, its motivations are shown by illustrating a problem that is common in the development of distributed autonomous systems, and then the tool itself will be presented, along with a brief discussion of its features and limitations. Note that the aim of this work is to provide a general-purpose software architecture, so not all of the solutions presented in this chapter, as well as all the features that the proposed architecture will have, may be necessary or even beneficial in every practical context. However, the goal is to provide a solution that can be adapted to a wide range

of applications, as a good toolbox to have at hand, offering a set of features that can be configured and used as needed. Indeed, recalling that one of the requirements is also to develop an architecture that can be maintained over time, the intention is that the tools presented in this chapter will be included because they represent valid solutions at the time of writing to the problems they address, which are relevant in this context, but since not a thing of this is set in stone each aspect of the proposed architecture can be subject to change in the future should the need arise.

3.1 Inter-process communication: the middleware

When developing distributed systems composed of many modules, especially software ones, the transmission of data among them becomes a topic of paramount importance. Note that this issue concerns both transmissions happening between software modules running on the same host and on different hosts. Moreover, alongside this topic there is that of *intra-process communication*, that is, the organization and regulation of data exchanges among different threads running in a same process. Examples of situations in which many software modules may coexist inside a single process, and why such a situation may be desirable, are presented and discussed in the following, for now let us focus on the general problem of inter-process communication and simply note that in the majority of the situations, this other problem can be solved by employing either the same tools that are used for inter-process communication, or simpler ones based on shared memory areas guarded by synchronization primitives. One has, in general, to figure out a variety of issues such as the following:

- **Data formatting**, *i.e.*, how information that should travel from one module to another should be encoded and structured in packets or similar entities.

- **Data serialization and deserialization**, *i.e.*, how packets should be transformed into a format that can be transmitted over a medium, which is usually some kind of point-to-point connection or a network.
- **Data transmission**, *i.e.*, how packets should be sent and received, and how the system should react to errors or delays in the transmission; this also includes things like reliability requirements, that is, whether the system should guarantee that a packet is delivered, and how many times it should try to deliver it if it fails.

The points listed above are usually dealt with by employing a specific *protocol*, that is, a predefined set of rules that regulate all those aspects of the communication. All the software that composes the system should adopt such protocol, which is usually implemented in a set of libraries that can be used in the code. However, dealing directly with networking protocols might slow down the development process: the host system configuration usually becomes very important, since all the operating systems involved must agree on the protocol implementation to support and use, and a lot of work may become necessary to proficiently use all the protocol APIs to achieve all the desired communication features and behaviors. Let us remember that this is not the original task: the goal is to develop a distributed autonomous system, and to write software for it, thus it would be desirable to have a tool that abstracts all the networking details and provides a simple and easy-to-use API that makes data transmission in software modules a straightforward operation, from both the coding and implementation points of view. This is where a particular kind of software, historically denoted by the term *middleware*, comes into play. To grasp its importance in the context of this work, a step back in time is necessary.

In 1968, a conference sponsored by the Science Committee of the North Atlantic Treaty Organization (NATO) was held in Garmisch, Germany, on the topic of soft-

ware development. Although not widely discussed, it was a historically significant event in this field: it provided a comprehensive overview of the design practices and methodologies used by leading companies in the growing and highly competitive field of computing. The goal was to identify standard strategies for addressing common problems. It should be noted that at that time, and for at least another fifteen years, there were virtually no standards in computer production and programming. Each company offered its own product developed from scratch, in both hardware and software. Standardization and collaboration among various manufacturers began only when information started being exchanged between different computers, first through storage media and later over the Internet, alongside a growing demand for new types of devices and standard formats to adopt when sharing data. A fairly complete transcription of the conference can be found in [14]. It includes the first official mentions of several concepts well known today, such as *component design* and *unit testing*. However, of particular relevance to this work is the representation of the hierarchical structure of software presented by one of the speakers. This structure, valid even today with certain extensions, is illustrated in the inverted pyramid scheme shown in Figure 3.1. The idea is that a complex system is like an inverted pyramid: it cannot remain operational without adequate support provided by secondary components. These might be completely invisible to the end user and thus may be considered somewhat external, yet are of vital importance for the correct functioning of the whole: *compilers* and *assemblers*, for instance, which are programs that translate code written in a given language first into the target architecture's assembly language and then into binary machine language, typically generating an executable file at the end of the processing pipeline. These tools must be reliable and efficient, as the implementation of everything else depends on them. Next, we have the layers of *Control Programs* and *Service Routines*: these are what we today call the *operating system*

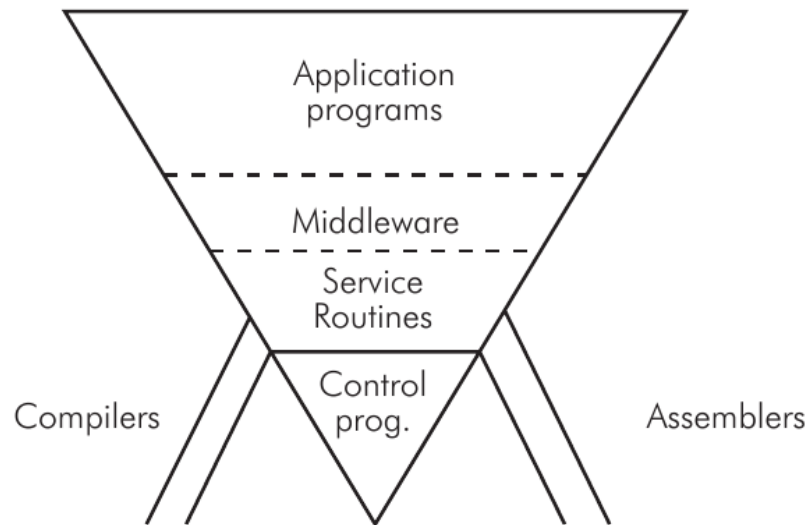


Figure 3.1: Software organization in a general-purpose computer, as presented at the 1968 NATO conference on software engineering [14].

and *device drivers*. They are typically developed with a strong dependence on the machine at hand, considering its specific characteristics, and heavily depend on the employed compilation tools. Their role is to provide an abstraction of the hardware that makes its use simple both for the end user, who interacts with applications, and for application developers themselves. The application software is precisely what we find in the topmost layer: programs that use the hardware more or less directly to provide various services to users. There is, however, an intermediate layer that is often overlooked because it was not always present or was difficult to characterize in the past: the *middleware*. Situated between system software and application software, middleware found its first definition and characterization at the aforementioned conference. Initially conceived merely as a connective layer between current and *legacy* software, middleware provides services and functionalities common to applications beyond what the operating system offers, building itself on top of it and abstracting its own implementation and protocol details. From the perspective offered by the topmost level of the inverted pyramid it appears as normal application software, often

consisting of shared or dynamic libraries. Middleware typically offers services to programmers aimed at, *e.g.*, data management, communication, and authentication. Therefore, middleware aids developers in building applications more efficiently by acting as a connective tissue between applications, data, and users. In distributed environments, middleware can facilitate the development and execution of large-scale applications. A typical service offered by middleware is a method for data exchange between applications, even across different devices, according to a common format referred to as an *interface*, which is defined as needed. In general, the services offered by middleware are specified only in terms of their dynamics but can be implemented on different architectures and devices, functioning consistently while keeping their implementation hidden from both the application programmer and the end user. For example, when middleware handles communication between applications, it might position itself within the network stack between the transport and application layers¹. Today, middleware can be categorized into two types: that which operates in *human time*, designed for direct use by human users (*e.g.*, web services), and that which operates in *machine time*, composed mainly of APIs that application developers can use to facilitate their work, benefiting from increased portability across a wide variety of devices. In light of all this, and considering the common difficulties faced when designing the software for a distributed autonomous system, it becomes clear how adopting communication middleware to solve the problem of data exchange among modules can simplify things: many functionalities that would have to be implemented from scratch would instead be offered by the middleware itself as services to be employed in the programs being developed. The selection of this particular class of middleware is again motivated by both the current trends in the software industry pertaining to the development of autonomous systems.

¹Considering the TCP/IP model, which is a four-layers reduction of the original seven-layers Open Systems Interconnection (OSI) model of the networking protocol stack.

The rest of this section will introduce three specific middlewares that are widely employed in the industry and in the open-source community, and that have become of pivotal importance in the two practical scenarios considered later in this work as a specialization of the proposed architecture: that of autonomous robotics and that of power plants. The first middleware is the *Data Distribution Service* (DDS), which has been the first to provide a standard for data exchange in distributed systems, followed by the Zenoh middleware which addresses some of the limitations of DDS, and finally the *Robot Operating System* (ROS), which is a middleware specifically designed for robotics applications. All these three components will be included in the architecture proposed in the following chapters, making some of its foundational layers.

3.1.1 Data Distribution Service

In 1989, eleven companies including Hewlett-Packard, IBM, Sun Microsystems, Apple Computer, and American Airlines, founded the *Object Management Group* (OMG), a consortium that is still active in the field of computing with the goal of defining standards for industrial software. This effort aims to foster the integration of solutions from different vendors into unified ecosystems, covering a variety of applications and technologies. The purpose of the OMG is to establish, for every new type of software under consideration, a common model to be followed during its implementation, thus enabling and promoting its development and use on any potentially relevant platform. The group provides the industry with specifications only, not implementations. However, any team submitting a new specification must provide a potential implementation before it is accepted, intuitively to avoid defining impractical or useless standards. OMG member companies encourage the development of solutions that can interoperate immediately without any limitations.

One of the standards defined by the OMG is the *Data Distribution Service* (DDS) [15, 16]. It specifies a particular type of middleware responsible for managing direct communications between computer systems, including the details of how data should be serialized, deserialized, transmitted, and routed, as well as how APIs should be structured for programmers to invoke these operations. Its definition has been carried over with the goal of allowing implementations to satisfy real-time determinism guarantees whenever necessary. Moreover, the goal of the DDS is to enable technologies, platforms, and devices from different vendors, operating in distinct contexts and locations, to immediately communicate, exchanging data for which only the format and minimal other attributes must be known beforehand. This facilitates the management of critical aspects, like performance and scalability of the various components and the network connecting them. Effectively, the DDS greatly simplifies what would otherwise be for the programmer a complex task of configuring networks and organizing data transmissions, which, as discussed, is one of the classic challenges faced during the development of a distributed autonomous system. As such, the DDS has become widely employed in industrial contexts ranging from manufacturing [17] to power grids [18].

The middleware model follows a *publish-subscribe* paradigm: entities involved in the communication declare their intent to transmit or receive packets of information, which are encoded according to appropriate interfaces, and are called informally *publishers* or *subscribers* and instantiated as `DataWriters` or `DataReaders` respectively. The publish-subscribe denomination arises from the way different transmissions are organized, from the programmer's perspective, into channels denoted as *topics*. Topics are defined by:

- a **name** that identifies the topic, generally encoded as a text string to be easy for

programmers to remember and use;

- an **interface**, described using a specific language (*e.g.*, an Interface Description Language (IDL)^{II}), which specifies the content of a single data packet and how it is encoded.

Interfaces are intended as structured data types, *i.e.*, data structures containing ordered fields of different types, ranging from simple integers and floating-point numbers to more complex structures like arrays and strings, or other interface types as well. One key aspect to consider is that, given an interface specification, the middleware will take care of generating the code necessary to serialize and deserialize data packets according to that interface, as well as to transmit and receive them over the network. This is a significant advantage, as it allows the programmer to focus on the data itself and on the logic of the application, without having to worry about the details of the network communication. Another key aspect to consider, which also makes the DDS very versatile, is that communications are asynchronous by nature: `DataWriters` publish data on a topic regardless of the presence of any `DataReader` listening on that topic. `DataReaders`, on the other hand, can subscribe at any time to a given topic and start receiving messages from that point on. The OMG DDS standard provides APIs for both synchronously waiting for new messages, as well as asynchronous notification and handling via *callback*^{III} functions.

The behavior of any entity in a DDS network, and thus that of every transmission, can be configured by specifying a Quality of Service (QoS) policy for it [19]. A DDS QoS

^{II}The acronym IDL identifies a class of languages that allow the description of operational elements intelligible by different languages. Various IDLs exist, and in this context, DDS employs one to make the specification of a data packet format simple for programmers, although no specific IDL is mandated by the OMG standard.

^{III}A callback, in a software program, is a routine that gets called asynchronously, *i.e.*, when an event occurs for which that callback is registered, and either an event handler or a job scheduler invoke such callback to process the event.

policy typically consists of items such as:

- the **history policy**, which indicates whether all packets transmitted on a specific topic should be delivered or buffers, limited to a certain size, should be put in place and rotated in case of network congestion;
- the **history depth**, which is the depth of the finite queues optionally set in the previous point;
- the **reliability policy**, indicating whether communications should be managed by the middleware to ensure that all subscribers of a topic receive all packets transmitted by publishers, or packet drop and loss are allowed, thus enabling *retransmissions* if necessary;
- the **durability policy**, allowing transmitted packets to be retained by publishers and delivered to subscribers that may connect later;
- the **deadline, lifespan, liveliness, and lease duration**, which are characteristic maximum times to determine when a packet has become outdated or an entity can be considered inactive.

Of course, there are default settings that are kept as consistent as possible across different implementations to ensure maximum compatibility. A complete scheme of a DDS network of applications can be found in Figure 3.2.

Even at this level, the significant simplification introduced by the DDS middleware becomes evident: with a relatively simple and quick configuration, issues such as choosing a transport protocol (*e.g.*, Transmission Control Protocol (TCP) or User Datagram Protocol (UDP)) and developing software modules to use it for transmitting information encoded according to specific rules and formats can be immediately

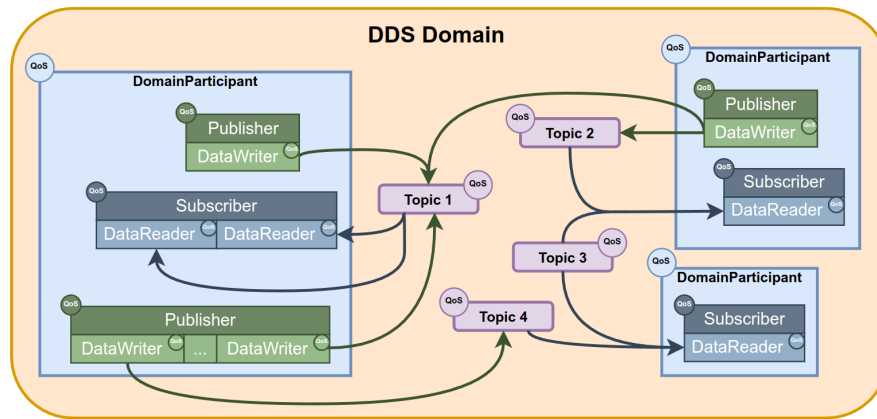


Figure 3.2: A network of applications using the DDS, each denoted by its own DomainParticipant [20].

resolved. The DDS takes care of all tasks necessary to organize communications: from the most basic, such as message transmission, to more organizational tasks like publisher advertising and node discovery within the network, as well as flow control and retransmissions. It also handles publisher redundancy and other pathological situations—all in a way that is completely transparent to the programmer. DDS, in effect, makes it very simple to build distributed applications or networks of applications, as it hides and simplifies many management tasks, allowing the developer to be abstracted from factors that could otherwise prove complex to manage. Commercially available DDS implementations today offer API libraries in various languages, such as Ada, C, C++, C#, Java, Python, Scala, Lua, Pharo, and Ruby, and since they are all built on the same standard, they are fully compatible and interoperable. There are also specifications concerning lower-level types of DDS, designed to be included in resource-constrained systems, such as embedded systems, while still maintaining compatibility with more comprehensive high-level implementations.

Communication modes, serialization, and deserialization of data are defined in the OMG standard called *Wire Protocol*. Despite the broad scope of this specification, it is instrumental to highlight two key parts for the benefit of the following discussions.

Discovery Module

This module of the DDS includes protocols that allow different DDS instances running on network-connected systems to discover each other and organize potential communications from that point on. Generally, there are two such protocols:

- **Participant Discovery Protocol (PDP)**, which allows DDS instances—each associated with a particular application running on a specific machine, hence called *participants*—to announce their presence to others while simultaneously obtaining information about any other reachable participants.
- **Endpoint Discovery Protocol (EDP)**, which comes into play immediately after PDP, and relates to transmitting information about the *endpoints* owned by the various participants. These endpoints, as publishers or subscribers, are deemed compatible if they share the same topic and interface, and if the QoS policies *offered* by publisher entities satisfy the *requests* of subscriber entities.

Implementations can choose to include, beyond the PDP and EDP protocols defined by the standard^{IV}, proprietary versions. If two instances initiating communication have at least one PDP and one EDP in common, they can exchange information and complete the discovery process.

These protocols would be located at the Internet Protocol (IP) level in a network stack, and can be implemented in either *unicast* or *multicast*, depending on whether all participants are known in advance or not. In the former case, preliminary network configuration is required to preset the addresses and port ranges of all involved machines, while in the latter, the protocols will automatically manage everything as soon as the DDS instances are launched. The potentially vast operational scale of these

^{IV}These are known as **SPDP** and **SEDP**, respectively, where “S” stands for “Simple”.

solutions has made DDSs attractive for mission-critical areas such as military and logistics, although, for now, relying entirely on the Internet for such communications is not advisable. Unless predefined configurations are in place, it is necessary for every network device along the path between two participants to support multicast and not block such transmissions.

Platform Specific Model

This model establishes how data of different formats and encodings should be serialized, transmitted, and deserialized in a uniform manner across all possible implementations and platforms. In an IP network, all data packets are transported over UDP. Whether the endpoints are associated using multicast or unicast, the Discovery Module's protocol opens UDP *sockets* for them with well-defined port numbers, which depend on:

- the **domain ID**, a numerical constant that selects a range of UDP ports in the implementation, allowing applications using DDS to communicate simply by specifying the same domains; multiple domains can coexist even on a single machine;
- the **base range** of UDP ports available in the system;
- the **participant ID**, a unique identifier for an instance within a machine;
- any other **manual**, specific configurations.

One may wonder why UDP is used for data transport. The reasons are similar to those in other contexts where a deterministic behavior is desirable: UDP is a basic, best-effort protocol without any communication management features beyond simple transport between endpoints. As such, it is lightweight and supported by a wide

variety of devices and platforms. This simplicity gives UDP implementations complete predictability, excluding the unpredictability of the network itself, and natural scalability. Therefore, it is the best choice for real-time systems as well as for the development of this type of software.

3.1.2 Zenoh

The Data Distribution Service middleware effectively solves the inter-process communication problem in distributed contexts, allowing applications to easily set up communication channels over a network of potentially very large scale. As the previous section clarified, it is particularly user-friendly from the point of view of the application programmer also because of its automatic configuration capabilities: thanks to the Discovery Protocol, DDS participants and their related entities can automatically discover each other and connect, without any prior network configuration required. If all of this seems too good to be true, that is because it actually is. Coming back to the origins of the DDS specification, one can see how they date fairly back in time: the first drafts of the protocols were redacted in the late 90s, while the first officially published implementations are from the early 2000s. In the meantime, the world of computing has changed a lot, and situations that were only theorized before are currently the norm. A wide variety of new technologies, which are all somehow related to networking, have seen the light of day in the meantime. As an example, consider the advent and diffusion of various high-bandwidth wireless networks like WiFi or 5G, which then led to other advancements like cloud computing, Internet of Things (IoT) and edge computing. All these technologies are nowadays part of the daily life of many people, and they have all contributed to shape the current scenario in information technology: digital devices are everywhere, on every scale, all interconnected at the same time using a variety of technologies and protocols.

That is also what made distributed autonomous systems possible, but also changed the world into something much different than the one in which the DDS was born. When the OMG DDS standard was designed, networks were still made of cables and routers, with point-to-point links which finally allowed little to no packet loss, and connected devices which were different declinations of the same concept: a big computer. Nowadays, the term “computer” has a much broader meaning, and networks connect devices that operate on many different scales using multiple types of physical links. One of the few things that is common among all is that they all need to share large, if not huge, amounts of data among themselves. This is where the DDS starts to show its limitations: it was designed for a world where the network was a mostly reliable and predictable medium, and connected devices could be assumed to have a certain level of computational power and memory. This is not the case anymore, and the DDS is starting to show its age. The main limitations of the DDS are related to these exact contexts, and came up in the last five years due to its wide adoption in new contexts like that of autonomous robotics, where having multiple heterogeneous devices connected over a wireless, lossy^V network is the norm. The most limiting factor of the DDS is, in fact, its intrinsic reliance on a network layer which requires little to no retransmissions. If this assumption is not met, communications based on the DDS can become very slow or even fail completely due to network congestion. This is mainly due to the fact that the Discovery Protocol has not been designed with this situation in mind, so that it requires a number of multicast transmissions that grows exponentially with the number of participants in the network. When the network is not reliable, *e.g.*, in the case of WiFi, these transmissions can be lost leading to bursts of retransmissions that might clog the network altogether. A second important

^VIn this context, the adjective “lossy” denotes a packet switching network subject to *packet loss*, *i.e.*, the possibility that packets flying get dropped by the devices handling them because of network congestion or similar phenomena, in an attempt to preserve overall network functionality.

limitation of the DDS is its inability to optimize bandwidth usage and retransmissions when dealing with large data, *e.g.*, video streams, which obviously were not a concern back in the days in which the DDS standard was defined; this leads to even more risks of ending up with a congested, unusable network. Moreover, these problems become unsolvable if one attempts to route DDS traffic over a Wide Area Network (WAN), such as the Internet, whose routers and switching devices might apply traffic control policies that might throttle or even prevent DDS packets to travel safely; again, this scenario was not considered in the original specification, but has become of pivotal interest due to relatively new concepts such as IoT. Finally, let us note that all of the features and functionalities that the DDS offers come at the price of an overhead in terms of both resource usage and protocol complexity. A straightforward DDS implementation that has all the features specified in the standard typically ends up being quite a large piece of software, not so easy to adapt to resource-constrained environments like embedded systems, and a DDS data packet usually has many control fields that increase the amount of data being transmitted.

This discussion highlights how the DDS middleware is useful in some contexts, mainly those in which the network is fully reliable and ease of configuration is of the essence, but inapplicable in others. This is where a new communication middleware, recently born as a successor of the DDS to address the challenges of the modern, interconnected world, comes into play. This middleware is called *Zenoh* [21], which is the crasis of “zero network overhead”.

The design of the Zenoh protocol starts from the following consideration: an application programmer needs communication APIs to handle three main types of data, which are:

- **data in motion**, *i.e.*, data that is being transmitted over the network among

multiple devices and software modules, all active at the same time;

- **data at rest**, *i.e.*, data that is stored in a device and can be retrieved at any time by other devices;
- **data for computations**, *i.e.*, data that is used as input for computations requested from one network endpoint to some other(s) and that is produced and returned as their output.

The publish-subscribe pattern was invented to handle data in motion, which eventually led to the definition of the DDS among many other protocols and to the diffusion of distributed systems, and is still the best solution to apply in this case. The last two sets of data, from a communication perspective, can be handled in the same way by implementing an API that allows to *query* network endpoints, formalizing both the interrogation phase and the answer phase. Zenoh is defined as a Pub/Sub/-Query protocol, since it implements three fundamental kinds of objects: publishers, subscribers, and queryables. Each object, once instanced, can *declare* itself as one of the three kinds, then exchange data according to the operations permitted by its type. This is very important because in many contexts in which the DDS is applied today, *e.g.*, that of autonomous robotics, there is in fact the need for a query-based or client-server type of communication which is not defined in the protocol, thus leading to its re-implementation on top of it in any context and requiring at least two DDS topics and a control protocol to organize the transmissions; this contributes to the overheads and the issues mentioned above.

In Zenoh, each data point is organized as a pair of type *(key, value)*, where the value is the actual data and the key identifies the communication channel, or channels, which that value concerns. A key is typically an array of characters made of many substrings separated by the / delimiter, as in `home/kitchen/thermostat`; wildcards

and regular expressions are also allowed in API calls to match multiple keys at once. Note how the use of the delimiter naturally enforces a partition of the space of keys, and thus of the data flowing in the network, into multiple subdomains. This is a key feature of Zenoh as it allows to optimize many phases in which this topology is involved like, *e.g.*, the discovery of network endpoints: as opposed to the DDS in which all endpoints must know everything about each other to be able to exchange data, in Zenoh endpoints query for their fellows only when they must share data that has a specific key, and discovery-related transmissions are directed only to hosts and endpoints that handle that key, starting from its left-hand side going towards the right-hand one. This implies that when, *e.g.*, a new subscriber wants to receive data published with the `home/kitchen/thermostat` key, it will first query which endpoints handle the `home` subdomain, then ask them directly for endpoints handling the `kitchen` subdomain, and so on until the end of the key. This way, the number of transmissions required to discover and connect endpoints is effectively reduced, since these transmissions are triggered only when necessary and happen among only the hosts and applications that are actually involved. To achieve this huge optimization over the DDS, however, Zenoh abandons the automatic discovery feature: there is no such automatic discovery functionality in Zenoh, and all applications or hosts must be configured manually to know which other such entities are present in the network and how to reach them. Multiple network topologies are possible, as explained in [21] and illustrated in Figure 3.3:

- **clique**, where all endpoints are peers and can communicate with each other;
- **brokered**, where communications are routed among endpoints by a central router application, which is implemented as a daemon^{VI};

^{VI}A daemon is a software that runs in the background, typically started when the system is booted and running continuously until the system is shut down.

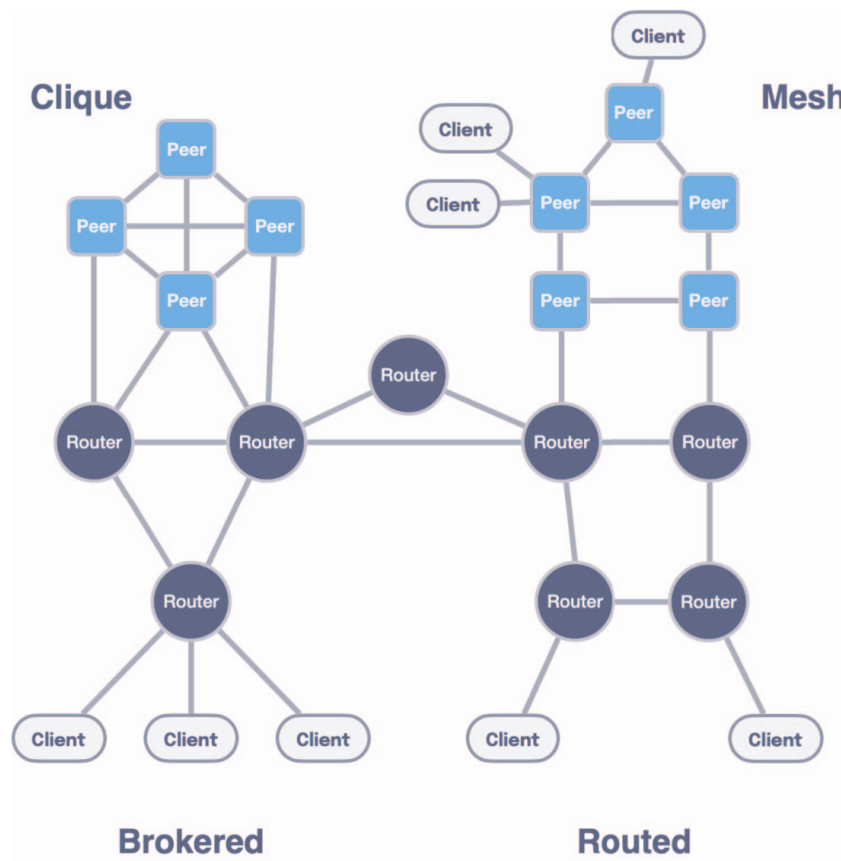


Figure 3.3: Possible network topologies in Zenoh [21].

- **routed**, in which many routers enable communication among multiple groups of hosts and applications in brokered mode;
- **mesh**, a hybrid situation in which endpoints may share partial network structure information or even route data among themselves.

This may seem a step back from the DDS specification but if one recalls that the Discovery Protocol is among the first causes of trouble when the DDS is used in relevant contexts, such as that of autonomous robotics, this has rightfully been deemed a reasonable price to pay to have a reliable and efficient communication.

Finally, another positive aspect of Zenoh is that it is implemented with the additional goal in mind of having the lowest possible resource and memory footprint, which

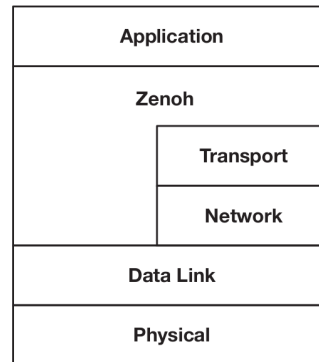


Figure 3.4: Placement of the Zenoh protocol in the network stack [21].

implies that it can run on a variety of different hardware platforms ranging from servers down to microcontrollers. For example, the control fields in its data packets can take up to as little as 5 bytes, as opposed to the at least 56 bytes required by the DDS. Moreover, it is almost completely agnostic of the underlying network layer, positioning itself in the protocol stack eventually as low as on top of the data link layer as showed in Figure 3.4, thus supporting a wide variety of protocols and physical links including UART, Bluetooth, LoRa, Unix Sockets, TCP/IP, UDP/IP, QUIC, WebSockets, and CANbus, offering the same set of APIs and functionalities in all cases. Thanks to this level of versatility, Zenoh is a very good candidate to be used in the development of distributed autonomous systems, especially in those contexts where the DDS is not applicable due to the reasons discussed above, and it has already been successfully employed in industrial contexts achieving good performance in terms of both throughput and latency [22, 23] despite being relatively new, as its version 1.0 has been released in October 2024^{VII}. For these reasons, it is included in the proposed architecture in its current state alongside some DDS implementations, to provide a complete set of communication tools that can be used in a variety of contexts, platforms, and applications.

^{VII}<https://github.com/eclipse-zenoh/zenoh/releases/tag/1.0.0>

3.1.3 Robot Operating System 2

The previous sections presented a class of software solutions whose objectives align with those that emerge during the design of a distributed system, and aimed at providing real-time communication services. With all these elements in place, the next solution to be described is a middleware specifically designed for robotics applications, which are a particular case of distributed systems, and that is also widely employed in the industry and in the open-source community. Since autonomous robotics is one of the application settings for which the proposed architecture has been developed and in which it has been tested and employed, this particular middleware is of paramount importance for the rest of this work and will be described in the following in due detail.

ROS 2 [24] is the second version of the *Robot Operating System* middleware [25], originally developed in 2007 by Willow Garage [26], a spin-off of the Stanford Artificial Intelligence Laboratory. Although its name might give a different idea, ROS is not an operating system in the traditional sense of process management and scheduling; rather, it started as a structured communications layer above the host operating systems of a heterogenous compute cluster. Then, ROS has evolved into a middleware that offers robotic and automation programmers services and APIs such as hardware abstraction, low-level device control, data exchange and inter-process communication, support for creating client-server modular architectures, and software package management. The entire project consists of a collection of software libraries and tools. It has always been open-source with a community-based development model and rigorous review processes: the source code of each version, ready for compilation and installation, is organized into various repositories hosted on GitHub, and documentation is published online alongside numerous *design documents* that explain and comment on the design choices that are made and on the new features that are



Figure 3.5: ROS 2 logo [25].

introduced. Fixes and innovations are tested in a *Rolling* [27] release and subsequently introduced in stable releases, which follow an alphabetical naming scheme similar to that adopted by Canonical for the Linux distribution *Ubuntu* [28] and are thus often referred to as “distributions”. Currently, the project is maintained and steered by the Open Source Robotics Foundation [29]. Its open nature led to a steady increase in popularity over the years, and has contributed immensely to a worldwide renewal of the interest in robotics thanks to code sharing and open scientific contributions, both in a theoretical and in a practical sense, favoring the birth of a large number of communities, research projects, frameworks, laboratories and startups all over the world, sparking in the world of robotics a revolution similar to that happened in the software engineering world in the 90s with the advent of open-source software. Today if, *e.g.*, a research team publishes a novel mapping algorithm for 3D LiDAR^{VIII} scans, it is very likely that the implementation of that algorithm will be made available as a ROS package, and that it will be tested and reviewed by the community, thus becoming a standard tool for all roboticists to use in their projects.

By its documentation, ROS 2 supports the Linux, Windows, and macOS operating systems, but the reality is that support levels are defined by three *tiers* that define

^{VIII}Light Detection And Ranging, a class of sensors that use laser beams to reconstruct the depth of a scene and get a spatial reconstruction of the environment that surrounds them.

support quality and frequency of updates. Currently, the only Tier 1 platform is Ubuntu Linux, in which a ROS 2 distribution can be installed either from binary packages or compiled from source; the latter installation technique can also be applied to different Linux distributions. As for Windows and especially macOS, since their closed nature with respect to Linux usually adds additional difficulties when developing robotic software, they are not so widely adopted and thus the community puts less effort into updating ROS for these platforms, also because of the lack of developers and testers that actually have these platforms available to develop on together with the expertise required to manage and integrate the whole ROS codebase.

For a summary of the history of ROS and an overview of its design goals, see [30]. From that document, it is evident that the project soon proved potentially suitable for complex use cases, such as distributed and real-time management of collaborative swarms of robots or the homogeneous integration of different types of hardware and communication interfaces. A more technical and detailed description of the new features of the version 2, which introduces many of the most popular features, can be found in [31].

Historically, the first service offered by ROS was inter-process communication: in ROS 1, that was implemented by custom protocols built on top of TCP and UDP. The most significant difference between ROS 2 and the first version, and currently its greatest strength, is the fact that instead of reimplementing such protocols it sits directly above a layer, denoted as the RMW or ROSMiddleWare layer, which houses support for various communication middleware implementations aimed at distributed systems, such as the DDS and Zenoh. The first communication middleware to be integrated with ROS was the DDS [32]: the entire middleware was indeed built on top of DDS, whose specific implementation is left to the user and is fully interchangeable, to

implement the basic inter-process communication and data exchange services. This means that all the benefits of the DDS mentioned earlier such as ease of use and interface definition, Quality of Service, transmission configuration through discovery, and serialization, are immediately available to robotic system developers, solving quite a few problems but introducing new ones as previously discussed. Developers can also take advantage of ROS's standard libraries and the vast open-source ecosystem of packages based on ROS that are publicly available, as well as contribute their own work to advance the community of engineers and developers who use it.

From its origins, each version of ROS supports the following two programming languages: C++ and Python. This choice has been motivated by the high diffusion of both on nearly every platform, as well as by their intrinsic complementary nature. C++ is the most used and recommended for performance-critical as well as production applications, while Python is more suitable for rapid prototyping and for the development of scripts that interact with the system, as well as for high-level software modules that have no strict performance requirement but whose development and maintenance may benefit from a higher-level language. Moreover, Python is the de-facto standard programming language in contexts like machine learning and artificial intelligence, both fields that are having a disruptive impact on robotics. The ROS 2 project has also introduced support for other languages, such as Rust, Java, and plain C, and has developed a set of tools to facilitate the integration of these languages with the middleware, thus making it easier to develop software for robots using the language that best suits the task at hand. This is a significant improvement over the first version, which was limited to C++ and Python, and it is a clear sign of the project's commitment to keeping up with the times and the needs of the community.

To provide an idea of the main services offered by this middleware relevant to this

work, the key features of ROS 2 will now be described following a top-down approach, eventually reviewing some auxiliary but very useful tools and frameworks; thus, many minor utilities are excluded for the sake of clarity but readers are referred to the technical documentation [33] and basic tutorials [34] for these aspects.

Build system

In the ROS ecosystem as in many similar contexts, software is naturally divided into units called *packages*. The idea is that each package constitutes a standalone software module, designed to fulfill a specific function within the system's overall architecture, potentially interacting with others at various levels and using specific services offered by the system itself through APIs or specialized hardware. Nevertheless, as has been highlighted several times, a robot presents a different scenario from those typical of software development and it is not uncommon for a developer to work on multiple packages simultaneously. Adding to this complexity is the fact that if different modules need to interact, their corresponding packages will inevitably be interdependent. Therefore, a common system is needed to simplify the creation of a *workspace* in which to develop, execute and test the software contained in multiple packages. The solution provided by ROS 2 consists of a universal build system applicable to any context where this middleware is employed and divided into two modules, with the reasons for this approach described in [35].

Initially, there is *colcon* [36]. It was originally intended to be a tool that automates and simplifies the compilation and installation of Python or C++ code packages via CMake^{IX} [37] or Setuptools [38] for use with ROS 2, but soon it became a more general project. Today, it is used even outside of the robotics world to manage large projects

^{IX}CMake [37] is a cross-platform free software for build automation, whose name stands for “cross-platform make.” It was developed to simplify the build process by providing a scripting language to easily automate it entirely even for complex projects.

and codebases: it is not uncommon to find software libraries and tools whose source code can be managed and built using colcon. ROS 2 relies on colcon to organize every aspect of a workspace, intended as a collection of packages either in source code or binary^X form. Essentially, colcon creates a workspace by defining some basic directories in the file system where both source code and build artifacts will reside, and then manages compilation and installation according to either automatic or specified rules resolving dependencies automatically, *i.e.*, ensuring that all external pieces of software that are required are actually present on the system and correctly configured. Each package, to be manageable by colcon, must include a *package manifest* in the form of an XML file^{XI} in which all the characteristics of a software module, such as its name or dependencies, are listed. Installation may result in the generation of an executable file or a shared library, and possibly also of symbolic links to a location within the workspace where builds are performed; this greatly simplifies matters when testing the software and compiling it multiple times, as it avoids having to modify the installation points each time, always pointing to the latest executable or configuration file. The many command-line options of colcon also allow interaction, if necessary, with lower-level tools used at various stages of compilation and build.

In addition to the typical challenges related to building interdependent packages, it is also necessary to address those concerning integration with preexisting libraries and interfaces that compose a ROS 2 installation. To address this and many other issues, the second module is used: *ament* [39]. It is a collection of Python scripts invoked automatically by colcon that complete the build process by resolving depen-

^XIn this context, the term *binary* identifies a way in which software can be shipped which does not include source code files, and is instead limited to compilation output in the form of either libraries to link against or executables to run.

^{XI}eXtensible Markup Language, based on a syntactic mechanism that allows the definition and control of the meaning of elements contained within a document or text. It is used in web pages or for collecting data in a text file so that it is also interpretable by a computer program: parsing can be easily performed using appropriate libraries.

dencies with other ROS 2 packages and ensure that paths, environment variables and scripts are correctly set up, so that the software can later be executed without issues and with a minimal set of commands being necessary. This system proves effective: when having multiple packages in a workspace, they can all be built together with a single command. Compare this ease of use and the integration of new solutions like CMake and Setuptools with, *e.g.*, the make-only build system of MARTe2 discussed in Section 2.3.6 and you may start to get an idea of the advantages of using ROS 2 in a complex software project for a distributed autonomous system.

Interfaces

In ROS 2, like in the DDS, it is possible to specify the format of the data packets exchanged among software modules, which are called *messages*, using special text files called *interface files* [40]. In these files, similarly to DDS IDL files, the structure of the packet is defined by specifying each field on a separate line by a name and a data type. Data types range from integers or floating-point numbers to strings, and even variable-length arrays. It is considered good practice to group these files into packages consisting solely of interface definitions, which are then built and installed using the same build system: the result in this case is a collection of Python classes, and C++ header files and shared libraries that define the types of the data packets as well as the serialization and deserialization operations, which can then be included and linked in other modules that need to exchange data in such formats. In practice, a ROS 2 developer never needs to concern themselves with anything related to format specification, serialization, transmission, reception, or deserialization: all of this is automatically managed by ROS 2 via its RMW layer.

Unlike a standard DDS, however, ROS 2 also allows other two communication primi-

tives, each with its own interface file format, which extend the communication services offered beyond the classic publish-subscribe paradigm. The communication primitives are:

- **Messages:** they are the most common and are used to exchange data packets among software modules in a publish-subscribe fashion; they are specified in `.msg` files. An illustrative scheme is provided in Figure 3.6, and the definition of the `Image` message from the `std_msgs` library is provided as an example in Listing 3.1.
- **Services:** they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateless* communication; they are specified in `.srv` files. Note that in this context, the term *stateless* identifies a kind of communication that has no intermediate states between its beginning and end: in a service call, a *client* requests a computation to the *server*, which performs it and returns the result to the client, without any further interaction between the two being possible and usually in a short time. This is useful for simple transmissions like quick parameter setting or data retrieval, but not for complex interactions that require multiple steps, long computation times, or many possible outcomes. An illustrative scheme is provided in Figure 3.7, and the definition of the `AddTwoInts` service, which allows to perform the addition between two integers and is widely employed in tutorials, is provided as an example in Listing 3.2: note how the two request and result messages are delimited in the file by the three dashes.
- **Actions:** they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateful* communication; they are specified in `.action` files. In this context, the term

stateful identifies a kind of communication that has intermediate states between its beginning and end: in an action call, a *client* requests a computation to the *server*, which performs it and returns the result to the client, but the server can also send feedback to the client during the computation, and the client may even request the cancellation of the ongoing operation to the server. Moreover, the server may reply to the client at the end of the operation sending back two items: the actual output value of the computation, if any, and various codes that describe nearly any possible outcome. This is the most flexible and powerful communication primitive offered by ROS 2, usually employed in software modules that perform complex operations like navigation or tracking, where the client needs to know the status of the computation and possibly to interact with the server during its execution. An illustrative scheme is provided in Figure 3.8, the state machine of an action goal is portrayed in Figure 3.9, and the definition of the `Fibonacci` action, which in many tutorials expresses the computation of the Fibonacci sequence up to a given order, is provided as an example in Listing 3.3: note how the three request, result, and feedback messages are again delimited in the file by the three dashes.

All of these are implemented on top of the RMW layer. In the case of DDS, they are all implemented using topics: a service maps to two topics, one for the request and one for the reply, while an action is implemented with two services: the *goal service* to place the request for a computation and the *result service* to request its output, and a *feedback topic*. In the case of other middleware that natively supports client-server communication, like Zenoh, the implementation is more straightforward and efficient, as the middleware itself manages the stateful communication and the client-server interaction, thus reducing the overhead and the complexity of the system.

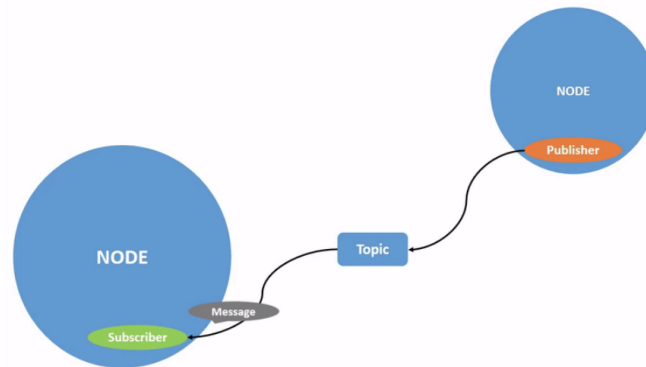


Figure 3.6: Scheme of the message communication primitive [41].

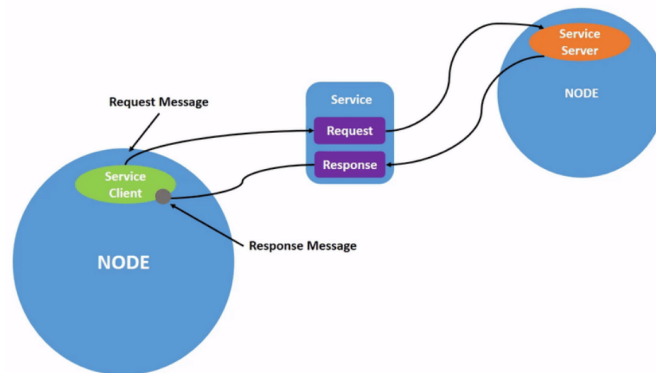


Figure 3.7: Scheme of the service communication primitive [42].

```

1 # This message contains an uncompressed image
2 # (0, 0) is at top-left corner of image
3
4 Header header # Header timestamp should be acquisition time of image
5               # Header frame_id should be optical frame of camera
6               # origin of frame should be optical center of camera
7               # +x should point to the right in the image
8               # +y should point down in the image
9               # +z should point into to plane of the image
10              # If the frame_id here and the frame_id of the CameraInfo
11              # message associated with the image conflict
12              # the behavior is undefined
13
14 uint32 height # image height, that is, number of rows
15 uint32 width  # image width, that is, number of columns
16
17 string encoding # Encoding of pixels -- channel meaning, ordering, size
18               # Taken from the list of strings in
19               # include/sensor_msgs/image_encodings.h
20
21 uint8 is_bigendian # Is this data big-endian?
22 uint32 step        # Full row length in bytes
23 uint8[] data       # Actual matrix data, size is (step * rows)

```

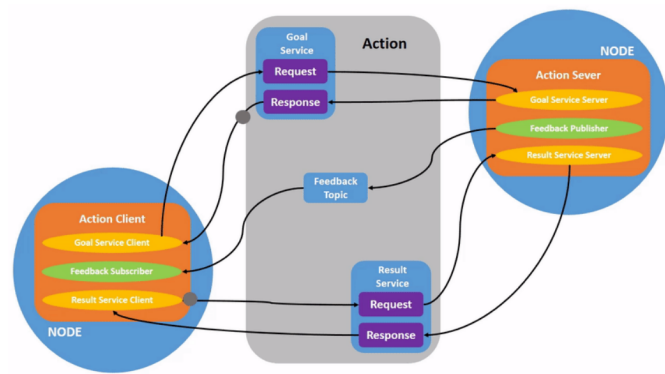


Figure 3.8: Scheme of the action communication primitive [43].

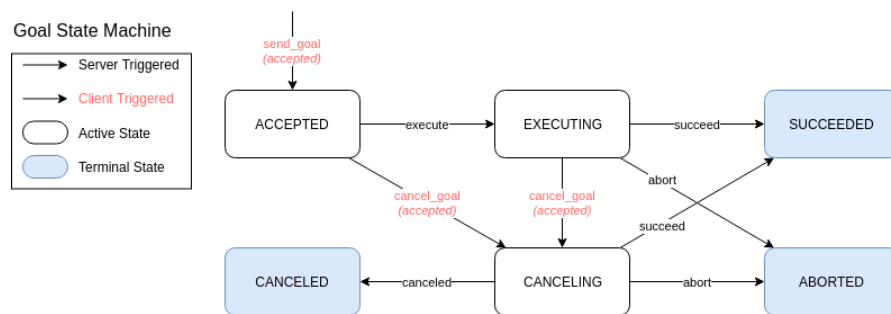


Figure 3.9: State machine of an action goal [44].

```

1 int64 a
2 int64 b
3 ---
4 int64 sum # = a + b

```

Listing 3.2: Definition of the example_interfaces/AddTwoInts service.

```

1 # Goal
2 int32 order
3 ---
4 # Result
5 int32[] sequence # Variable-length array of 32-bit integers
6 ---
7 # Feedback
8 int32[] sequence # Variable-length array of 32-bit integers

```

Listing 3.3: Definition of the example_interfaces/Fibonacci action.

Core API organization

To give an idea of how an architecture based on ROS for a distributed autonomous system may work, it is instrumental to present briefly the organization of the core API of ROS 2, which is shown in Figure 3.10. The core API is the set of libraries and tools that implement the basic functionalities offered by the ROS 2 middleware, intuitively starting from the communication layer and going up to the services offered to the application layer. Below the core API lies the communication middleware layer, which hosts the selected implementation(s) of the communication middleware(s) to employ; such implementations, when made available to the open-source community, are usually shipped together with ROS 2 base packages. This layer is represented in Figure 3.10 by the blue blocks at the bottom. On top of it all, there are code and applications that use the services offered by ROS 2 and everything else above them: these levels are represented by the top gray block in Figure 3.10. The core API, laying in the middle, is divided into several modules, each of which is responsible for a specific aspect of the ROS 2 middleware. The most important levels are, from top to bottom:

- **rmw level:** the `ROSMiddleWare` level hosts a set of C libraries that abstract different implementations of communication middlewares like, *e.g.*, many DDS implementations and Zenoh, offering a common interface to upper levels to achieve basic communication functionalities.
- **rcl level:** the `ROS Client Support Library` level hosts a C library that implements basic entities like *nodes*, discussed in Section 3.1.3, *publishers* and *subscribers*, service and action *clients* and *servers*, and *timers*.
- **ROS Client Library level:** at this level, various libraries are implemented to offer support for programming languages other than C, like Python and C++. The

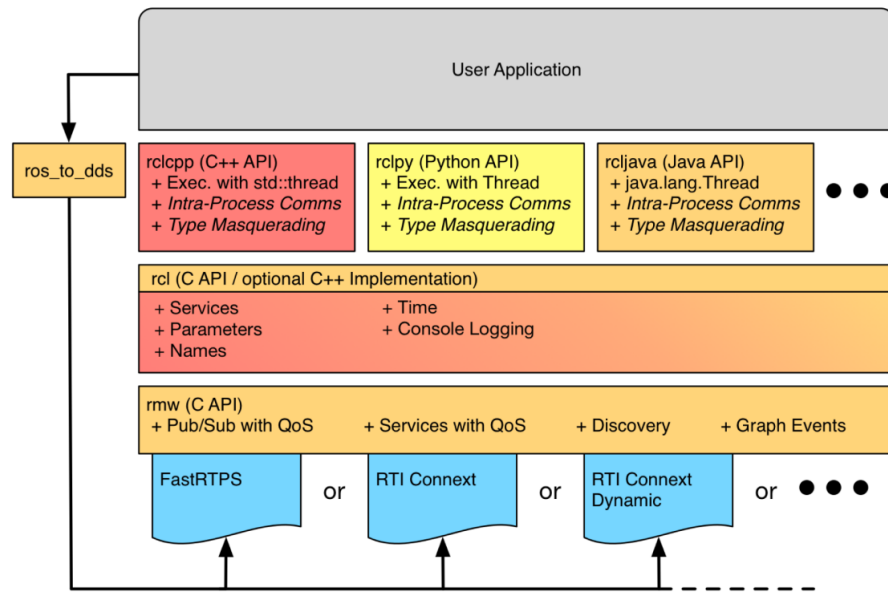


Figure 3.10: Core ROS 2 API organization [45].

entities defined in the lower levels are extended here, adding language-specific features and abstractions. Moreover, another core entity is defined at this level: the *executor*, which manages the execution of ROS-related jobs in dedicated OS threads, discussed in Section 3.1.3.

Nodes and executors

In ROS 2 applications, the most important entity is the *node*. Conceptually, a ROS 2 node is an operational unit of the distributed system, capable of performing a specific task for which it has been designed. In the code, regardless of the programming language employed, a ROS 2 node is an object of a class, that usually expands the base Node class of the specific client library used, adding features that are specific for the operations that the node has to perform. Primarily, and thanks to class members and methods they inherit, nodes act as access points towards the middleware that enable application code to use ROS 2 features. For example, a node can create publishers and subscribers to exchange messages, service servers or clients, and even action

servers and clients. Nodes have *parameters*, which are variables that can be set from the outside to configure the behavior of the node using a very versatile API: a node parameter is identified by a name and a type, and can be set at the startup of the node or at run time; in the context of an autonomous robot, a ROS 2 node parameter may be, *e.g.*, a gain for a control law being applied to the underlying physical system. Parameters can be queried and updated from both users and other software modules, *i.e.*, other nodes in the architecture, possibly deployed on different hosts in a distributed fashion.

The main idea is always that, to describe and design a distributed system, the node is a unit that performs computations that are aimed at performing a specific task, implementing a specific subsystem of the overall architecture. To achieve this, and to fully comply with the distributed paradigm even in real-time contexts, nodes generate computational *jobs* when specific events occur. Such events must be handled accordingly, in a way that is orchestrated by the middleware. The situation at hand is described in Figure 3.11, which shows the event-based execution model of ROS 2. In this model, aside from synchronous computations that are part of the user code, part of the processing is handled asynchronously when the following specific events occur:

- a subscriber receives one or more messages;
- a timer expires;
- a service request is received;
- an action goal or result request is received;
- a parameter is updated.

Apart from timers, the parameter subsystem is implemented using services which

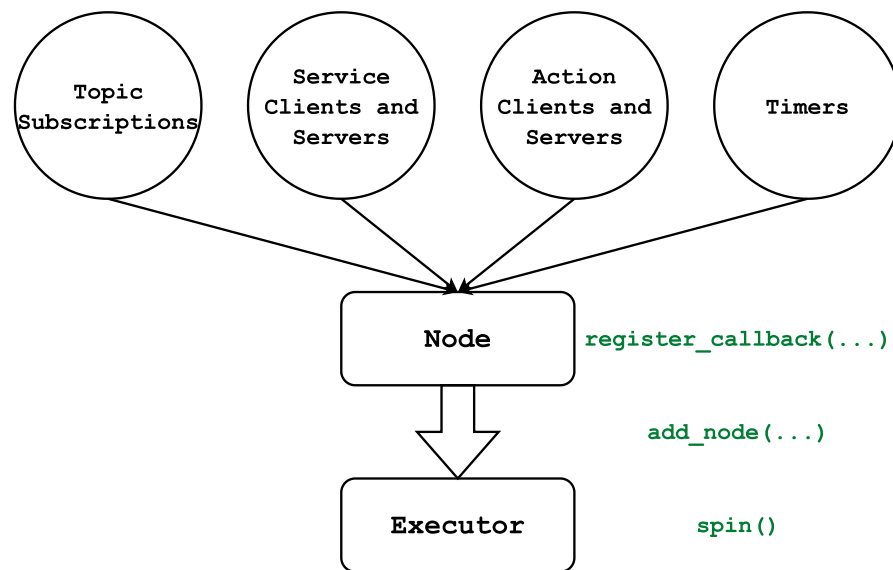


Figure 3.11: ROS 2 event-based execution model.

map to topics together with actions, but for the sake of clarity Figure 3.11 highlights the most important event groups. Each of those can be processed by a user-specified callback, that is registered as part of the node configuration. Then, nodes carrying their asynchronous workload are added to an executor, which handles callbacks in groups over one or more OS thread; node that executors can handle multiple nodes at the same time. To set things in motion, hence the name, in any ROS 2 application the executor is then started by calling its `spin` method.

Over the years, the real-world performance of this paradigm has been thoroughly evaluated [46, 47, 48, 49]. What emerged, especially in the early versions of the framework, is that although conceptually correct it was affected by some design flaws. First of all, while in its ideal contexts the DDS performs well with little to no real-time overheads, the RMW abstraction could introduce some unwanted latencies. Moreover, the ROS 2 executor, which is the entity that manages the execution of the asynchronous jobs, was not designed to be real-time capable: its algorithm lacks any notion of priority and it seems to favor timers over any other kind of event, to

empirically maximize responsiveness. If one thinks that commercial robots with ROS stacks are not only available but employed in the industry since almost a decade, it is clear that the ROS 2 executor is not a showstopper: in many contexts, things work fine because that level of strict real-time requirements is not needed, and as long as the system resources are not saturated both throughput and latencies remain acceptable even with such primitive algorithms. Nonetheless, the community has got the message: many solutions to these issues have been put in place and the core ROS 2 libraries listed in Section 3.1.3 are constantly updated and improved. The interested reader can find references to some of these solutions in [46], while a novel priority-based executor suited for real-time applications is presented in [50]. Finally, problems induced by the DDS, as discussed in Section 3.1.2, are addressed by the ongoing adoption of Zenoh as a communication middleware in ROS 2 [51], which is expected to be completed in the next few years. Another solution that deserves a mention is the *node composition* [45], which is similar to the way in which software modules are handled in the MARTe2 framework presented in Section 2.3.6. In ROS 2, C++ nodes can be compiled in two ways: either as parts of executables or as standalone shared libraries^{XII}, denoted as *components*. Components can either be loaded and unloaded at run time, inside a process that hosts executors for their jobs acting as a *component container*, or they can be statically linked to an executable. The thing is that, in either of these ways, multiple nodes end up running inside the same process, thus sharing the same memory and address space. This allows to avoid the overhead of inter-process communication, completely bypassing the RMW layer and anything underneath, leading to very low latencies and high throughput. This is a very powerful feature that can be used to implement complex systems in an efficient way, as demonstrated

^{XII}A shared library is a collection of compiled functions and routines that can be used by multiple programs simultaneously, allowing for code reuse, efficient memory usage, and simpler software updates by sharing a single copy of the library in memory at runtime.

in [45], provided that nodes are stable enough to run in the same process: in case one of them crashes, the whole process will be terminated, and all the other nodes will be stopped. This is a trade-off that must be carefully evaluated when designing a distributed system based on ROS 2, and the usual way of running each node in its own process remains the suggested one for the early stages of development and testing.

Launch system

A ROS 2 executable, or a group of such, must be launched using specific middleware commands to ensure that all necessary daemons and logging subsystems are activated. The middleware subsystem responsible for this is called the *launch system*, and the action of starting up multiple modules composing an architecture is called *launching*.

There are two ways to launch modules: either by using command-line tools to start each module individually, or by running scripts that specify the whole set of modules to run, together with their parameters and configuration options. These scripts are called *launch files*, and they are usually added as part of any package that contains executable code, and to any architecture once all the modules have been collected, to boot everything up with a single command. A launch file specifies, according to their format [52], the configuration with which the executable should be launched, including details ranging from input arguments to how logging should be managed, either in the console or in files, or both. Unless specified otherwise, any log files generated by the process output will be saved in a standard path, typically a subdirectory of the home directory (*e.g.*, `~/ .ros/` on Linux).

Data recording and playback

For various purposes, such as testing or post-analysis, it may be necessary to capture all the messages that were handled by the middleware during a specific time interval and on particular topics. ROS 2 provides this service through a system called *bagging* [53]: using specific commands, it is possible to record messages sent on certain topics into a file, with configurable format and compression algorithms. This file, which also includes information about the message interfaces it contains, can be processed by other software to review the data. Bags can also be replayed from the beginning using another command, which creates ad-hoc publishers that will re-publish the recorded messages at an original or user-defined rate. This way, it is possible to develop, test, debug, and even tune algorithms offline, without the need to perform an experiment with the actual robot each time, just by replaying the recorded data, avoiding potentially risky or costly operations. This feature is so popular that the vendors of many proprietary scientific software like, *e.g.*, MATLAB [2] or LabVIEW [54], have developed plugins to read and write ROS 2 bags, and many open-source tools have been developed to parse and process them, some of which are part of the ROS 2 ecosystem themselves.

Visualization and simulation

A complete installation of ROS 2 includes several useful tools for debugging, such as commands for inspecting active topics, examining their interfaces, measuring the publication rate, and launching publishers as needed. There are also software tools that are not required for the operation of a robot and, in fact, are superfluous in that context but extremely useful during the development phase. It is appropriate to mention at least the following:

- **Gazebo** [55]: this is an open-source simulator natively compatible with ROS, capable of interacting with middleware topics to send and receive data. It allows for the creation of 3D models of robots—complete with sensors and actuators—as well as the environments in which they operate, using XML-based syntax, and provides a platform to test control and perception algorithms. Gazebo offers a 3D visualization of the simulated world, and it can be used to simulate the behavior of a robot in a virtual environment thanks to its embedded real-time physics engine. Moreover, thanks to its open-source nature, Gazebo can be extended with plugins that add new features or modify existing ones, like implement simulated sensors and scenario elements, and it can be integrated with other software tools like MATLAB [2], Simulink [3], or LabVIEW [54].
- **RViz** [56]: this is a 3D visualization tool that can be used to display the state of a robot and its environment, as well as the data that is exchanged among the various modules of the system. It is a very useful tool for debugging and testing, as it allows to visualize the data that the robot is processing in a way that is easy to understand, and to interact with the system by sending commands and receiving feedback. RViz can be used to display the robot's position and orientation, the data from its sensors, the map of the environment, and the planned path, among other things. It can also be used to display the data exchanged among the various modules of the system, like the messages sent on the topics, the services called, and the actions performed. RViz can be used to visualize the data in 3D, 2D, or even in a tabular form, and it can be configured to display only the data that is relevant to the task at hand. As Gazebo, it is open-source and extensible with plugins that users can develop themselves.

An example of the capabilities of these two tools together is shown in Figure 3.12.

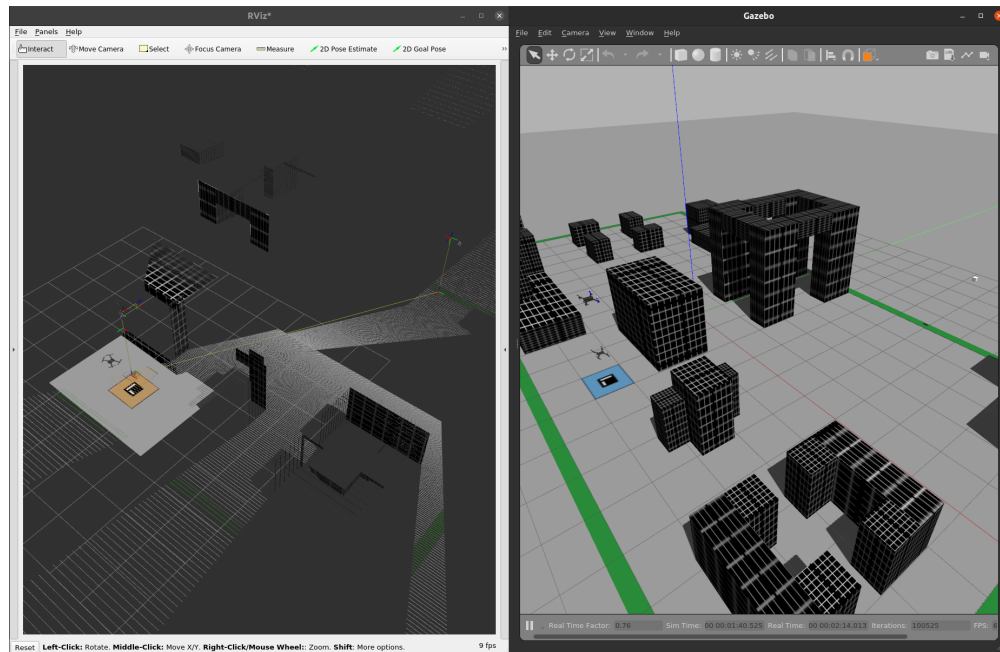


Figure 3.12: Simulation of an autonomous drone with imaging and depth sensors in Gazebo (right) and visualization of the data in RViz (left).

Applications and related frameworks

The previous sections highlighted the main features of the Robot Operating System 2 middleware, aimed at the development and organization of software for distributed autonomous robotic systems. The ROS 2 ecosystem is vast and varied, and it is used in many different application areas, from industrial automation to service robotics, from autonomous vehicles to drones, from medical robotics to space exploration. The ROS 2 middleware is also used in many research projects, both in academia and in industry, and it is the basis for many open-source software libraries and tools that are widely used in the robotics community. Its features, discussed so far, clearly defined its relevance in the context of the present work, and it will be included in the proposed architecture as illustrated in the following chapters. To conclude the presentation of ROS 2, some further references are in order to clarify its diffusion in both the research and industrial communities, as well as the variety of projects that are based on it.

One of the areas in which ROS has historically been employed the most is that of mobile, terrestrial robotics. In this field, there are a variety of problems to solve, ranging from localization to mapping, from path planning to obstacle avoidance, from perception to manipulation. One of the most famous projects related to this setting is Nav2 [57, 58, 59, 60, 61], the official ROS navigation stack for mobile robots. This package, originally developed to expose APIs for autonomous movement of mobile, planar robots, has evolved into a vast framework covering nearly every aspect of autonomous navigation: environment mapping and reconstruction, path planning and smoothing, obstacle avoidance, localization, and even human-robot interaction. A variety of algorithms for each case is available, and the system is designed to be modular and extensible in each of its parts, including visualization plugins for RViz and simulation elements for Gazebo. Operations carried out by the many modules of the framework are orchestrated by automata such as behavior trees [62].

Another area that robots revolutionized is that of industrial automation, in which manipulators of different sizes and shapes are employed to perform repetitive tasks with high precision and reliability. In this context, ROS 2 is used to control the robots and to interface them with other systems, like sensors and actuators, and to manage the data that is exchanged among them. One of the most famous projects in this field is MoveIt [63, 64, 65, 66], the official ROS manipulation stack for industrial robots. As Nav2 for mobile robotics, MoveIt is very large in terms of functionalities it offers, all of which are built on ROS 2 tools and libraries. First, it allows the developer to define a physical and kinematic model of the robot using the Unified Robot Description Format (URDF), which is a particular kind of XML file in which the geometrical and dynamic properties of a robot can be specified: masses, inertias, joints, links, and even sensors. Then, it provides extensible software modules to interface with the robot hardware, which is usually made of custom microcontrollers with their own

communication protocols and semantics. Finally, it allows the specification of states and movement operations thanks to many path planning and tracking algorithms, and it offers plugins for the RViz visualizer to turn it into a Graphical User Interface (GUI) for the robot.

One setting which is still relatively young is that of aerial robotics. Autonomous drones and aircraft can be used in a variety of applications thanks to their ability to fly, from surveillance to search and rescue, from agriculture to entertainment. The capabilities of the ROS 2 framework in this context are still being explored by the robotics community, particularly because a flying platform is more complex to control than a terrestrial one, and because the environment in which it operates is more dynamic and less predictable. Some preliminary results which led to the development of this work and can be considered its embryonic stage, are presented in [67, 68] and further discussed in Section 5.3.

Finally, it is important to assess the kind of hardware platforms on which ROS 2 has been designed to run. If one recalls the discussions held in the previous sections, it appears clear that the lowest tier of hardware on which ROS 2-based software can run is a Single-Board Computer (SBC), which is usually the highest grade of hardware that can be found on autonomous robots. However, as previously remarked, robots are complex systems that usually also include specialized hardware like microcontrollers to drive both sensors and actuators. It would be desirable to be able to organize the code for such devices in a similar way to that based on ROS 2, even leveraging similar communication paradigms and abstractions. This is where the micro-ROS project [69] comes into play. Developed by eProsima, one of the major developers of DDS implementations as well as one of the greatest contributors to the ROS 2 codebase, micro-ROS is a reduced software stack implemented mostly in C, which

offers a subset of the core ROS 2 functionalities with a minimal memory footprint and computational overhead. Its software stack, shown in Figure 3.13, is designed to run on a variety of Real-Time Operating Systems (RTOS) supporting their task scheduling and memory allocation APIs, and it is developed to preallocate or statically allocate memory to avoid dynamic memory allocation and deallocation, which is a common source of bugs and performance issues in real-time systems. The micro-ROS stack is designed to be used in conjunction with the full ROS 2 stack, exchanging data over topics in both directions thanks to a bridge put in place by the Micro XRCE-DDS [70] implementation, composed by an agent application running on a PC or SBC which talks with a client running on the microcontroller. Supported physical links include Ethernet, Wi-Fi, and even serial connections. As every solution presented up to this point, micro-ROS is open-source and its community is growing rapidly, as is the number of experimental evaluations, projects, and supported microcontrollers like, *e.g.*, the STM32 series from STMicroelectronics [71] or the ESP32 series from Espressif Systems [72].

3.2 Containerization

Recalling the Modularity (M) requirement, let one suppose that a software module requires external dependencies, *e.g.*, libraries or tools to be built and run. These external dependencies may not be made part of the module sources but have to be set up in the systems that host it. Setting up a development and execution environment addressing situations like this is common in this line of work, but usually also quite a complex task. Multiplying the complexities by the number of different hardware platforms that the software module has to run on, one may get a set of problems that are hard to manage. This is where the Abstraction (A) requirement comes into play: the

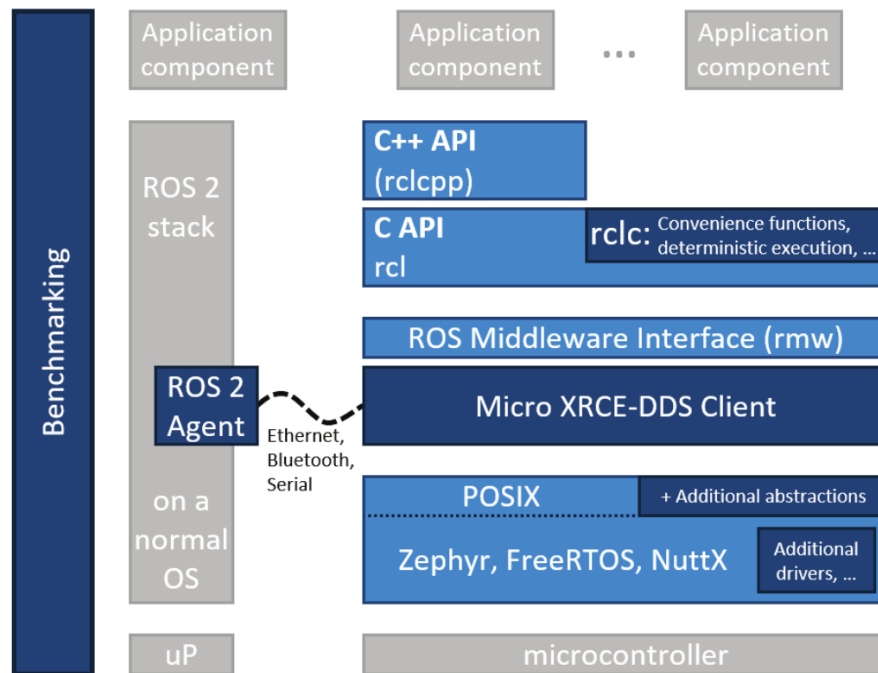


Figure 3.13: micro-ROS architecture [69].

architecture should provide a way to abstract the software modules from the hardware platforms they run on, to allow for the development of software that is portable across different platforms, and to simplify the setup of the development and execution environments. Historically, the technology that allowed this kind of abstraction is called *virtualization*, which in general comes at the price of a significant overhead in terms of computational resources and performance. Nonetheless, in Section 2.2, the Overhead (O) requirement was also stated, which requires the architecture to be as lightweight as possible, to avoid any unnecessary computational overheads that may affect the performance of the software modules. The technology that addresses all these requirements, forming a base layer of the proposed architecture, is called *containerization*.

Containerization has emerged as a disruptive paradigm in software deployment and infrastructure management, addressing a multitude of longstanding challenges asso-

ciated with portability, efficiency, and environmental consistency [73]. As software systems have grown increasingly complex, developers have faced considerable difficulties in ensuring that applications behave consistently across diverse machines and runtime environments. These challenges often arise from discrepancies in system configurations, dependencies, and library versions, giving rise to the notorious "it works on my machine" syndrome. Containerization effectively mitigates these issues by creating isolated environments that bundle an application with all of its necessary dependencies, ensuring consistent behavior across a variety of deployment contexts. This capability is of particular importance for modern, distributed software architectures, where consistency, rapid deployment, and scalability are essential components of operational success.

Containerization is often conflated with virtualization, but these technologies are intrinsically distinct in their architectures, use cases, and performance characteristics [74]. Virtualization relies on hypervisors to create virtual machines (VMs), each of which operates with its own guest operating system atop a host system. This approach provides a complete, independent virtual instance of a computer, encompassing its own operating system and resources, effectively emulating a full physical machine. However, the inclusion of a separate OS image for each VM introduces significant overhead, which can lead to suboptimal resource utilization—particularly when deploying numerous VMs on the same physical hardware. In contrast, containerization leverages the host operating system's kernel to create multiple isolated environments known as containers. Unlike virtual machines, containers share the host OS kernel while maintaining logical isolation at the process level. This design results in considerably less overhead, allowing for much higher resource density: many more containers can run concurrently on the same hardware compared to virtual machines. Consequently, containerization is preferable when the objective is to maximize resource efficiency

while maintaining isolated environments for different stages of development, testing, and deployment, without the need for complete OS-level isolation.

The origins of containerization can be traced back to early innovations in Unix-like operating systems. In the early 2000s, technologies such as FreeBSD Jails (2000) and Solaris Containers (2004) introduced the concept of partitioning a single operating system's resources into distinct, isolated instances. FreeBSD Jails enabled administrators to create secure, isolated environments capable of independently running services, while Solaris Containers provided an integrated method for managing multiple isolated application environments on a single instance of Solaris. Concurrently, the Linux ecosystem began to develop foundational features that would later form the bedrock of modern containerization technologies. Notably, Linux introduced kernel features such as cgroups (control groups) and namespaces—both essential to containerization's core principles of isolation and resource control. Linux namespaces provide process isolation, determining the visibility of system components like the filesystem, network interfaces, and other processes. In parallel, cgroups regulate the allocation of hardware resources—such as CPU, memory, and I/O—ensuring that containers do not monopolize or starve system resources, thus enabling fair resource sharing and predictable performance.

The introduction of these kernel-level features marked a pivotal advancement, laying the groundwork for the subsequent evolution of container management tools. Linux's cgroups and namespaces offered the first glimpse of what would eventually become a full-fledged container ecosystem, allowing developers to create environments where applications could operate independently while sharing the underlying system's resources efficiently. The development of these technologies represented a crucial step in enabling the practical implementation of containerization at scale, as it provided

the required process isolation and resource control that would later underpin all modern *container engines*.

Containerization's primary appeal lies in its inherent efficiency and flexibility. Unlike traditional virtual machines, containers do not incur the overhead of hosting multiple operating systems, leading to significantly faster start times, reduced resource consumption, and efficient utilization of hardware capabilities. These properties make containers ideal for microservices architectures, which decompose applications into smaller, independently deployable components. Containers offer an effective mechanism to encapsulate each microservice along with its respective dependencies, ensuring consistent operation regardless of the underlying infrastructure. This isolation also helps mitigate compatibility issues, allowing different microservices to use conflicting versions of dependencies without interference. The modularity offered by containerization enhances maintainability and promotes scalability, as individual microservices can be scaled independently according to demand.

The distinction between virtualization and containerization also has significant implications for operational strategies. Virtualization remains relevant in scenarios requiring full OS-level isolation, such as when multiple tenants need to be isolated from each other for security or compliance purposes. Virtual machines are effectively self-contained environments, providing an additional layer of abstraction that can be crucial in maintaining strict separation between different workloads. This makes virtualization an optimal choice for situations demanding rigid isolation or for supporting legacy applications that require their own dedicated operating environments. However, in scenarios where such complete isolation is not mandatory, containerization offers substantial advantages, including improved scalability, faster deployment times, and more efficient resource use. Containers are inherently more lightweight than VMs,

making them ideal for scenarios demanding rapid provisioning, horizontal scaling, and resource optimization. As a result, containerization has become the preferred solution for many use cases in software development, particularly those involving cloud-native applications, microservices, and environments requiring rapid scaling and adaptability.

Containerization represents a paradigm shift in how software is developed, deployed, and managed. By offering a lightweight, resource-efficient alternative to traditional virtualization, containerization maximizes resource utilization while maintaining isolated environments for reliable application execution. Although virtualization still plays a role in certain contexts, particularly where complete isolation is required, containerization has emerged as the favored approach for a wide range of use cases, offering unparalleled efficiency, portability, and flexibility. The progression of container technology highlights its critical role in supporting modern software infrastructure. It allows developers to achieve consistent and streamlined application deployment across any environment, driving the efficiency and scalability needed to meet the demands of contemporary software systems and distributed computing architectures.

In the context of this work, containerization can effectively help to build a common architectural ground that abstracts the software modules from the hardware platforms they run on, at the same time packaging with them every external and third-party dependency they require to be built and run. This way, the software modules can be developed and tested in a controlled environment, and then deployed on any hardware platform that supports the containerization technology, a condition that is not restrictive since today container engines are supported on nearly every commercial platform that the proposed solution may target. For this reason, modules and projects based on the proposed architecture will be managed through containers: the technical

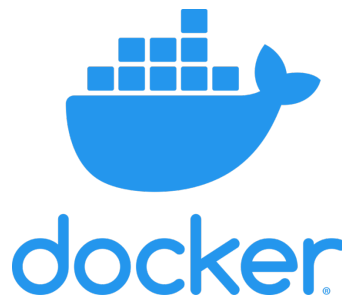


Figure 3.14: Docker logo [75].

details of this process are postponed to the next chapters, where the architecture will be described in detail. To conclude this dissertation, the chosen containerization technology is illustrated in the following section.

3.2.1 Docker Engine

The release of Docker [75] in 2013 was a watershed moment in the evolution and widespread adoption of containerization [76, 77]. Docker unified the previously fragmented ecosystem of container technologies, incorporating existing Linux kernel features into a cohesive, user-friendly tool that dramatically simplified the process of container creation, deployment, and management. Docker's innovation lay in its ability to abstract the underlying complexity of containerization, providing a standardized interface and image format that made the technology accessible to a broader audience of developers and IT professionals. This accessibility facilitated a shift in how software was developed, tested, and deployed. Developers could now package their applications into lightweight, portable containers, ensuring consistent behavior across different environments, from local development machines to large-scale cloud deployments. Docker also contributed significantly to the establishment of container orchestration systems, which allow organizations to manage large numbers of containers seamlessly across clusters of machines.

The impact of Docker's introduction cannot be overstated; it altered the software development lifecycle, enabling teams to deploy applications with unprecedented speed and consistency. By providing developers with an easy-to-use toolset, Docker democratized the use of container technology, which had previously been accessible only to those with in-depth systems expertise. Docker images standardized the definition of application environments, reducing the configuration drift that plagued software deployment efforts. Docker Hub [78], an associated public *registry* of container *images*, further accelerated adoption by providing a central repository from which developers could share and reuse container images, promoting community collaboration and reducing redundancy. Moreover, since its birth as a Linux-only tool, Docker has evolved to support also the Windows and macOS operating systems through its Docker Desktop release [79], although with a limited set of features available on these platforms, which also require a thin virtualization layer to work. This revolution has also impacted the research world outside of the cloud computing and software development industries, as it has been adopted in many scientific fields to manage the software tools and libraries that are used to analyze data and run simulations, as well as to distribute the results of the research and novel algorithms and procedures in a way that ensures reproducibility, transparency, and accessibility [80].

The first key component of Docker's utility is the Dockerfile, which serves as the blueprint for creating Docker images. A Dockerfile is a simple text file that contains a series of instructions on how to assemble a particular Docker image. These instructions include the base image to use, any software dependencies to be installed, environmental variables, and commands to be executed when building the image. When a Dockerfile is processed by the Docker engine, it creates a Docker image, which is a read-only template containing the application and its environment. Docker images can then be instantiated as running containers. Containers, therefore, are the

execution units created from Docker images, providing a live instance of the encapsulated application. This workflow is at the core of Docker's portability and consistency, ensuring that the entire environment is replicated across different systems with high fidelity.

Another fundamental feature of Docker is its use of union filesystems [81], which are specialized filesystems that merge multiple layers into a single coherent view. Union filesystems are particularly useful in containerization as they allow multiple layers to be stacked, where each layer represents a set of *filesystem changes*. This approach enables efficient storage as new layers can be added or modified without altering the underlying base layers, thereby conserving disk space and allowing for rapid image creation and reuse. Specifically, Docker employs OverlayFS by default to create image layers [82]. Docker images are built in layers, with each layer representing a set of filesystem changes such as installing a package or adding a configuration file. The union filesystem allows Docker to stack these layers on top of each other, presenting them as a single coherent filesystem. This approach provides significant efficiencies in both storage and resource usage, as multiple images can share the same underlying layers. For example, if multiple images are based on the same base operating system, Docker stores only one copy of that base layer. OverlayFS, for example, enables efficient management of these layers by allowing new changes to be made in an uppermost layer without altering the lower layers, enforcing a copy-on-write mechanism^{XIII}. This layered approach not only conserves disk space but also accelerates the building of new images by reusing existing layers.

Docker Compose [83] is another powerful tool within the Docker ecosystem that facilitates the management of multi-container applications. With Docker Compose,

^{XIII}In this context, *copy-on-write* denotes a paradigm according to which stored data is shared among multiple entities keeping a single copy, until one of such entities must modify it; in this case, data is actually copied and a new instance is assigned to such entity that can now modify it.

developers can define and run multi-container Docker applications using a YAML^{XIV} file to configure the services, networks, and volumes required by the application. This capability is particularly useful for orchestrating the deployment of complex applications that require multiple components—such as databases, backend services, and front-end web servers—to work together seamlessly. Docker Compose allows these components to be defined as individual services, each in its own container, and specifies how they interact. This simplifies the process of setting up development environments and ensures that the entire application stack can be easily replicated and deployed.

Another crucial feature of Docker are *volumes* and *volume mounts*, which are of paramount importance for data persistence. Containers are ephemeral by nature: when a container is stopped or deleted, all data within it is lost. To overcome this limitation, Docker uses volumes, which are directories or files stored outside the container's writable layer. Volumes can be mounted into one or more containers, allowing data to persist even if a container is removed or updated. This capability is essential for stateful applications, such as databases, which require durable storage that outlives the individual container instances. By using volume mounts, developers can also share data between containers, enabling efficient communication and data exchange between different parts of an application or of a distributed software architecture.

Networking is another key area where Docker provides significant capabilities, enabling containers to communicate with each other and with the outside world. The networking model of Docker includes several types of networks hereby only simply mentioned such as *bridge*, *host*, and *overlay*, each suited to different use cases. Bridge networks are the default and are typically used to enable communication between containers running on the same host. Host networking, on the other hand,

^{XIV}Yet Another Markup Language, a popular markup language for configuration and text files.

allows a container to share the network stack of the host, providing greater network performance and simplifying access to services running on the host. This mode is particularly useful in scenarios where the containerized service needs to avoid the overhead of NAT^{XV} and requires direct access to the network interfaces available on the host.

Docker also provides mechanisms to control the resources allocated to containers, such as CPU and RAM using, *e.g.*, cgroups on Linux. Resource limiting is critical to ensure that no single container can consume disproportionate system resources, which could negatively affect other containers running on the same host. Docker allows users to specify limits on the CPU shares and memory usage for each container, ensuring predictable performance and efficient utilization of system resources. By configuring these limits, administrators can create a balanced environment where all containers receive their fair share of resources, preventing resource contention and ensuring stable operation. This capability extends, through additional tools, to the management of additional dedicated processing hardware like GPUs^{XVI}, which can be shared among containers in a controlled way.

Finally, Docker can use the Linux capabilities subsystem to manage hardware and system call access in a granular manner. Instead of giving containers full root privileges, which could pose security risks, Docker allows specific capabilities to be granted as needed. For instance, a container may need the capability to bind to low-numbered network ports or change the system time, but it does not need broader privileges that could further compromise the host. By selectively enabling only the necessary capabilities, Docker minimizes the attack surface of containers, enhancing security while maintaining the required functionality for each application.

^{XV}Network Address Translation.

^{XVI}Graphics Processing Unit, specialized high-power processors originally designed for videogames and now commonly used for parallel workloads in many industrial and scientific applications.

This section presented and described the Docker Engine containerization solution alongside many of its features. The choice of the functionalities to discuss here was not coincidental: each of the features described is essential to the architecture, to fully employ Docker as the solution that addresses the Abstraction (A) requirement. Let one bear in mind also that, thanks to the features outlined so far, applications that run inside Docker containers may do so within a simple partition of the host operating system, thus with no additional overhead with respect to, *e.g.*, applications that run on the same system but outside containers. Some configuration work on the host is required to achieve this, as the next chapters illustrate, but once everything works this constitutes also a solution to the Overhead (O) requirement. The Docker Engine is used in the present work to build and manage hardware and system abstraction layers that establish a common environment for both the development and the execution of software modules and projects among many different platforms. The following chapters explain their implementation in detail, showing how the architecture is built on top of these technologies to satisfy all the requirements stated in Section 2.2.

3.3 Version control

Let one recall the Modularity (M) and the Development (D) requirements. In essence, the architecture must be modular in the sense that it must allow for the development of software modules that can be easily maintained as standalone units, but at the same time it should provide a straightforward integration process when a larger-scale project, like a prototype, requires to set up many of these modules at the same time. Furthermore, when working in any of these two situations, the architecture should provide easy means of maintaining the single units composing a larger project and of tracking the changes made to them while working on the project itself. Moreover, it

should be possible to issue updates to these modules and propagate them to other projects that integrate the same modules, in case they do not require a specific version of them. The technology that addresses these requirements is called *version control*.

Version control [84] is an essential element of contemporary software design and development, serving as a systematic mechanism for managing changes to source code, documentation, and other artifacts associated with a project over time. Version control involves the practice of tracking and managing modifications to software artifacts, thereby allowing developers to maintain a comprehensive history of changes, collaborate efficiently, and ensure consistent software quality. Version control systems (VCSs) enable concurrent development by multiple contributors without conflicts, thus mitigating issues such as overwritten code and conflicting updates. Furthermore, VCS facilitate the exploration of new features or bug fixes through branching, which isolates development efforts into independent streams, and merging, which integrates these branches into a cohesive codebase. This adaptability is crucial for modern software teams, who must concurrently manage multiple development lines, such as feature development, bug resolution, and experimental changes.

The criticality of version control in software development is profound. It provides detailed tracking of every individual change made by team members, which facilitates an understanding of software evolution and captures the rationale behind each modification. This meticulous tracking becomes invaluable during debugging, as developers can analyze the precise changes that contributed to a malfunction and revert to a previous stable version if necessary. In large-scale software projects, where multiple developers are working simultaneously on a variety of features, a clear history of changes helps ensure accountability and transparency, both of which are vital for quality assurance and effective collaboration. The ability to revert changes or establish

a timeline of modifications ensures that teams can efficiently correct mistakes without sacrificing valuable work or disrupting ongoing development processes. Ultimately, version control transcends the role of a mere tool for tracking changes—it is a foundational practice that underlies efficient, collaborative, and high-quality software development.

For these reasons, it is clear that the adoption of a version control system as a basic element of the architecture would provide the means to satisfy the (M) and (D) requirements: as for the Modularity (M) requirement, the version control system would allow to maintain the software ranging from modules to larger projects as standalone repositories under version control, eventually linked among each other. Furthermore, thanks to the branching and merging capabilities of the version control system, the development process would be streamlined, and the integration of the modules into a larger project would be facilitated. The version control system would also provide the means to track the changes made to the software, to issue updates to the modules, and to propagate them to other projects that integrate them, thus satisfying the Development (D) requirement. The inner workings of these processes are described in the following section, which introduces the VCS of choice for the proposed architecture.

3.3.1 Git

Git [85] is one of the most influential version control systems in contemporary software development [86, 87], recognized for its robustness, efficiency, and distributed architecture. Its inception dates back to 2005, when Linus Torvalds, the creator of the Linux operating system [88], sought an effective and flexible version control system for managing the Linux kernel's development. At that time, the Linux kernel project was experiencing significant growth, with thousands of developers from around the globe



Figure 3.15: Git logo [85].

contributing code. Initially, the project relied on a proprietary version control system called BitKeeper, which, although effective, became unsuitable for the open-source nature of the kernel due to licensing conflicts. This prompted Torvalds to develop Git, a system specifically tailored to meet the unique requirements of the kernel project, particularly distributed collaboration, speed, and support for non-linear workflows. There is no official explanation of its name, but some well-known theories suggest that it may be a play on the word “get” or some underground slang term in an unidentified language meaning “simple” or “stupid”, also since Torvalds himself refers to it as *the stupid content tracker* [85].

Linus Torvalds designed Git with several essential requirements in mind, derived from the practical challenges faced during the Linux kernel’s rapid and distributed development. First, it needed to accommodate a distributed development model, whereby each developer could maintain a complete copy of the entire codebase, including its full history. This decentralized approach provided resilience, ensuring that developers were not dependent on a central repository and could work independently without risking the loss of progress. The distributed nature also meant that individual contributors could experiment freely, with the assurance that any inadvertent changes would not compromise the project’s integrity. The requirement for speed was equally paramount, as the system needed to handle the scale of contributions typical of the Linux project efficiently and support quick retrieval of project states. Furthermore, Torvalds emphasized the importance of strong support for non-linear

development workflows, a feature critical for managing the extensive branching, merging, and contributions that characterize an open-source project of such magnitude and diversity.

The distributed architecture of Git is one of its defining strengths, distinguishing it from centralized version control systems [89]. In a centralized model, all developers rely on a single repository, which serves as the central point of authority for the project. In contrast, Git's distributed model allows each contributor to maintain a complete local clone of the repository, which includes the full history of all changes. This design not only provides a safety net, as the loss of a single repository does not impact the project's integrity, but also enhances productivity by allowing developers to work offline and synchronize changes when network access becomes available. In open-source projects like the Linux kernel, where developers are distributed across different geographies and time zones, the distributed model provides autonomy, enabling contributors to make progress independently while maintaining synchronization with the broader team. The distributed architecture also facilitates peer-to-peer collaboration, allowing developers to exchange changes directly without relying on a central authority, thereby promoting a more resilient and scalable development workflow.

Git's architecture and features make it a uniquely powerful and flexible version control system, which has revolutionized how teams approach software development. One significant advantage of Git is its use of snapshots rather than deltas. Unlike earlier version control systems, which typically stored differences between successive versions (deltas), Git creates a snapshot of the complete project state at each commit. This design choice ensures that the project's state can be easily and efficiently retrieved at any point in history, and it enhances data integrity through the use of cryptographic

hashing. Each snapshot is uniquely identified using a hash function, which not only guarantees the integrity of the entire repository but also makes the history robust against corruption. This snapshot model forms the basis for the entire Git system, making retrieval, comparison, and merging of different states exceptionally efficient.

Git's branching and merging capabilities are also notable strengths, allowing developers to create branches with minimal overhead and merge them seamlessly when ready. Branches in Git are lightweight, which encourages experimentation without destabilizing the main codebase. This capability is particularly valuable for projects that require multiple, concurrent lines of development, such as new feature development, bug resolution, or exploratory work. The ability to create branches easily means that developers can work on different features or tasks simultaneously without fear of conflicting changes, which is vital in large-scale collaborative environments. Git's merging tools are designed to handle complex merging scenarios effectively, providing features such as conflict markers and automatic merging, which significantly reduce the complexity and risks associated with combining different branches of development.

Another advanced feature that Git offers, particularly beneficial in large and modular projects, is the concept of submodules. Submodules allow a Git repository to include and track the content of other repositories, effectively enabling complex projects to integrate external dependencies while maintaining version control over them. Submodules are indispensable in managing dependencies, particularly when different components of a larger system are developed independently. By incorporating submodules, developers can ensure that specific versions of external libraries or components are consistently tracked, which guarantees compatibility and stability across different versions of the main project. This is especially useful when projects

rely on third-party libraries that need to remain stable across multiple updates, preventing unforeseen compatibility issues that may arise from updates that are not synchronized.

Submodules also play a key role in enabling modular development practices. Large projects often comprise multiple components that are logically separate but need to be version-controlled together. Submodules provide a means to maintain these components as distinct units, each with its own repository, while ensuring that they can be brought together in a cohesive manner within the parent project. This modularity is invaluable when different teams are responsible for different components, as each team can work independently on their respective modules, while the main project maintains coherence and consistency. Submodules thus foster a more manageable and scalable development process, particularly for projects that involve multiple dependencies with distinct development cycles. By leveraging submodules, teams can maintain a balance between modular autonomy and integrated consistency, ensuring that all parts of the system work harmoniously together while evolving independently.

The use of submodules extends beyond dependency management to foster more effective collaboration across different teams. When multiple teams work on separate but interrelated components, submodules provide an effective mechanism for versioning each component while maintaining an overarching structure. Each submodule is linked to a particular commit in its respective repository, which allows the main project to keep track of stable versions of dependencies. If updates are needed, the submodule can be updated to point to a new commit, ensuring that any changes are deliberate and controlled. This controlled mechanism ensures that all updates are tested before being integrated, which prevents disruptions to the main project's stability. Furthermore, submodules make it easier to reuse code across different projects,

as they provide a clear and organized way to incorporate existing repositories without duplicating code, thereby fostering a more efficient and reusable development ecosystem.

A similar and complementary feature to submodules is that of subtrees. Subtrees and submodules are both mechanisms for managing dependencies in a Git repository, but they offer distinct workflows and advantages. Subtrees allow developers to embed the content of an external repository into a subdirectory of the main repository. Unlike submodules, subtrees do not require additional configuration or extra repositories to be manually initialized when cloning the main repository. This makes subtrees more straightforward for new collaborators, as the embedded content becomes a natural part of the repository's history and structure, without any extra setup steps.

On the other hand, submodules are more suited for maintaining clear separations between distinct modules or components that need to evolve independently while remaining linked to a specific version in the main project. Submodules provide a finer level of control and facilitate updates across multiple repositories, which is particularly useful in larger projects where different teams are responsible for maintaining different components. Thus, while subtrees offer simplicity and convenience by integrating dependencies directly into the repository, submodules provide flexibility and modularity, enabling a cleaner separation of concerns.

These foundational features have led to Git becoming the de facto standard for version control in the software industry, trusted for both small-scale projects and large, highly distributed software systems. Git's flexibility, efficiency, and robust support for collaborative development have established it as an indispensable tool for modern software engineering, facilitating complex workflows and enabling distributed teams to innovate with confidence. Its influence extends beyond individual projects, shaping

the entire software development landscape by enabling more effective collaboration, version management, and deployment practices. As software systems grow increasingly complex and teams become more distributed, the role of Git in managing the intricacies of software evolution remains pivotal, making it an essential element of any serious software development endeavor, as the one that this work proposes. To this end, the design of the architecture starts from the organization of every work it supports as a Git repository, as illustrated in the following chapters. To fully satisfy the requirements (M) and (D), the architecture will also include support for both submodules and subtrees, as they are both useful in different contexts and for different purposes. While submodules are suited to allow for the combined integration and development of single software units into larger projects, they require a more complex setup and management, which is not always necessary. Subtrees, on the other hand, are more straightforward to use and are more suited in scenarios in which it is not expected that active development on the embedded content will be carried out, but rather that it will be used as a dependency and eventually only updated to newer versions. The architecture will be designed to support both mechanisms, allowing for the best of both worlds to be exploited.

4 The proposed Distributed Unified Architecture

The aim of this chapter is to present the core contribution of this work. The introductory chapters defined the problem setting, outlining a situation in which distributed autonomous systems are becoming ubiquitous in many aspects of everyday life, but at the same time increasingly complex to develop and manage. From the researcher's and the developer's perspectives, a versatile framework to work on such systems is needed, but once some specific yet broad requirements are stated one finds that the current state of the art does not provide suitable solutions. Still, if the trends from the last decade in software engineering and computer science are examined, one can find that many tools have been developed that address specific portions of the original problem.

Recalling once again the leading principle of this entire work, we are about to build a new type of wheel without starting completely from scratch: we have some good tools ready at hand, but most importantly some pieces are already available, whereas some others are still missing. Before we begin, however, it is important to further clarify the actual setting in which this work has been developed, in particular with respect to the Abstraction (A) requirement from Section 2.2 and to supported operating systems: the scope of the design process has been to build a framework that

can be adapted to many target platforms but naturally, due to availability and time constraints, the actual implementation at the time of writing has been tested on a limited yet representative set of them. This process has been driven by the need to provide a proof of concept for the framework, and at the same time to demonstrate its feasibility and effectiveness in real-world scenarios over the years in which this work has been carried out. Nonetheless, the chosen set of target platforms is broad enough that the resulting framework works on all of them and can be adapted, with minimal effort and specific changes, to other platforms as well, basically repeating the same setup process that has been followed for the current set. Furthermore, as clarified in the following, other assumptions stand regarding the configuration of the host operating system: for simplicity, and to benefit from the existing solutions presented previously, the framework has been designed to work on systems that support the solutions described in Sections 3.2 and 3.3. Having defined this starting point, the framework follows by integrating what is ready and then developing what is missing, to first provide a system abstraction layer as a solid foundation and then to build an architectural template that can be used to develop and deploy modules and software for distributed autonomous systems satisfying the original requirements. Additional tools can subsequently be developed and integrated to further enhance the framework, addressing issues that are specific to some use cases like, *e.g.*, the ones described in subsequent chapters of this work.

The organization of this chapter reflects this description, which is nothing but the recollection of the design and development process that led to the *Distributed Unified Architecture* (DUA). After the discussion of the implementation of the framework, some concluding remarks are provided also with respect to the other solutions presented in Section 2.3.

4.1 Assumptions

Before delving into the details of the system abstraction layer offered by DUA, it is instrumental to describe the host system configuration that it expects. Recalling what has been said at the beginning of this chapter, the following brief set of assumptions holds mainly for the sake of simplicity: satisfying them allows to employ solutions that are readily available, represent current industry standards in their respective fields, and help to solve significant portions of the problem at hand. What follows from them constitutes a working proof of concept, but nothing prevents the framework from being adapted to different configurations should any requirement change, or should the need arise to support different host system configurations. For now, what is provided in this work satisfies the initial requirements, and any expansion or adaptation is left as future work.

Moreover, for clarity, the following definition is in order here: the term *system configuration*, in this context, refers to a combination of hardware and software that identifies a specific class of systems; each class shall offer and support a defined set of functionalities, hardware components, and operating systems. When a *host* system configuration is mentioned, it refers to a system configuration on which Docker is also present and on which containers are run, *i.e.*, not the one that would be found inside containers built and managed by DUA.

Hardware platform type The framework is designed to work on systems that range from single-board computers to more powerful general-purpose computers or even servers, of any architecture, provided that they support the solutions described in Sections 3.2 and 3.3. This assumption excludes lower-end systems like, *e.g.*, microcontrollers, which do play important roles in the field of

autonomous systems. This choice has been motivated by two facts: first, microcontrollers and similar systems usually have very limited and constrained sets of resources, which make them incompatible with the software solutions required as of now to build and manage the proposed framework. Second, the variety of microcontroller architectures available today is so vast and each of them is so different from the others, also in terms of operating system support on platforms where this is available, that extending the proposed approach to this class of systems would require a significant effort. This is not to say that the proposed framework cannot be adapted to work with microcontrollers, but that is left as future work since their integration would certainly require a dedicated design process.

Support for version control The host system must support a version control system, as described in Section 3.3. This is a very important requirement, as the framework relies on version control to manage the software modules that are developed and deployed on the components of the distributed system. The choice of Git as the version control system is motivated by its many features that make it highly efficient when working with projects that embed many modules, as described in Section 3.3.1.

Support for containerization The host system must support containerization, as described in Section 3.2. This is another very important requirement, as the framework relies on containerization to isolate and manage software components. The choice of Docker as the containerization solution is motivated by its widespread adoption and the functionalities that it offers, enabling it to build replicable environments on a multitude of platforms, as described in Section 3.2.1.

Operating system support The framework is designed to natively work on the Linux operating system, as it is the most widely used operating system in the field of distributed autonomous systems. This choice is motivated by the fact that Linux, in its many distributions, offers the widest support for hardware devices and software tools. Moreover, it is built to be developed onto, being natively open-source and offering a stable and fully equipped environment right from the start for the development of software of any kind, especially that which is intended to directly access any hardware resource connected to the host machine. Furthermore, Linux is the operating system that best supports both version control and containerization solutions, whose importance in this project has been highlighted in the previous points. For these same reasons, when a SBC is released to the market, it happens frequently that its natively supported operating system is a Linux distribution, so to offer a consistent environment across all platforms supported by DUA it is natural to assume that the host configurations share this level of similarity. Among the many possible Linux distributions to choose from this work does not impose any specific one, but Ubuntu Linux in one of its latest Long-Term Support (LTS) versions available at the time of writing^I has been employed for the development and testing of the framework, and is used as the reference for the system abstraction layer described later on. Again, this choice is motivated by similar reasons: Ubuntu is the most popular and easy-to-use Linux distribution, and it is widely supported by many software tools and packages as well as by many SBC vendors. As in any other such case, the use of a different distribution is possible provided that one manages to obtain a working environment with Git, Docker, and any other software dependency that they might require. To conclude, let us note that this

^I22.04, 24.04.

assumption is not a strict requirement since the framework can be adapted to work also on Windows and macOS operating systems thanks to Docker, but it is recommended to use a Linux distribution since some features that are crucial for this work are not supported on those OSes due to their different support for containerization which, as explained in Section 3.2.1, always involves a layer of virtualization. Such features will be presented and described in the following sections together with their role in the proposed framework but for now, since they mainly concern containerization and map to specific Docker functionalities, a summary table is provided in Table 4.1 in which each feature appears with the name of the corresponding Docker functionality. For completeness, note that while macOS offers a good support to containers running a Linux-based image thanks to Docker Desktop, the best way to run them on Windows is currently to use Docker Desktop in conjunction with the Windows Subsystem for Linux (WSL) [90].

	Linux	Windows	macOS
Volume mounts	✓	✓	✓
Host networking	✓	×	×
Graphical applications	✓	✓	×
Hardware devices	✓	×	×
Capabilities	✓	×	×
IPC namespaces	✓	×	×

Table 4.1: Docker features relevant for the DUA framework and their support on different operating systems.

4.2 System abstraction layer: dua-foundation

The most important problem solved by DUA is also the first that it addresses: providing a similar environment across multiple system configurations. Achieving this is not a trivial task: the hardware platforms and operating systems that are of interest are very

different from each other; yet, the aim is to provide an environment that offers the same OS APIs, libraries, and tools across all supported platforms. Nonetheless, DUA operates within the limits of what is possible without unnecessarily increasing any overheads: some aspects of the underlying hardware are not abstracted on purpose since it is not necessary to provide a consistent experience, but instead would imply introducing significant overheads in the execution of any application. This is the case of, *e.g.*, the hardware architecture: as illustrated in the following, currently supported platforms include solutions based on both the x86-64 and ARM64 architectures, and the framework is designed to work on both of them without hiding them or their differences under an abstraction layer. This is due to multiple factors: first, there is no practical necessity to do so, and secondly, the developer using the framework can benefit of the peculiarities and specific features of each platform on which they want to deploy their software. Last but not least, abstracting the hardware architecture would imply using hardware emulators such as QEMU [91], which are very powerful but also introduce significant overhead in the execution of any application, they are not always available on any platform, and they complicate access to system resources such as hardware devices which are instead of paramount importance in the practical contexts of this work, as shown in Table 4.1.

This first part of DUA is called `dua-foundation` [92], and is in essence a collection of Dockerfiles. The incipient intuition is that environments defined by a set of OS APIs and software tools can be easily replicated and managed in the form of Docker images, as described in Section 3.2.1. Building a Docker image tailored to every supported platform would allow to account for the specific characteristics of each one, providing a consistent system configuration inside containers that use such images. Each researcher or developer, *i.e.*, each *user* of the DUA framework, would then have to add their own software dependencies and packages to work on their

specific modules. Let us call a user-developed software module based on DUA, a *unit*. To define the system configuration that a unit requires, the Docker image is still the best tool available for the same reasons. For the sake of consistency along the whole chain, Docker images related to units would have to start from the same set of base images as explained in Section 3.2.1, which are the ones built from the Dockerfiles found in `dua-foundation`; the specific one is chosen depending on the hardware platform to work on. Let us then refer to images built from `dua-foundation` as to *base units*.

The way a base unit is created differs even significantly for each, but the setup process they follow is the same for all. They all start from a platform-specific base image available on public registries like Docker Hub. Then, this image is enriched by a set of system packages which represent common dependencies in this field like, *e.g.*, compilation toolchains. Finally, some more libraries are installed that are specific to the currently evaluated application areas of DUA: even if they may not be required by all units, they are included in the base units due to the frequency of their use in real-world scenarios, *i.e.*, it would subsequently be very likely to find different units that require them. At the end of the build process, each image is pushed on Docker Hub [93] to be available to all users of the framework.

To conclude, note that each supported hardware platform has its own base unit mainly to adapt the abstraction layer to the specific characteristics of the platform, but also to leverage and expose to the user the specific features of the platform itself. In fact, the aim of DUA base units is to provide *similar* but not *equivalent* environments: it is clarified in the following, when the currently supported platforms are presented, that some have specific resources that others lack. The framework is designed to work on all of them, but it is also designed to take advantage of the specific features of each

one, and to expose those to the user in a consistent way. This is the reason why the setup process for each base unit is made of the same steps but performed differently each time, and why the resulting images are different from each other. It is then up to the user to develop their units on top of the base units that offer the best environment and features for their specific needs, which could be all or only some depending on the specific requirements of the software contained in the unit.

The next sections explain the contents of `dua-foundation`: first, the set of software dependencies to be installed in every base unit is discussed, to describe what is the final system configuration achieved by every base unit. Then, currently supported hardware platforms are briefly illustrated, to show the variety of hardware that DUA integrates but also provide a first hint of the differences that the framework deals with. Finally, currently available base units are explained in more detail, together with the setup process that they follow.

4.2.1 Common software dependencies

Providing here a full list of common software dependencies that are provided in every base unit would be a daunting task, as it would include a vast number of packages and libraries. The interested reader can directly inspect the Dockerfiles in [92]. However, to give a general idea of the environment that base units provide, as well as to facilitate the inspection of their Dockerfiles, a summary of the most important software dependencies is provided here.

Each base unit starts from a set of system packages, that is, software packages that can be installed using the package manager of the Linux distribution that the base unit is built on, and provide basic functionalities. Since the distribution of choice for every base image is Ubuntu Linux, the package manager is `apt`; the choice of Ubuntu as

first layer of every base unit is motivated by the same reasons given in Section 4.1 and by the fact that an image containing a basic Ubuntu environment is relatively small in size, currently around 30 MB [94]. The first set of installed packages includes a GCC compilation toolchain [95], a Python system environment [96], a Java development kit [97], shell utilities, and a set of libraries that are required in the subsequent steps and that may depend on the specific base unit, as explained later on. These are installed now, all in a single command, to reduce the number of layers in the Docker image and optimize its size, a well-established practice when building Docker images.

Next comes the middleware that is specific to the current main application areas of DUA, which are also subjects of this work and discussed in the next chapters. Usually, the first one to be installed among those is the ROS 2 middleware, presented in Section 3.1.3. The amount of ROS 2 packages installed depends on the target platform of a base unit: if the target is a development PC, a wide variety of packages is installed including simulators and visualizers, whereas if the target is a SBC, a reduced set of packages is installed skipping, *e.g.*, visualization tools. Alongside a ROS 2 installation, DUA base units include both the eProsima Fast DDS [7] and the Eclipse Cyclone DDS [98] implementations of the DDS middleware presented in Section 3.1.1, as well as the Zenoh middleware illustrated in Section 3.1.2. This way, considering the intended application to distributed autonomous systems and the specific applications of robotics and real-time control architectures for nuclear fusion that are presented in this work, DUA base units offer to each user a complete set of tools to develop their software modules and build a distributed architecture.

Then is the time for the OpenCV computer vision library [99, 100] and the Eigen linear algebra library [101]. The former is currently the de-facto standard in open-source computer vision, originally developed by Intel in the late 90s and then expanded

by a vast team of developers, it now offers a wide range of functionalities and the implementation of thousands of algorithms ranging from image processing to mathematical operators. The latter is a C++ template library for linear algebra, which is widely used in the field of robotics and computer vision for its efficiency and ease of use. Finally, the Gazebo simulator presented in Section 3.1.3 is installed in base units for development systems.

The setup of a base unit concludes with specific tools that vary from time to time, depending on the evolution of application scenarios and supported platforms. Among these, the most important is currently the `dua-utils` collection, which is part of DUA and is illustrated in [TODO dua-utils sec. ref.](#). Their position at the end of the setup process is intended: these are the last stages of each Dockerfile contained in [92], hence they correspond to the last layers in the image. This is done to allow to easily change these stages in case some tools are not needed or some new ones are to be added, without having to rebuild the whole image from scratch but only the last layers. This is another common practice in Docker image building, made possible by the ability of Docker to *cache* image layers among subsequent builds, and it is used here to optimize the time required to build a base unit.

4.2.2 Currently supported hardware platforms

To develop a framework that could be applicable to a wide variety of practical cases, a good variety of hardware platforms has been considered for the initial implementation. The goal of this section is to present them in due detail, to show the variety of hardware that DUA integrates but also provide a first hint of the differences that the framework deals with. The categorization of the platforms made here is based on their system resources and intended use: it is not a strict classification, but a way to group them to

better understand the differences among them. Note that, for simplicity, only 64-bit architectures are currently supported by DUA, as they have become the standard in the last decade and are now common in the microprocessor market. A real example is illustrated for almost all categories: such example items have also been used to test the framework and implement real autonomous systems prototypes during the development of this work. At the end of this section, Table 4.2 summarizes the most important system specifications of the example items, while some of the platforms that are supported by DUA, either fully or with some limitations as discussed above, are illustrated in Figure 4.1.

General-purpose PCs

In the context of this work, this is the simplest category to address and also the most common. The first base units, as well as the other pieces of the DUA framework, were put together and tested on this kind of platforms to ensure that the main concepts could work. By *general-purpose PC*, we mean a computer that is not specifically designed for a particular task, but that can be used for a wide variety of applications. This category includes desktop computers, laptops, and servers. In this context, it is assumed that among this category of systems, the ones that are of interest for this work are based on either the x86-64 or the ARM64 hardware architectures. In the practical and applicative contexts relevant to the DUA framework, these systems are used for development and testing purposes, as well as for running simulations and visualizations, or intensive real-time computations. The software tools that are used on these systems are the same as the ones that are used on the other platforms, but the hardware resources available on them are usually more powerful: many CPU cores, and much RAM and storage space are generally available. Dedicated coprocessors like, *e.g.*, GPUs may also be available on these platforms.

Single-board computers

In the field of autonomous systems, and especially in robotics, SBCs are very common. They are used in many applications, from small robots to drones, and they are also used in many research projects as fast prototyping platforms. The main reason for their popularity is that they are very cheap, small, and easy to use and integrate in larger systems, also thanks to their multiple communication interfaces. They are also very versatile, as they can be used for many different tasks. In the context of this work, SBCs are intended to be used to run the software modules that make an autonomous system perform its tasks. The hardware resources available here are generally more limited than the previous case, and GPUs and similar dedicated solutions may also not be available. This is also what makes SBCs so versatile in this field: their variety allows the system designer to choose the best solution that fits all the requirements, from software support, to mechanical constraints and energy consumption. In this work and for the explanatory purposes of this section, three classes of single-board computers are considered, making broad distinctions based on their hardware architecture and intended use.

The first class consists of single-board computers based on the ARM64 architecture. They are probably the most common and versatile type of SBC, due to some renowned features of the ARM architectures. First, ARM architectures usually have a very low power consumption, which makes them ideal for battery-powered applications. Second, their open design allows for a wide variety of implementations, which makes them very versatile and has paved the way for some very small but powerful implementations, perfect for SBCs and embedded systems. Last, ARM architectures have a very tidy instruction set, which contributes to both their low power consumption and high efficiency. The most famous ARM64 SBC is probably the Raspberry Pi series

[102], and DUA has been tested on the model 5.

The second class involves SBCs based on the x86-64 architecture. These are generally less common and more expensive than their ARM64 counterparts, since x86 processors are usually more powerful and have more features. They are usually employed for applications where more computational power is needed, or where the software that is going to be run on them is already available for x86 architectures. With respect to ARM-based SBCs, x86 ones usually have higher single-core frequencies and more cache memory, but at the price of higher power consumption. The selection of communication interfaces is also usually wider, and the support for peripherals is usually more complete, but each item on the board requires more power to operate. Moreover, the x86 architecture has been expanded over the years with many specific instruction sets, such as vector instruction sets like MMX, SSE, and AVX, which are very useful in many applications that require fast computations over large datasets. The most famous x86-64 SBC is probably the UP series [103], and DUA has been tested on the UP Squared and Xtreme models.

The last class is made by Nvidia Jetson SBCs [104]. These are ARM64-based solutions for edge computing on robotic systems, which also include a GPU, all integrated inside a Tegra SoC^{II}. The GPU is the most important feature of these SBCs, since it enables a wide variety of applications that require fast parallel computations ranging from image processing to object recognition and artificial intelligence models [105]. Many models and series have been released over the years, with models within a series being distinguished by their different set of system resources and price, so that customers can choose the solution that best suits their needs. The host system can be easily set up using the JetPack distribution by Nvidia [106] which includes an OS installation

^{II}System-on-Chip, a single chip that integrates all the components of a computer.

based on Ubuntu, device drivers, and many Nvidia toolkits and SDKs^{III}. DUA has been tested on the Jetson Nano, TX2, Xavier NX, Xavier AGX, Orin NX, and Orin AGX models; as illustrated by this list, the employment of DUA on Jetson spanned over four series and six different models, showing the versatility of the framework. Different series, however, require different base units, due to both the host system configuration and the base image that is used to build the base unit: the setup process for each of them is described in the following. Note that, as shown in the following section, a specific JetPack version is used in every Jetson base unit image: to ensure compatibility with the hardware and software tools that are available on the Jetson platform, the JetPack version is chosen to be the one that is recommended by Nvidia for the specific model that the base unit is built for, and has to be the same that is actually installed on the host Jetson system.

4.2.3 Base units

Section 4.2.1 and Section 4.2.2 have presented the concept of base units, which are the building blocks of the DUA framework. They are Docker images that provide a consistent system configuration across all supported platforms, preserving the specific characteristics and features of the host platform wherever possible. Software modules, *i.e.*, units built with DUA are then developed within containers whose images are built on top of these base units. Section 4.2.2 has illustrated the currently supported hardware platforms, giving a general overview of their variety. It is now time to complete the description of the base units contained in `dua-foundation` by illustrating the setup process that each one follows in more detail, as well as the specific features that they offer to the user. Keep in mind that the description of each base unit can now be more detailed but not so long, since much of the setup process

^{III}Software Development Kits.

SBC	CPU	RAM	GPU
Raspberry Pi 5	4c/4t Cortex-A76 @ 2.4 GHz	8 GB	VideoCore VII
Up²	4c/4t Pentium N4200 @ 2.5 GHz	8 GB	Intel HD Graphics
Up Xtreme	4c/8t Core i7-8665UE @ 4.4 GHz	16 GB	Intel UHD Graphics
Jetson Nano	4c/4t Cortex-A57 @ 1.43 GHz	4 GB	NVIDIA Maxwell 128 CUDA cores
Jetson TX2	4c/4t Cortex-A57 + 2c/2t NVIDIA Denver 2 @ 1.95 GHz	8 GB	NVIDIA Pascal 256 CUDA cores
Jetson Xavier NX	6c/6t NVIDIA Carmel ARM64 v8.2 @ 1.9 GHz	16 GB	NVIDIA Volta 384 CUDA cores
Jetson Xavier AGX	8c/8t NVIDIA Carmel ARM64 v8.2 @ 2.2 GHz	64 GB	NVIDIA Volta 512 CUDA cores
Jetson Orin NX	8c/8t Cortex-A78AE ARM64 v8.2 @ 2.0 GHz	16 GB	NVIDIA Ampere 1024 CUDA cores
Jetson Orin AGX	12c/12t Cortex-A78AE ARM64 v8.2 @ 2.2 GHz	64 GB	NVIDIA Ampere 2048 CUDA cores

Table 4.2: Summary of the technical specifications of some of the single-board computers currently supported by DUA.

is common to all of them and has already been described in Section 4.2.1. Moreover, some base units are obtained by extending others, so the setup process for the former is a subset of the latter; this is the case of, *e.g.*, base units for x86-64 systems and generic ARM64 systems, which can start with a configuration that suits a SBC, and then be evolved into a configuration that suits a development PC by adding more software tools, as anticipated in Section 4.2.1. One last thing to note is that, even if these particular base units could be built on top of each other by, *e.g.*, obtaining the x86-64 development image starting from the x86-64 SBC image as described in Section 3.2.1, this feature has not been employed and each base unit is built separately.



Figure 4.1: Some of the platforms currently supported by DUA, either with full support or with some limitations as in Table 4.1; from top left to bottom right: a Dell XPS 13 laptop, a Microsoft Surface tablet, a MacBook Pro, an Nvidia Jetson Xavier AGX mounted on a mobile robot prototype, an Up Squared, an Nvidia Jetson Nano, an Nvidia Jetson TX2 development kit, a Raspberry Pi, and an Nvidia Jetson Xavier NX.

This decision has been taken considering each build step that is performed in every such base unit, which led to this conclusion: installing some development tools for, *e.g.*, ROS 2 or similar middleware after their core implementation has been installed in the SBC portion of the image would force Docker to go back to work on layers that have been already built, which would slow down the build process and complicate the management of the image layers. Instead, building the base units separately allows to install the right set of packages in the right order and in single build steps, optimizing the image layers.

Before discussing each base unit in detail, a few words can be spent on the final sizes of the corresponding Docker images. Table 4.3 offers a summary of the sizes once the layers are optimized, compressed, and stored on Docker Hub. As one can see, images can be quite large, reaching up to 10 GB for the most complex development ones, with the ones for the SBCs being smaller and the ones for the Jetson systems being in the middle. The size of the images is not a concern for the framework, since they are not meant to be used directly by the user but only as a base for the development of software modules: as explained in Section 3.2.1, even if multiple DUA images are built on a same host system, Docker will reuse layers from the base unit since they are common to all images used on a host system. Regarding the build steps, the Dockerfiles in [92] show that care has been taken to reduce the size of the layers as much as possible by, *e.g.*, installing packages using the minimum number of commands that is possible, and removing temporary files and build artifacts. The final size of the images is then a compromise between the need to provide a complete environment to the user and the need to keep the images as small as possible to reduce the time required to build them and the space required to store them. Development images are larger than the others because they include more tools, many of which have graphical elements, and Jetson ones are also the largest among those dedicated to SBCs since they include many specific libraries by Nvidia and third parties, as discussed below. Recall that the aim of this framework, in its current version and state, is to provide a complete, easy- and ready-to-use environment for research, development, and prototyping purposes: the size of the images as well of the amount of software tools that are installed in them are a consequence of this aim, in order to provide a complete, generic, and versatile environment.

x86-base

The x86-base base unit is the simplest one, and it is intended to be used on SBCs based on the x86-64 architecture, as well as to deploy modules on general-purpose PCs when there is no need for additional development tools. It has also been the first one to be developed to test the framework. It starts from the official Ubuntu base image, and installs a set of system packages that are common to all base units, as described in Section 4.2.1. Since there is no need for additional development tools, the setup process ends here.

x86-dev

The x86-dev base unit is intended to be used on general-purpose PCs, and it is the one that is used to develop software modules that are intended to run on the x86-64 architecture. It starts from the same build stages of the x86-base base unit, adding in some of them development tools that are required to build software modules that are based on the ROS 2 middleware. Afterwards, the Gazebo simulator is also installed. Since this base unit is intended for a simple host platform, the setup process ends here.

x86-cudev

This is where things start to get interesting, since this is the first image that accounts for peculiar characteristics of the host platform as well as for the specific features of the base image that is used to build the base unit. The x86-cudev base unit is intended to be used on general-purpose PCs based on the x86-64 architecture and that include an Nvidia GPU. They require a specific set of device drivers and libraries to be installed on the host system, as well as a specific Docker runtime to give them

access to Nvidia GPUs on the host. The main difference is the base image: instead of a plain Ubuntu image, the `x86-cudev` base unit starts from the Nvidia CUDA+cuDNN base image, which is a Docker image based on Ubuntu that includes a CUDA Toolkit [107] and the cuDNN library and development files [108]. This is done to allow the user to develop software modules that can take advantage of the GPU on the host system, as well as many libraries that offer APIs ranging from parallel computations to machine learning. After the Nvidia ones, other libraries and tools aimed at machine learning and artificial intelligence are also installed, such as PyTorch [109], Keras [110], ONNX [111], TensorFlow [112], and Ultralytics YOLO [113]. The installation of these and other Python libraries, here as well as in all the other base units, takes place inside a Python environment specifically created for DUA and that extends the system environment. The rest of the setup process is the same as the one for the `x86-dev` base unit, with the addition of some compilation options for libraries built from source such as, *e.g.*, OpenCV, to enable GPU support.

`armv8-base`

The `armv8-base` base unit is the simplest one for ARM64-based SBCs, and it is intended to be used on them when there is no need for additional development tools. It starts from the official Ubuntu base image, and installs a set of system packages that are common to all base units as described in Section 4.2.1. The main difference of this base unit with respect to the `x86-base` one is that the build processes of libraries built from source such as, again, OpenCV are configured to take advantage of the ARM64 hardware architecture. This is another simple base unit so the setup process ends here.

armv8-dev

The `armv8-dev` base unit, like its `x86-dev` counterpart, is intended to be used on general-purpose PCs running on ARM processors such as, *e.g.*, Apple MacBook laptops with the recent Silicon series of SoCs. It starts from the same build stages of the `armv8-base` base unit, adding in some of them development tools that are required to build software modules that are based on the ROS 2 middleware. Afterwards, the Gazebo simulator is also installed. Source builds are again configured to take advantage of the ARM64 hardware architecture.

jetsonnano

This is the first of the series of base units aimed at Jetson systems. Since they are all SBCs not particularly suited for on-board development, all the `jetson` base units starting from this one do not include development tools. The Jetson Nano is today an old platform, and its last compatible JetPack version is 4.6.2. At that time, Nvidia did not provide a base image for Docker, so the `jetsonnano` base unit starts from the official Ubuntu 20.04 base image and performs a series of steps that transform it into a JetPack installation corresponding to that version. The choice of Ubuntu 20.04 is motivated by the need to maintain compatibility with the Linux kernel version running on the Nano, which is 4.9 since JetPack 4.6.2 is based on Ubuntu 18.04. A JetPack installation is obtained by configuring some environment variables and configuration files, and adding Nvidia repositories to `apt` from which Nvidia packages are then downloaded. Then the setup process proceeds mostly as in the `armv8-base` base unit, with the addition of some Nvidia-specific packages and libraries that are required to run GPU-accelerated software on the Jetson and the source build of ROS 2. In this image and the next `jetson` ones, building ROS 2 from source is required to keep its

version consistent with the other base units: to use the most recent ROS 2 version available, in x86 base units which are based on Ubuntu 22.04, the ROS 2 Debian packages are used to install the Humble Hawksbill version, while in jetson base units which are based on Ubuntu 20.04 the only way to install the same version is to build it from source. This is not a trivial task at all and requires many dependencies, build options and paths to be set correctly, but it is the only way to ensure that the same version of ROS 2 is used across all base units, which is a key requirement to ensure consistency of the environment provided by DUA.

jetsontx2

The jetsontx2 base unit is intended to be used on Jetson TX2 systems, and its setup process is almost identical to the one of the jetsonnano base unit. The main difference is that the JetPack version is 4.6.3, which is the last one compatible with the Jetson TX2.

jetson5

This is the last of the series of base units aimed at Jetson systems, and thanks to recent support to Docker by Nvidia is one of the most simple to set up. It is meant to run on Jetson systems running a minor version of JetPack SDK version 5, a category which includes Xavier NX and AGX, and Orin NX and AGX. The base image is the official Nvidia JetPack SDK Docker image for machine learning, called `14t-m1`, which is available from the Nvidia Container Registry [114]. This image is based on Ubuntu 20.04, and includes a CUDA toolkit, cuDNN, and a set of libraries and tools for machine learning and artificial intelligence similar to that included in the x86-cudev base unit. Thus, the installation begins already as a JetPack environment, and what is left to

install is the same set of tools and middleware as in the previous base units, set up in a similar way as in the other `jetson` base units.

Base unit	Docker image compressed size
<code>armv8-base</code>	1.7 GB
<code>armv8-dev</code>	1.93 GB
<code>jetson5</code>	9.04 GB
<code>jetsonnano</code>	3.6 GB
<code>jetsontx2</code>	3.6 GB
<code>x86-base</code>	1.69 GB
<code>x86-cudev</code>	13.11 GB
<code>x86-dev</code>	2.96 GB

Table 4.3: Sizes of the compressed Docker images of the base units currently available in `dua-foundation`.

4.3 Unit template: `dua-template`

The previous section showed how the consistent environment provided by DUA is designed and built to support an already wide variety of hardware platforms and host system configurations. Recalling that the intended user of this framework is a researcher or a developer tasked with the design of a prototype for a distributed autonomous system, what is left to do is to provide them with a development environment based on the base units discussed before. In other words, since frameworks inherently define a standard way of developing and organizing software as shown by the other solutions presented in Section 2.3, DUA has to provide a *template* for the development of a unit. This is the purpose of the next core DUA module: `dua-template` [115].

The `dua-template` is implemented as a GitHub template repository, *i.e.* a special type of Git code repository hosted on GitHub [116] which exists only to be *forked*, *i.e.*, copied to generate new projects which start with its contents. It contains a set

of files that make up a structural template for the development of software modules for distributed systems as DUA units. Once appropriately forked on GitHub, it can be configured as the starting point for a new project as hereby described. The main goal of this project is to offer a common, shared work environment that minimizes the time and effort required to set up prototyping and experimenting of new solutions, as well as to provide a common framework for teams that can be versatile enough to support any development workflow. It aims at achieving so by providing three main features: workspace organization for both development and deployment, hardware and operating system abstraction via the base units and their contents, modular structure for easy integration of multiple components into a single macro-project or distributed architecture.

A key component of a software development process is today a good Integrated Development Environment (IDE). During the development of this work, from its embryonic stages to its current version, the IDE of choice has been Visual Studio Code (VS Code) by Microsoft [117]. Visual Studio Code is a free, open-source, cross-platform IDE that can be used to develop software in many different languages. It is particularly useful for this use case, where it can be used to develop code in C/C++, Python, and other languages, and it can also be used to manage Git repositories and Docker containers. It is also very customizable and can be extended with plugins that add support for many different languages and tools. As the following sections show, this template is configured to work with VS Code out of the box, but it is just a suggestion: it is possible to use other IDEs, or none at all, by simply managing differently the files and directories that are included in the template according to the specific needs of the project at hand.

The description of the functionalities offered by this template starts from the explana-

tion of the project layout that it provides, hereby informally referred to as the *filesystem layout* of a unit. Then, it can be explained how to set up a DUA unit on top of one or more base units, and how to eventually set up a bigger project by integrating multiple units. The last part of this section is dedicated to the discussion of a GitHub workflow that is used to keep units and projects hosted on GitHub up to date with respect to the main `dua-template` version.

4.3.1 Unit filesystem layout

The filesystem layout of a DUA unit is designed to be as simple and intuitive as possible, to allow the user to quickly understand the structure of the project and to easily find the files that they are looking for. The layout is also designed to be consistent across all units, to allow the user to quickly understand the structure of a new unit even if they have never seen it before. The layout is also designed to be flexible, to allow the user to easily add new files and directories to the project as needed. However, a standard set of directories is provided to help the user organize their project in a way that is consistent with the DUA framework. The layout is also designed to be compatible with the structure of the base units, to allow the user to easily integrate their software modules with the base units and to quickly deploy them on the target platform. A visual inspection of the layout of [115], which is also depicted in Figure 4.2, shows that it resembles the organization of a colcon workspace discussed in Section 3.1.3: this is due to the fact that the template is designed to be used also to develop software based on ROS 2, and the colcon workspace is the standard way to organize ROS 2 projects. Nonetheless, a set of countermeasures are put in place to avoid adding build artifacts and similar contents to version control: almost every directory among those listed below contains an appropriate `.gitignore` file that prevents known file types to be added to Git, and the user is free to modify them as they see fit to customize

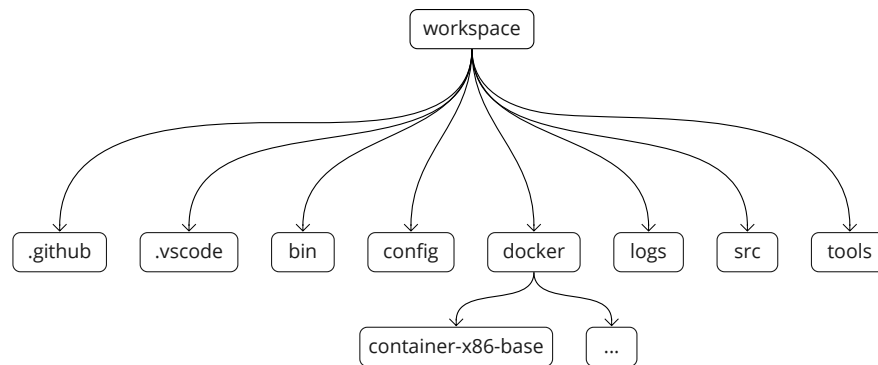


Figure 4.2: Directory tree of a DUA unit based on `dua-template` [115].

this behavior. The best way to understand the `dua-template` filesystem layout is to examine the purpose and contents of each directory it contains, keeping in mind that the actual usage of some files is clarified later on.

`.github`

This directory contains the GitHub workflow files that are used to keep the unit up to date with respect to the main `dua-template` repository. The workflow is based on GitHub Actions, a feature of GitHub that allows to run automated tasks when certain events occur in a repository. It is contained in the file `sync-dua-template.yaml` in the `workflows` subdirectory, and it is triggered when a new commit is pushed to the repository or at a specific time of the day. It is presented in Section 4.3.3.

`.vscode`

This directory contains configuration files for Visual Studio Code. They are present to provide each user with a good starting point for their development environment, and they can be customized as needed. Keep in mind that they are applicable since, as discussed later on, Docker containers based on DUA base units for development are also meant to be opened and managed with VS Code, and when this is done the

settings for the IDE are automatically loaded from this directory. The files are:

- `c_cpp_properties.json`: this file contains the configuration for the C/C++ extension for VS Code, which is used to develop C/C++ code.
- `launch.json`: this file contains configurations to run software modules from a VS Code instance.
- `settings.json`: this file contains the general settings for VS Code, such as the theme and the font size, as well as system paths to be used by the IDE.
- `tasks.json`: this file contains the tasks that can be run from VS Code, such as building the software modules or running tests.

`bin`

This directory is intended to contain any sort of standalone executables and scripts concerning a project. As for any other directory that has an intended purpose, the user is encouraged to put such files here, but as many other directories in the template, this is just a suggestion and the user is free to organize their project as they see fit. In the template, only the `dua_setup.sh` script is present, which is used to set up a DUA-based project as explained in the following sections.

`config`

This directory is intended to contain configuration files that concern an entire unit or project. Again, this is just a suggestion, and for the moment the template does not require any file to be present from the start in this directory.

`docker`

This directory starts empty, but will contain the configuration files for the Docker containers that are used to develop and run the software modules. As explained previously, there may be a Docker container for each base unit that a project is intended to run on, so this directory may have multiple subdirectories, one for each base unit. This aspect is also explained later on, since the contents of this directory are also set up by the `dua_setup.sh` script.

`logs`

This directory is also empty at the beginning, because it is intended to contain log files generated by the software modules when they are run, including sampled data of experimental runs. To ensure that such large files are not added to the version control system by mistake, a `.gitignore` file is present in the directory that excludes all files in it.

`src`

This is probably the most important and most used directory of the entire template. It is intended to contain the source code of the software modules that are developed for the unit, as well as subdirectories containing units that are added to a larger project. This directory is also meant to be found and managed by `colcon`, in case it is used as a build system for the project.

tools

This last directory also starts as an empty directory, and is also meant to contain source code and software modules which are under active development withing the context of the unit or project at hand. The difference with respect to the `src` directory is that the files in this directory are not meant to be built using `colcon`, which in fact ignores this directory thanks to the `COLCON_IGNORE` file that is present in it.

4.3.2 Configuring a new unit

The setup of a new unit or project is a process that is meant to be as simple as possible, to allow the user to quickly start developing software modules for a distributed system. The setup process is performed by running the `dua_setup.sh` script that is present in the `bin` directory of the template. The script is designed to be run from the root directory of the unit, in one of the ways shown in Listing 4.1.

```
1 dua_setup.sh create [-a UNIT1,UNIT2,...] NAME BASE_UNIT
2 dua_setup.sh modify [-a UNIT1,UNIT2,...] [-r UNIT1,UNIT2,...] BASE_UNIT
3 dua_setup.sh delete BASE_UNIT
4 dua_setup.sh clear BASE_UNIT
```

Listing 4.1: Possible invocations of the `dua_setup.sh` script.

An explanation of all the possible invocations of the script is given in the following sections. Before going further, another definition is in order here: with the term *target* we identify a single Docker image that is used in a project, and that is built from a single Dockerfile pair plus many other configuration files discussed later on. A target can be used to run a single unit, or multiple ones, or even a whole control architecture; the name resembles the meaning of the term *target* in the context of a Makefile, and

is used to refer to the same concept in this context: each unit or architecture can have multiple targets, *i.e.*, Docker images, each one based on a different base unit. Thanks to base units, the software that makes up a unit or project is ensured to run appropriately and similarly across all supported targets.

Creating a target

The command at line 1 of Listing 4.1 creates a new target, *i.e.*, a Docker image for a unit or project, plus a set of configuration files. The `-a` option can be used to specify a list of units that will be integrated in the new target, a feature that is explained later on in Section 4.3.2, and the `NAME` argument is the name of the project. The `BASE_UNIT` argument is the name of the base unit to use. A password is required for all projects since the container's internal user runs with elevated privileges and has full access to the host's network stack and hardware devices. It will be asked for during the creation of the target, and will be stored in hashed form in the target's Dockerfile.

`create` will create a new `container-BASE_UNIT` directory inside `docker/`, and will copy and configure all the template files stored in `bin/dua-templates`. These include Dockerfile and `docker-compose.yaml` files, as well as many shell configuration scripts and other configuration files that are used to build and run the image. Once copied, they can be modified to fully support the specific features of the unit or project at hand. In particular:

- `aliases.sh` is meant to contain shell aliases.
- `commands.sh` is meant to contain shell functions that execute commands that involve the software package that make up the unit or project at hand.
- `ros2.sh` contains configuration options and functions for the ROS 2 installation

that is provided in every DUA image.

- Other files configure the shells and other basic programs like text editors.

Once a target has been created, it can be managed using Docker or VS Code extensions interchangeably. Containers are meant to be run in detached mode, *i.e.*, with the `-d` option, and can be accessed using the `docker-compose exec` command as shown in Listing 4.2.

```
1 docker-compose exec NAME-BASE_UNIT zsh
```

Listing 4.2: Docker Compose command to open an instance of the Zsh shell inside a DUA container.

This is due to how DUA containers are brought up, which is written at the end of the most important file among those generated by the setup script in this invocation: the `docker-compose.yaml` file. This file is meant to be used with Docker Compose (see Section 3.2.1), either directly or through VS Code, to start the container specifying the many configuration options for the Docker Engine that would instead result in an unreasonably long command line for `docker run`. Each base unit requires specific configurations, which is why the setup script copies and configures the right files for the right base unit from the template ones stored in `bin/dua-templates`. Keep in mind that the contents of this file can be modified accordingly once the target configuration has been generated. The settings specified in the `docker-compose.yaml` file can be summarized as follows:

- Build context and other useful options for the container build process.
- Environment variables that allow terminals and graphical applications to work properly.

- Host networking, shared memory and IPC namespace, to provide to applications running in the container full communication between the container and the host, as well as with remote hosts.
- Access to GPUs, if present.
- Access to all hardware devices with full privileges and `/dev` and `/sys` mounts.
- Access to the unit or project files from the host filesystem: the root directory of the unit is mounted in the container at `~/workspace`, and all the configuration files for the container respect this convention.

These settings, apart from those exclusively dedicated to the image build, make up the core of the integration of Docker in DUA as a tool to build and deploy replicable system environments: applications running inside a container configured in such a way have access to the host system in pretty much the same way they would if they were running directly on it, with direct access to any hardware device and driver ranging from the network stack to communication interfaces and even GPUs, if they are present. The advantage is that the entire installation and dependency configuration is performed inside a Docker image, avoiding the risk of tainting the host configuration with the risk of compromising it, leading to a reinstall that would be a timely process especially on SBCs. Furthermore, a Docker image can be simply packed and downloaded to any number of hosts once it is finalized, making the replication of a system environment a matter of seconds, compared to traditional solutions for, *e.g.*, SBCs, that would involve flashing the board with a new OS image and then configuring it from scratch, manually or using scripts. Moreover, a Docker image constitutes a snapshot of the system environment at a given time, including the version of the software packages and libraries that are installed in it, which can be useful to reproduce experiments or to ensure that a system is always running the same software version, guaranteeing

stability and reproducibility. The only overhead is due to the size of the images, as in Table 4.3, but considering that once the containers are started there is no more overhead, due to the fact that the unit files are mounted from the host filesystem, the advantages of using Docker in this context are clear. Nonetheless, these options deactivate by default many security features that Docker enforces, to achieve full isolation of a process running inside a container with respect to the rest of the host system. Note that this work, as similar solutions that are starting to leverage Docker in the field of autonomous systems, some of which are discussed in Section 2.3, does not intend to offer that kind of isolation for two main reasons. First, the end goal is that of easily replicating system configurations across multiple hosts, allowing the possibility of applications accessing the full set of resources offered by the host system: this implies that only a subset of the functionalities of Docker are necessary by the definition of the original problem. Moreover, the context of interest is that of a development environment for research and prototyping in which things have to be made to work and tested quickly first, and then eventually robustified and gradually isolated. This is why the default settings of the Docker containers are set to allow full access to the host system, but nonetheless they are fully exposed to the user of the framework, who can modify them as they see fit to achieve a configuration in which containers are properly isolated, being exposed only to a subset of the features and resources of the host system that are necessary for their applications to run. Some of these options have not been considered for simplicity, such as configuration of the *secure computing mode* (`seccomp`) module to restrict access to specific system calls, but given the complete set of configuration files offered by the template, the user can easily add them to the container configuration.

Finally, an item that deserves a thorough explanation is the `command` item, which specifies the command that is run when the container is started. This is set by default

to what is shown in Listing 4.3, which is essentially a Bash shell process that sleeps indefinitely, being woken up only by a `TERM` signal that triggers a graceful exit. This is done to keep the container running and to allow the user to access it at any time, and in any way they see fit, *i.e.*, opening within it multiple shells of the flavor they prefer, or running graphical applications, or running applications, or accessing server applications running within it from remote hosts, or connecting to it using a VS Code instance, etc. Keep in mind that these containers are meant to be deployed on autonomous systems, so they should stay active with an arbitrary set of processes running within them. This is again not the originally intended use of Docker, *i.e.*, that of running single microservices inside each container, but is instead a new way of using Docker to provide a complete, versatile, and consistent environment, which is currently being exploited also by other frameworks in different contexts.

```
1 /bin/bash -c "trap 'exit 0' TERM; sleep infinity & wait"
```

Listing 4.3: Default command of a DUA container: an idle shell process.

Modifying a target and integrating units

The remaining core functionalities of `dua-template` concern the configuration of a unit to run the software modules that it contains. In this context, the main aim of DUA is not only to provide a stable and reproducible way to do so, but also a mechanism to simply integrate a unit alongside multiple other ones in a bigger project, in a modular and simple fashion.

Since units are software modules in essence, the system configurations they require usually boil down to two categories: system settings, like environment variables and configuration files, and dependencies, such as packages to install. Recalling how base

units are set up as explained in Section 4.2, these steps should be somehow added to the image build process on top of what the base unit already provides. This is the purpose of the `Dockerfile` file, generated by the `dua_setup.sh` script and placed in the directory of every configured target.

The interested reader can find templates for the `Dockerfile` in [115]. As for the previous discussion, this section aims to guide the inspection of such templates by detailing their contents. The operations performed by the `Dockerfile` are, in order:

1. Start from the target's base unit image.
2. Perform unit-specific operations.
3. Configure the internal user of the container.
4. Create mount points for configuration files and the workspace.
5. Configure the shell.

While most steps are standard and should not be altered in general, although the user is given complete control over how the framework works, the second is the one where manual intervention is encouraged. This is because, initially, that step is nonexistent, as illustrated by Listing 4.4.

```
1 # NAME SETUP START #
2 # NAME SETUP END #
```

Listing 4.4: Unit configuration step in a `Dockerfile` as generated by the template.

`NAME` is, of course, replaced by the name given to the unit by the user as in line 1 of Listing 4.1. Between those two lines, the user can add any `Dockerfile` directive they

require to configure the system environment of the container to support the software that will make up the unit. Once built, the container will have all the features and functionalities of its base unit, plus what the user added to support their own software module.

This section of the `Dockerfile`, and the way it is delimited, are instrumental to achieve the desired level of modularity, making the integration of multiple units in a single project again a matter of a few commands, listed in the remaining lines of Listing 4.1.

Suppose that a unit must be integrated in a project. The first step consists in adding the unit codebase to the one of the project, *i.e.*, copying it in the `src/` directory. It is assumed that all codebases are managed by the Git version control system, as stated in Section 4.1, which offers two functionalities meant to this purpose: submodules and subtrees, illustrated in Section 3.3.1. They are both capable of linking the unit's repository to the project one, but they are meant for different use cases.

Submodules are the preferred way to include units in a DUA project when the user plans to actively work on the units themselves, while working on the project. A submodule is a Git repository that is included in another Git repository as a subdirectory. This means that the submodule is a separate repository, with its own history, branches, and tags, that is included in the parent repository as a subdirectory. This is particularly useful when the submodule is actively developed, and when changes to it must be pushed back to the original repository. The main drawback of submodules is that they require additional commands to be managed, and that special attention must be paid to the point of the Git history in which the submodule is found before making any changes inside it, or worse, committing them. Moreover, the reference commit of the submodule in the parent repository must always be updated manually. Nonetheless,

submodules are a very powerful tool that, when used correctly, can make the simultaneous development of multiple projects much easier. For this reason, given the degree of modularity that DUA aims to achieve, they are supported by `dua-template` and their usage is encouraged as the preferred way to include units in a DUA project in any case.

Subtrees, on the other hand, are the preferred way to include units in a DUA project when the user does not plan to actively work on the units themselves. Essentially, integrating another Git repository as a subtree still ensures the possibility of specifying the commit, branch, or tag of the repository to be integrated, but requiring a simpler management with respect to submodules, since files and Git histories will be merged in those of the parent repository, with little to no additional commands required to keep them in sync, resulting in a single repository that contains all the files and histories of its subtrees. However, they have one major drawback: they may make Git history management more computationally difficult, since the history of the subtree is merged in the one of the parent repository. This often leads to Git having to go through the whole history of the subtree to find the changes that are relevant to the parent repository, to ensure that a coherent history is maintained; this could easily become a slow process, which is why the use of subtrees is discouraged when the subtree is actively developed.

Then, as for every DUA codebase, a target for the project must be configured. What follows can be applied to any number of targets a project shall support. The `modify` command of the `dua_setup.sh` script, at line 2 of Listing 4.1, is intended for this purpose. The `-a` option is used to integrate one or more units into the specified target, while the `dua -r` option is used to remove one or more units from the target. To ensure that all the system configurations that the supplied units require are performed

at the same time within that target, what the script does in case of an addition is simply to look for the relevant units in the `src` directory, and then for their corresponding target directory; in case the target at hand is, *e.g.*, `x86-base` and the unit is called `sensor-driver`, the script will look for the directory `src/sensor-driver/docker/container-x86-base`. If the search is successful, the script will proceed to copy the unit configuration section of the `Dockerfile`, delimited as in Listing 4.4, within the target's `Dockerfile` and into its own unit configuration section. This way, the user can be sure that all the system configurations that are required to run the software module are performed at the same time, and in the same way, for all the units that are integrated in the target. The same operation is performed in case of a removal, but in reverse: the script will look for the unit configuration section in the target's `Dockerfile`, and will remove it if found. If some units need to be set up in a specific order, it is up to the user to specify them in the desired order in the `-a` option of the script. This description also clarifies the remaining reasons behind the filesystem layout of a unit, and the presence of the `Dockerfile` in the subdirectories of the `docker/` directory, as illustrated previously in Section 4.3.1.

The commands at line 3 and 4 of Listing 4.1 are pretty straightforward and are provided for convenience. The `delete` command can be used to quickly and easily remove a target from a unit or project, effectively deleting its subdirectory from the `docker/` directory. The `clear` command can be used to remove all the lines between the delimiters of the unit configuration section of a target's `Dockerfile`, effectively resetting the target to its base unit configuration.

4.3.3 Updating `dua-template` in units and projects

As every software component, and because the DUA project is under active development, `dua-template` is subject to updates. Such updates can be of various types, ranging from bug fixes to new features, and they are meant to be applied to all units and projects that are based on the template. To ensure that all units and projects are up to date with respect to the main `dua-template` repository, a GitHub workflow is provided in the `.github` directory of the template, as explained in Section 4.3.1. Every GitHub repository forked from the main `dua-template` repository will have this workflow enabled by default, and it will be triggered when a new commit is pushed to the `master` branch of such repository or at a specific time of the day. As for any feature of DUA, the user can modify or disable this workflow as they see fit.

The interested reader can find the code of the workflow at either [115] or in Listing A.1 in Appendix A. The workflow, meant to run on virtual machines called *runners* provided by GitHub, clones both the main `dua-template` repository and the repository where it is running, and then it compares the two repositories to find out if the main repository has been updated, *i.e.*, either modified or created or even deleted, with respect to the forked one. The files that the workflow compares are only those that are part of `dua-template`, *i.e.*, it does not compare nor modify files belonging to targets that have already been created and configured. This behavior is intended by design, and due to two reasons: first, it is not possible to deterministically distinguish user-applied modifications to target files from those that are part of the template, and second, the user always has the last word on how their project is configured, having the ability to modify every template file as they see fit to ensure that their software works correctly. Thus, the user is the only one that can decide whether and how to apply the update, either manually reconfiguring or regenerating the targets as

explained in Section 4.3.2 using the updated template files. To provide a convenient way of doing so, the workflow creates a pull request in the forked repository with the changes to the template portion of the codebase, and the user can then review it, update their unit files, and merge the pull request to apply the changes.

4.4 Remarks

Recalling Table 2.1, it is now possible to compare DUA against the other frameworks presented in Section 2.3. The comparison is summarized in Table 4.4.

As one can see, the Modularity (M) requirement is satisfied by almost all frameworks. This is a common feature of modern software engineering practices, and in the context of distributed and autonomous systems it clearly improves the development process and the overall system maintainability. The Overhead (O) requirement is also satisfied by most of the frameworks, which is a good indicator of the efficiency of the solutions presented, including DUA. Using the same framework for both development and deployment is clearly a good intention, thus the overhead that it introduces should be minimal.

DUA is the only framework to completely satisfy the Abstraction (A) requirement. This is a key feature of DUA, as it allows the developer to focus on the high-level aspects of the system, while the framework takes care of the low-level details. This is a clear advantage of DUA over the other frameworks, as the developer who uses it can focus on the functionality of the system, rather than on the system configuration. The key feature of the implementation of DUA that allows this is the use of the base units discussed in Section 4.2.

The only requirement that DUA does not fully satisfy is the Development (D) require-

ment, which is also an issue for other frameworks. Because of how the requirement is stated, there are a lot of aspects that may be considered while evaluating it. The issue that DUA still does not solve is that of facilitating testing and debugging processes: the system abstraction layer of `dua-foundation` does a great job at simplifying the development process over different platforms, but it does nothing to help the developer while testing their modules among the many different targets on which they may run. Moreover, there is currently no support for cross-compilation or network-based debugging, which would be a great addition to the framework. These aspects have been identified as future work for DUA, and will definitely be addressed in subsequent versions of the framework, but for now have been overlooked due to time constraints during the development of this work.

	Modularity	Development	Abstraction	Overhead
RoboStack	✓	×	+	✓
Vulcanexus	×	+	+	✓
EPICS	✓	✓	×	✓
BlockNet	✓	+	+	×
F Prime	✓	+	+	✓
MARTe2	✓	+	+	✓
DUA	✓	+	✓	✓

Table 4.4: Comparison of DUA against the solutions presented in Section 2.3, with respect to the requirements listed in Section 2.2; “✓” denotes a completely satisfied requirement, “+” indicates that the solutions provided do not fully satisfy a requirement, and “×” marks an unsatisfied requirement.

5 The case of autonomous robots

The previous Chapter 4 presented the Distributed Unified Architecture, the novel framework proposed in this work. Its design and implementation have been described, from the system abstraction layer to the unit template structure. The various features and functionalities of the framework, provided by either existing tools presented in Chapter 3 or custom-made components, have been described in detail and discussed.

This work is the result of research activities spanning some years in the field of autonomous and distributed systems, as illustrated at the very beginning of this book. The Distributed Unified Architecture is the materialization of multiple attempts and ideas whose goal was to simplify research work involving autonomous systems and control architectures, as well as the activities of single developers or small teams. Many of the ideas, concepts, and requirements on which DUA is based have stemmed from long and fruitful discussions with colleagues and friends, usually held after a research project was completed, to assess what went well and what could be improved during the development process. This chapter and the next tell about the fruits of these practical experiences, showing some first applications of the DUA framework and the technologies that it integrates in real-world scenarios; their aim is to complete the comprehensive description of the proposed framework, discussing some of its notable applications and the preliminary results obtained so far by applying it to real scenarios.

Although DUA is now a general-purpose framework for distributed systems, its first versions were designed with a single application in mind, which is also the subject of this chapter: autonomous robotics. As discussed in Chapters 2 and 3, autonomous robots of today are intrinsically distributed systems, from both the hardware and the software perspectives. DUA was designed to offer a unified approach to the development of such systems, providing a common ground for the integration of heterogeneous components and tools and enabling the development of complex behaviors and functionalities in a modular and scalable way.

The contents of this chapter summarize the additional features that the DUA framework provides to the autonomous robotics domain, as well as the first results obtained by applying it and the technologies on which it is based to a real-world scenario. First, some robotic software packages developed for and integrated into DUA are presented. Then, to provide a concise yet explanatory example of the capabilities of the framework, two software modules developed with DUA are briefly presented: their design and development process and the challenges they posed were among the main motivations for the creation of the framework. Finally, some preliminary results obtained by applying the technologies and solutions on which DUA is based to real-world scenarios are illustrated.

5.1 Software utilities for ROS 2: dua-utils

With the system abstraction layer and the unit template finalized, a single problem is left to solve. It is of practical nature, and arises only when one starts to use the framework and, in particular, the middleware tools integrated into it for robotic software development, such as ROS 2 (see Section 3.1.3). The features of the ROS 2 middleware compose a set of services that the robotic software developer can use to

build applications and modules of various degrees of complexity, and the open-source nature of the framework has allowed for a wide community to improve it over the years, reviewing existing functionalities and adding new ones. Nonetheless, the ROS 2 middleware is a complex system, and its many features can be overwhelming for a beginner or a developer who is not familiar with all of its functionalities. Moreover, some of its APIs are not as intuitive as one would expect, requiring complex code to be written to perform simple tasks which cover instead the vast majority of their use cases.

The ROS 2 community is aware of these issues, and the development effort is improving the quality of the software every day. However, over some years of experience and active development of ROS 2 modules, the necessity arose to create a set of utilities that would simplify some parts of the framework, wrapping its complexity and providing a more intuitive interface to the developer. While waiting to propose the same set of changes to the community and have them integrated into the official ROS 2 distribution, a process that can notoriously happen but also take a long time, the choice has been preferred to develop the same set of utilities as packages that could be integrated in the system configuration provided by every DUA base unit, so that they could be immediately distributed and used in the development of robotic software modules based on DUA. Note that these software modules, as shown later on, are maintained as independent projects with no specific ties to DUA, so that every published work that uses them can be easily integrated into other systems, frameworks, or architectures for the benefit of the scientific community; this is also the case of the two example modules presented later on in this chapter.

This led to the creation of the third foundational pillar of DUA: the software collection called `dua-utils` [118]. Note that, although at the time of writing it is a collection of

utilities for ROS 2 and robotic software development, it may host different kinds of third-party software packages that can be useful in the development of distributed systems, as long as it is open-source and can be built with `colcon`. This can be seen as a limitation but it is only due to how it is currently integrated into DUA base units: at the very end of their setup process, the last stage clones the repository into the image filesystem and builds the packages contained in it, cleaning up build artifacts afterwards. This process has two main advantages: first, when the modules are built in this way they end up being part of the DUA base unit image, so that they can be used in the development of software modules based on DUA without the need to install them manually; secondly, the process is automated but performed at the end of every base unit, so if one of the utilities in `dua-utils` is updated the base unit image can be rebuilt by performing only this last step, reusing previous cached layers and saving time and bandwidth. What described so far may of course change in the future, should the necessity arise to include in `dua-utils` software packages that cannot be built with `colcon` or that simply concern other domains of distributed systems development. The motivating idea is that `dua-utils` is a way to customize the very base layer of the DUA framework which is provided to and intended for its users, who can actively suggest and modify its contents to adapt the framework to their most frequent necessities and usage scenarios.

To give an idea of the set of improvements that DUA provides to the ROS 2 middleware, as well as a comprehensive overview of the work that has been done so far not only with DUA but also on it, the following sections introduce some very representative utilities that are part of the `dua-utils` collection.

5.1.1 Managing ROS 2 processes: `dua_app_management`

`dua_app_management` is a collection of software modules and source files for the management of applications and processes in the Distributed Unified Architecture. This package contains modules, both compiled and source-only, to be included in DUA-based projects. It is particularly aimed at the development of ROS 2 applications, with the aim of simplifying some common tasks and wrapping boilerplate¹ code, especially that which involves the lifecycle of a process and requires calling the OS to do so.

The package currently contains the `ros2_app_manager` C++ package, which provides two libraries and the implementation of a component container (see Section 3.1.3). The first goes by the name of the package itself, and is a header-only library that implements a wrapper object for the usual main thread code of ROS 2 applications, *i.e.*, processes that run single nodes or collections thereof or act as component containers. The necessity for this library comes from the fact that, if one wants to write the main function for a ROS 2 application going a little beyond what the examples from the official documentation show, one finds after some time that the same code is written over and over again, with only the node name and the node class changing. Operations performed by this code usually include initializing the ROS MiddleWare context, which is the entrypoint towards the underlying communication middleware used by ROS 2, configuring options for the ROS 2 node, creating a proper executor for the application, and finally adding nodes to it and spinning it. The lifecycle of such an application ends when an asynchronous event, such a signal, occurs and it must shut down, stopping the executor and destroying all such objects in

¹In the context of software development, *boilerplate* code refers to long code portions that must be written each time in the same way, to perform what is usually a simple and frequent operation. Programmers usually prefer either simpler APIs or wrappers, *i.e.*, equivalent code items that take less options but are simpler to write, addressing the most frequent use cases.

reverse order to correctly release resources and terminate middleware instances. The `ros2_app_manager` library provides a simple API to perform all these operations:

- The constructor of the `ROS2AppManager` class does all the aforementioned initializations, as well as disabling I/O streams buffering for the current process to reduce latencies, which is a common practice in ROS 2 codes.
- The `ROS2AppManager::run` method spins the executor, and returns only when a signal is caught, which is the usual way to handle the termination of a ROS 2 application.
- The `ROS2AppManager::shutdown` method stops the executor and releases all resources as previously described.

The class is implemented as a template class whose template parameter is the ROS 2 executor class to use. The library also includes the relevant system headers.

Since signals are the most common way, at least on Linux systems which are the focus of DUA, to request the termination of a ROS 2 process, the user would benefit from a finer-grained level of control over their action. To clarify the description of what follows, let us briefly introduce the concept of POSIX signals. In operating systems design and programming, and in particular for systems that follow the POSIX standard, *signals* are a way to notify asynchronous events to userspace^{II} programs, a sort of software interrupt that can be delivered to a process by the kernel either on its own behalf or on behalf of another process, which may even do so due to the user's request. Different signals are identified by numerical codes, and represent different situations and errors. For example, when we press Ctrl-C to stop a program running in a terminal,

^{II}In this context, the term *userspace* denotes application programs that are run and managed by an operating system kernel.

the terminal tracks that key combination and sends the SIGINT signal to the process it is running, which is the default signal for interrupting a process. The process can then catch the signal and perform some cleanup operations before terminating, provided that it includes a handler for that signal in its code.

In ROS 2 applications, signals are not extensively used for two reasons: first, applications running on a robot should usually run as long as the robot is on, and second, they require a signal handler to be installed in the application code and some knowledge of the OS APIs. For this reason, ROS 2 usually handles signals by itself, which is good and handy but limiting in some cases. A first example scenario is if debugging is being performed to track a bug that crashes the applications: appropriately coded signal handlers have access to information concerning what was the memory state when the crash occurred, what kind of error occurred, what was the state of the application and of buses it was accessing, and so on. A second scenario is that of a prototype robot which is being tested, a situation in which applications are usually started and stopped via remote shells opened by users. Suppose that the robot is, *e.g.*, a flying drone, and that such applications allow it to fly safely: it would not be desirable if, *e.g.*, the WiFi link crashes and the drone follows as well because the shells that the applications were running within close abruptly; in this case, ignoring some signals like SIGHUP or SIGPIPE would be a good idea.

The second library of this package, called `ros2_signal_handler`, provides a POSIX-compliant signal handler module for ROS 2 applications, which replaces the default signal handler installed by the ROS 2 middleware. The library is a header-only one, and provides a class called `SignalHandler` implemented as a singleton^{III}. It uses the

^{III}In object-oriented programming, the *singleton* pattern is a software design pattern [119] that restricts the instantiation of a class to a singular instance within an entire application. Such instance is usually *static*, *i.e.*, it is created, and thus not constructed, by the compiler within the data segment of an executable and is already present in memory when the execution of the program begins.

`sigaction` API to manage handlers, allowing fine-grained control over the signals that are caught and the actions that are performed when they are caught, and can be given access to the executor running in the application to stop it when a signal is caught. The library also includes the relevant system headers. The following methods are provided:

- `SignalHandler::get_global_signal_handler` returns a reference to the singleton instance of the class.
- `SignalHandler::init` installs the signal handler, which is a static method that must be called before any other method of the class. This handler can call user-defined functions when a signal is caught, and can be configured to ignore some signals.
- `SignalHandler::install` to install a handler for a specific signal.
- `SignalHandler::ignore` to ignore a specific signal.
- `SignalHandler::uninstall` to restore the default handler for a specific signal.
- `SignalHandler::fini` to reset the `SignalHandler` instance to its initial state and restore the default signal handlers for all signals.

All these features, although useful, would be useful only when a ROS 2 application is run as a standalone process, which is not the case for all robotics applications. In reality, what happens is that nodes may also be loaded into a process at run time: such process acts as a component container and nodes are denoted as components, as explained in Section 3.1.3. In this case, the container application is usually already compiled and provided as part of the ROS 2 installation. This is where the third part of

this package comes in: the `dua_component_container_mt` application. It is essentially a component container based on the `MultiThreadedExecutor` executor class, just like the standard `component_container_mt` provided by ROS 2, but with the addition of the `ros2_app_manager` and `ros2_signal_handler` libraries. The application is intended to be used while testing ROS 2 components which may be dynamically loaded and unloaded at run time on real robot prototypes, thus accounting for the potential issues such as the one described in the example scenario above. For this reason, this container application uses the `SignalHandler` object to handle many signals and ignoring some of them, and the `ROS2AppManager` object to manage the lifecycle of the application.

5.1.2 Handling Quality of Service correctly: `dua_qos`

In Sections 3.1.1 and 3.1.3, the concept of Quality of Service (QoS) has been presented as a way to configure the behavior of the communication middleware in a distributed system. In ROS 2, QoS policies are used to configure the behavior of the underlying communication middleware, being it either the DDS or Zenoh (see Section 3.1.2) or something else, and are applied to topics to specify the reliability, durability, and other properties of the communication. The ROS 2 middleware provides a set of predefined QoS profiles that can be used to configure the communication, but it also allows the user to define custom QoS profiles to meet specific requirements. This system is very flexible and powerful, but often used in an inconsistent way. First, note that ROS 2 libraries provide default QoS profiles whose names range from `qos_profile_default` to `sensor_data`, also explaining those purposes. However, these profiles as well as the rationales behind them come from the ROS 1 age, in which the communication was implemented by the middleware itself on top of UDP and TCP. Since there were mostly only these two protocols to choose from, one had almost

always to distinguish between a reliable and an unreliable communication, which led to the establishment of a set of rules of thumb like, *e.g.*, the common belief that data produced continuously by a sensor should be sent with a best-effort policy, since packet loss is always allowed. This is the case unless algorithms processing that data may also account for late packets, in which case a reliable policy should be preferred. Moreover, with the plethora of options that DDS QoS offers, these binary distinctions do not apply anymore. The ROS 2 API is quite flexible and allows the user to define custom QoS profiles, but the user is left alone to decide which profile to use and how to configure it. This can lead to inconsistent configurations across different nodes and topics, which can result in unexpected behavior and performance issues.

More configuration options imply that a finer control over the transmission policies can be achieved, accounting for an issue that ROS 1 struggled with: intrinsic difference in data types being transferred. For example, an inertial sensor may produce many samples every second but they will all correspond to small packets since they are made of a few numeric fields. Conversely, a LiDAR scan may have a lower frequency but produce large packets, since each scan is made of many points; a similar situation may arise with cameras. Reliability is not the only issue to consider: the frequency of the data, the size of the packets, the importance of the data, and the network conditions are all factors that can influence the choice of the QoS profile to use.

The `dua_qos` library [120] provides API that return QoS profiles that account for these issues, distinguishing first between the desired reliability of the communication and then between the data type being transferred. The repository contains two software libraries: `dua_qos_cpp` for the C++ language and `dua_qos_py` for the Python language. Apart from their actual implementation, whose details concern only the syntax of each language, the functionalities they offer are the same. Before going further, note

that DUA does not enforce the use of these QoS profiles in any way nor place: they are only included as a suggestion for the user to simplify the development of ROS 2 applications, and are also designed to be compatible as much as possible with the QoS policies that are both provided by the ROS 2 middleware and used in the majority of the ROS 2 software available today. Lastly, the profiles are intended to be used only with topics. One can modify QoS for internal communications of services and actions too, but that is usually not recommended: those primitives are synchronous and reliable by nature and, in the case of actions, require specific QoS policies to achieve stateful communication. For these reasons, the `dua_qos` libraries provide only QoS profiles for topics.

The first two categories are `Reliable` and `BestEffort`: these map to two distinct libraries in both packages, whose contents are almost the same. The difference they enforce is the `ReliabilityPolicyKind`, *i.e.*, the reliability part of the QoS policies provided by each library. There is no additional distinction between the two categories since it is actually up to the user to decide which one to use: today, the `Reliable` category applies to practically every transmission since network congestion and bandwidth optimization are managed independently by the communication middleware used by ROS 2, mainly thanks to the `HistoryPolicy`. This policy is set to `KEEP_LAST` by default, which means that only a fixed-size buffer of samples is kept in memory to be either sent or received, and that the rest is discarded, independently of the configured reliability which comes into play shortly after, when samples are actually being transmitted; this mechanism, although not sophisticated, acts as a sort of congestion control that works well in most cases. All APIs in the `dua_qos` libraries allow to set the depth of such buffers, but provide a default value that is usually good enough for most cases.

For each package, both `Reliable` and `BestEffort` libraries then provide QoS profiles via the following APIs:

- `get_datum_qos`: returns a QoS profile for generic data, *i.e.*, data that is not particularly large in size or requires special handling on the QoS size. This is due to the above reasoning, thanks to which both commands and setpoints, and sensor data are, *e.g.*, the same thing.
- `get_scan_qos`: returns a QoS profile for LiDAR scans, spatial scans, or similar data. The peculiarity of this data is that while it may have relatively high or low frequencies, it gets easily very large in size, so a low history depth is suggested here to minimize the risk of network congestion and memory exhaustion.
- `get_image_qos`: returns a QoS profile for images, whose size may vary from small to very large, but whose frequency rarely exceeds a few tens of Hertz. The history depth default value is not as low as for scans, but still lower than for generic data and configurable by the user.

The only difference among the two libraries is that the `Reliable` one also provides the `get_persistent_qos`, a particular type of policy intended for reliable transmissions that may be resumed at a later point in time. This is possible thanks to a feature of the DDS standard called the `DurabilityPolicy`, discussed in Section 3.1.1, which allows a publisher to store all samples in a persistent storage and send them again when a new subscriber appears. This is useful for data that may not be produced continuously but is still important, like configuration data or state information, and is used in this library together with a higher history depth to ensure that all samples are stored and sent.

5.1.3 Improving node development: `params_manager`, `dua_node`

Among the many ROS 2 features (see Section 3.1.3), there is one of paramount importance concerning the nodes: the possibility of specifying configurable variables for them that can be inspected and set at any point in time, known as *node parameters*. These parameters are usually set at launch time, either by the user or by the launch system via configuration files, and are then read by the node to configure its behavior. The ROS 2 middleware provides a set of APIs to manage these parameters, whose task is that of building and storing in each node a database of the parameters that it has. The keys used in this database are strings, which contain the name of a parameter. Each parameter is then associated with its type, which can be a string, an integer, a floating-point number, a boolean, or a list of any of these types, and its current value. There are also other flags and fields, identifying whether a parameter is read-only or, in case it is of numeric type, its valid ranges.

ROS 2 configuration files allow the user to specify only parameter values, meaning that all the aforementioned information must be provided when the parameter is *declared*. In ROS 2 code, a *parameter declaration* is the act of calling an API that creates an entry into the parameter database of the node, writing all such information which is passed as arguments to the API. To correctly declare parameters passing all the correct and relevant information, one has to write quite the quantity of code. This is not a problem when the number of parameters is small, but it can become cumbersome when the number of parameters grows, especially if they are of different types and have different constraints. As mentioned earlier, this is a classic example of boilerplate code.

Another issue concerns parameter updates and their usage in the actual code of the node. Each time a parameter must be read, the node must call an API that retrieves

the parameter from the database, and then cast it to the correct type. This is not a great concern if the parameter is read only once or with a low frequency but when it is, *e.g.*, the gain of a control law that must be evaluated at each high-frequency iteration of a control loop, the overhead of the database queries might become significant. Assuming that the parameter type is never going to change, something that the ROS 2 APIs allow but is seldom necessary in practice, it would be preferable to store the parameter value in the node and update it only when the parameter is updated. This way, parameter reads would be much faster. To achieve this, a callback must be registered with the ROS 2 node that is called when an update is requested for a set of parameters: when this happens, some checks like the range of the parameter have already been performed but other ones like the type of the parameter have not. Moreover, there can be only one such callback in a node, which means that it must check for which parameter an update has been requested and act accordingly.

The amount of code that one has to write to achieve the aforementioned functionalities, in every ROS 2 software and every time in the same way, grows uncontrollably with respect to the amount of parameters that each node has. The code that manages parameters in a ROS 2 node becomes unnecessarily long and complex, difficult to manage and debug. This is the motivation behind the development of the `params_manager` package [121], which provides a set of utilities to simplify the management of parameters in ROS 2 nodes. This package currently contains only an implementation for C++.

Before the `params_manager` C++ library comes into play, there is a Python script that can be used to automatically generate the code that declares the parameters in the ROS 2 node class. This script is automatically called by a CMake directive that extends the ament installation (see Section 3.1.3). Thus, the first step, as shown in

the documentation of [121], is to add a YAML file to the source code of the node that, with a specific syntax shown in Listing 5.1, contains the list of parameters that the node must manage together with all their metadata. The script will then parse this file and generate a C++ source file that gets added to the build process, containing a routine called `init_parameters` that declares all parameters in the node and sets up the update callback. To make this work, a bit of code must be added to the node class, as shown in the documentation: a `Manager` object must be instantiated as a member of the node class, and the `init_parameters` function must be declared as a method of the node and called in its constructor, better if at its beginning since from that point on all parameters will have been declared, set, and ready for usage. This way, the boilerplate code is reduced to a bare minimum of five lines, and all relevant parameter data must be written in a configuration file which is much easier to manage and debug than the code itself.

```
1 header_include_path: dir/node_header.hpp
2 namespace: node_namespace # This is optional
3 manager_name: manager # This is optional
4 node_class_name: MyNode
5
6 params:
7   camera_offset:
8     type: double
9     default_value: 0.1
10    min_value: 0.0
11    max_value: 0.5
12    step: 0.0
13    description: "Distance between camera and drone center."
14    constraints: "Must be in meters."
15    read_only: false
16    validator: function_name3 # This is optional
17
18   hsv_from:
19     type: integer_array
20     default_value:
21       - 0
22       - 1
23     min_value: 0
24     max_value: 255
25     step: 1
26     description: "HSV color lower range."
27     constraints: "."
28     read_only: false
29     var_name: hsv_from_ # This is optional
```

Listing 5.1: Example of params_manager YAML configuration file [121].

Listing 5.1 shows also two other useful features of the params_manager package. The first is the possibility to specify a *validator* routine or a variable name: the autogenerated parameter update callback checks the name and type of a parameter and, if a match is found, may perform other operations before approving the update. One of these is to perform an additional check, whose outcome decides if the parame-

ter update is approved, which may be coded by the user in a function whose name is specified in the YAML file, informally referred to as a *validator*. Alternatively, to account for the necessity to quickly store the new value directly inside the node as explained above, the user can specify the name of a variable member of the node class that the callback will access and overwrite with the new parameter value. Of course, the two things can be done at the same time: it is up to the user in the first case to code the variable member update inside the validator routine. To keep track of each parameter's metadata together with these two additional values, the `params_manager` library uses the `std::set` data structure from the `<set>` C++ standard library header, which is implemented as a red-black tree for maximum efficiency [122].

After the `params_manager` library was implemented and successfully tested, the question arose whether it would be possible to eliminate the necessity of the user to write the five lines of code that instantiate the `Manager` object, call the `init_parameters` function in the node constructor, and then clear the `Manager` at the end. Hiding the management of the `Manager` would be possible in case the base ROS 2 node class that each node is derived from already included an object of the `Manager` class as a variable member. In the `dua_node` package [123], an alternative ROS 2 base node class is provided that includes the `Manager` object and handles its lifecycle together with that of the node. This way, the only necessary lines of code to manage parameters in a ROS 2 node are those that declare and call the `init_parameters` function; this is due to the fact that, by the current implementation, `init_parameters` is a member function of the derived class and can only be called by its constructor, after the base class one has run. The `dua_node` package is intended to be used in conjunction with the `params_manager` package, and is included in the `dua-utils` collection.

5.1.4 Wrapping clients: `simple_serviceclient`, `simple_actionclient`

Section 3.1.3 presented ROS, in both its first and second version, as a project aimed at providing a middleware to robotic software developers that offers many services, the most important of which is distributed communication. To this end, ROS 2 provides three communication primitives: topics, services, and actions. The last two are synchronous communication primitives, meant to implement client-server communication over a short time span in the former case, and a possibly longer one in the latter case, with the additional feature of being stateful. The ROS 2 middleware provides APIs to create and manage services and actions, which are grouped in the server and client sides. Servers are usually simple to implement: the user instantiates the server object and defines the relevant callbacks for it. In the case of actions there are usually multiple callbacks involved, to account for every possible asynchronous event requested by the client or terminal state (see Figures 3.8 and 3.9), but the developer can choose which ones to implement based on which features of the communication their code shall actually support.

For both services and actions, the client side is instead a completely different beast. In both C++ and Python, client-side APIs are based on asynchronous programming, which is a programming paradigm that allows the user to perform multiple tasks concurrently, without the need to wait for the completion of one before starting the next. This is achieved by using callbacks together with *futures*, which can be essentially described as variables whose value becomes available at a later point in time. Things get increasingly complicated as one steps from service clients to action clients, so this is the order in which the two situations are addressed here.

The developer writing code making use of a service client must first instantiate such object, then send a service request through it. The result of this API call is a future,

which will eventually hold the value of the service response. The developer can act in three ways here, all made possible by the ROS 2 API: they can block and synchronously wait for the future to get a value, they can attach a callback to the future that will be called when the value is available, or they can poll the future at any time they want in the code and check for its value. The API to manage each of these cases is not particularly complex, but it can get quite verbose. As for all the previous sections describing packages of `dua-utils`, the next one is motivated by a necessity that arises when one writes actual ROS 2 software: in the vast majority of cases, if you call a service, you need the response data right after, and you need it to be available before the next line of code is executed. This is why the first eventuality among the three described above is the most common, but it still requires the developer to write a lot of code to manage the future, put the node in a synchronous wait state by spinning an executor, and then get the value from the future. This is the reason why the `simple_serviceclient` package [124] was developed.

This package offers two equivalent libraries for both C++ and Python, which offer the same methods with the exact same signatures in both languages. The libraries implement a client class that wraps the original ROS 2 service client, extending its functionalities. Since the other situations are better handled with the official ROS 2 APIs, the `simple_serviceclient` libraries offer only two methods:

- `call_sync`: this method sends a service request and waits for the response, returning it when it is available. The method is blocking, and takes a flag argument telling whether the node must be added to an executor or not; this is to support the situation in which the node is already added to an executor and callbacks execute in separate threads, so waiting for the future consists in blocking the calling thread waiting for it to get a value using the future object API. In both

scenarios, the only code that can execute while the future is being waited for is node callbacks.

- `call_async`: this method sends a service request and returns a future that will hold the response when it is available. The method is non-blocking, and provided for convenience.

In C++, these libraries contain template classes whose template parameter is the type of service, so that the code can automatically handle the service request and response types.

The wider variety of cases that the stateful communication provided by actions defines is the main reason behind the complexity of its client-side. Essentially, each operation that the client requests to the server is implemented as a service call, and must be treated in the code in a way similar to what has been described before, *i.e.*, using futures and eventually callbacks. Note that even the most desirable outcome for an action from the perspective of the client, *i.e.*, completing a goal, requires two service calls: one to request the acceptance of the goal and one to request and get back the result (see, again, Figure 3.9). This easily leads again not only to a lot of boilerplate code, but also to complex code that is difficult to manage and debug. Moreover, there is an additional problem: the ROS 2 action API is not as well organized as the service API, due to its history. It has been one of the last features to be ported, regarding the communication services, from ROS 1 to ROS 2, and its implementation is as such split among two sets of libraries: the language client library, that is `rclcpp` for C++ and `rclpy` for Python, and the `rclcpp_action` and `rclpy.action` libraries. This dichotomy, still unresolved, has led to a situation in which the style of the client API might differ even significantly between its different parts, which reduces clarity and increases the learning curve for the developer. One of the most evident examples

of this is the numerous types that are involved in client-side action programming, which model goals, their handles, futures, feedbacks, etc., and their signatures do not follow a consistent style. The API is very powerful and allows one to achieve very complex asynchronous, event-based behaviors to handle scenarios of various degrees of complexity, but the problem is the same as before. In reality, there are mostly two ways in which actions are called: either the client sends a goal and waits immediately after for the result, or the client sends a goal and then either polls for the result or requests a goal cancellation. The `simple_actionclient` package [125] was developed to address these issues, providing an action client class that acts as a wrapper for the original action client. The package contains again two libraries, one for C++ and one for Python, whose contents and implementations are the same. For simplicity, the following explanation will refer only to the C++ library, which implements a template class.

The first big improvement of this class over the original API is that it completely hides all the type definitions that are required to handle an action client: the only template parameter is the action type. This leads to a very simple instantiation of an action client, where all type definitions are handled internally by the class. The class constructor can also take feedback callback as argument, to be called when feedback messages are received from the server. Then, the class implements three sets of methods, whose names are self-explanatory and that implement the most important moments of an action call lifecycle. For each, this library also provides both synchronous and asynchronous variants, which counts to six methods in total:

- `send_goal`: sends a goal to the server, and returns a future that will hold the result of the goal request.
- `send_goal_sync`: sends a goal to the server, and returns the result of the goal

request.

- `cancel`: cancels the goal that is passed as argument and returns a future that will hold the result of the goal cancellation.
- `cancel_sync`: cancels the goal that is passed as argument, and returns the result of the goal cancellation.
- `get_result`: requests the result of the goal that is passed as argument, and returns a future that will hold the result of the goal.
- `get_result_sync`: requests the result of the goal that is passed as argument, and returns the result of the goal.

Finally, the `simple_actionclient` library also provides a method to handle the entire lifecycle of an action call in one go, from the goal request to the reception of the result: the `call_sync` method. The signature of this method for the C++ library is shown in Listing 5.2. The first thing to note about it are its arguments: apart from the goal to send, it takes a flag and a series of timeouts. The flag tells the method whether to cancel the goal if a timeout occurs, and the timeouts are the maximum time that the method will wait for the goal request to be sent, for the result to be received, and for the goal to be cancelled. The method returns a tuple, implemented in C++ with the `std::tuple` standard library class, with three elements:

- a boolean flag that tells if the goal has been accepted or not by the server;
- a `ResultCode` object, which is an enumeration defined in the `rclcpp_action` library that tells the result of the goal request, *e.g.*, if the goal has been accepted, rejected, or cancelled;

- a shared pointer to the result of the goal, which can be valid or not depending on whether it was returned by the server, and that has type `ActionResultT` which is derived from the action type, the template parameter of the class.

Internally, the method performs each step by requesting each operation to the server and then synchronously waiting for the response, spinning the node. As the call progresses, the method checks for timeouts and cancels the goal if necessary, handling all possible outcomes and return values. When the return value of this method is accessed, by the way it is designed as a tuple holding the three values discussed above, the caller can immediately determine how the action call went since those three values together encode every possible outcome of the sequence of operations that the method performs. This method is the most powerful and flexible of the library, and is intended to be used in the vast majority of cases where an action is called. The other methods are provided for convenience, and to allow the user to handle each step of the action call lifecycle separately if needed. For more advanced use cases, the original ROS 2 API is again the best choice.

```
1 std::tuple<bool, rclcpp_action::ResultCode, std::shared_ptr<ActionResultT>>
2   call_sync(
3     const typename ActionT::Goal & goal_msg,
4     bool cancel_on_timeout = false,
5     int64_t send_goal_timeout_msec = 0,
6     int64_t get_result_timeout_msec = 0,
7     int64_t cancel_timeout_msec = 0);
```

Listing 5.2: Signature of the `Client::call_sync` class method from the C++ action client wrapper library `simple_actionclient_cpp` [125].

5.1.5 Implementing ROS 2 automata: `transitions_ros`

As illustrated in Section 2.1, the software that operates a complex robot typically follows a hierarchical organization. The lower levels usually consist of firmware, controllers, and device drivers. More advanced modules rely on the abstractions provided by these lower layers to perform sophisticated operations. At the top, a single module typically acts as a coordinator, ensuring the proper functioning of the entire system and offering interfaces to human operators. This module usually requires complete access to the software stack of the robot for control purposes, and its design must be sophisticated enough to handle the diverse situations that might arise during the execution of the prescribed tasks that the robot must perform. The higher the level of autonomy required, the more complex this kind of software is going to be.

Finite-state machines (FSMs) are a renowned tool in computer science [126]: their mathematical foundation derived from graph theory describes a system whose behavior is represented by a sequence of discrete configurations within a finite set, called *states*. These states are interconnected by a collection of *transitions*, which may lead to another state or loop over the same one. Transitions occur when specific conditions are met, which alter the current state of the machine. FSMs have been widely utilized in modeling various hardware and software systems, ranging from basic algorithms and communication protocols [127] to digital control systems [128, 129] and distributed systems [130].

Before moving forward, it is essential to clarify the type of FSM relevant to this use case. In software engineering, two main categories of FSMs are recognized. The first category, often referred to as a *passive* FSM, is the more commonly used of the two. This type mainly serves as an internal representation of the state of an object or algorithm, acting as a data structure that holds information about the current

state without performing any actions. Methods that interact with a passive FSM are called externally and their primary role is to keep a record of the execution state of the program based on the information encoded within it. The module presented in this section focuses on the second category of FSM, known as an *active* FSM. Unlike its passive counterpart, an active FSM is an autonomous entity that retains state information and actively executes operations on data and interfaces with other system components by calling appropriate APIs. An active FSM plays a crucial role in autonomous robotics by governing the behavior of the robot. This includes decision-making processes, executing movement commands, and interpreting data from its various subsystems. It is one of the most widely used paradigms for developing supervisory software for autonomous robots, as it provides a structured approach to managing the complexity of the robot's behavior; it is not the only one though, since its limitations start to show up when the automaton's complexity grows and a more modular and intrinsically nested structure would be preferable, such as behavior trees [62].

This discussion clarifies that, when looking forward to develop a supervisory software for an autonomous robot that can be modeled as a finite-state machine, an active FSM is the most appropriate choice. Moreover, if the software running on the robot is based on ROS 2, having an active FSM implementation which is designed to directly support ROS 2 communication primitives and other middleware functionalities would be highly beneficial. The `transitions_ros` package [131] was developed to address this need: it provides the implementation of an active FSM capable of seamless interaction with ROS 2 software, thereby enabling sophisticated autonomous control functionalities. Such an active FSM is intended to facilitate high-level autonomy in robotic systems, allowing them to operate effectively in dynamic environments by integrating sensory inputs, executing control logic, and interfacing with other

modules cohesively.

When dealing with a ROS 2-based robot, one can broadly isolate three types of tasks that an automaton must handle:

- **Asynchronous:** the robot must react to some external events that do not alter its state but require data processing, *e.g.*, detecting a target.
- **Synchronous:** the robot must execute operations specific to its current state, *e.g.*, arming, takeoff, landing, or exploring the environment.
- **Collaborative:** the robot must perform complex operations requiring real-time coordination with another agent or human operator.

Given the three communication primitives offered by ROS 2 (see Section 3.1.3), a logical approach for implementing a ROS 2-enabled FSM would be to handle asynchronous tasks through appropriate topic subscription callbacks. In contrast, services and actions would be ideal to manage synchronous and collaborative tasks. The FSM implementation should thus include a reference to a ROS 2 node object, acting as a communication endpoint towards the middleware. To ensure the FSM is active in the sense described above, each state should be associated with a specific *routine*, *i.e.*, a function executed upon entry to the state, whose return value determines the subsequent transition. Furthermore, the FSM object should provide a `run` method that, when invoked by a running thread, sets the automaton in motion within the context of the calling thread, and returns control only once a terminal state is reached and no further transitions are possible.

The `transitions` [132] Python library is one of the most famous and versatile open-source software libraries that implements a finite-state machine. It has many standard

features of software FSM implementations, such as support for hierarchical state machines and guard conditions evaluation. However, the only crucial feature here is the possibility of easily defining states and transition tables, operations that the library allows by specifying a dictionary for the former and a nested list for the latter. Strings identify both states and transitions by name, and the library supports regular expressions to make some definitions quicker such as, *e.g.*, catch-all transitions to an error state. With respect to the other use cases described so far in previous sections, the choice of this library which is based on the Python programming language has been motivated by its high-level nature, which reduces the time and code complexity required to encode and test even sophisticated routines, deeming it appropriate for a supervisor software module whose implementation should be as straightforward as possible.

The proposed `transitions_ros` library [131] starts by extending the `State` object of the `transitions` library, by deriving its class: each state must hold a reference to a ROS 2 node object, and a function to call as their state routine by the new routine method of the `State` class. Such a routine must be defined elsewhere in a software module that uses this library to build a specific automaton, must take a ROS 2 node as argument to interact with the rest of the software architecture, and must return a string that identifies the next transition that must occur based on the outcome of the routine itself; such string should be empty in the case of a terminal state. An example of transitions and state tables for the `transitions_ros` library is given in Listing 5.3. Next, the library extends the `Machine` object in a similar way: leveraging a specific use case of the `transitions` library, in `transitions_ros` a machine is created as an independent object that holds information about states and transitions, as well as a `run` method that sets things in motion. The `run` method, when called, executes the routine of the initial state, performs subsequent transitions, and calls new state

routines based on their string return value, stopping only when an empty string is returned by some routine which indicates that the automaton execution must stop. Finally, to receive and process the incoming data and requests of the node while the automaton is, *e.g.*, synchronously waiting for some task to finish, the Machine object offers the `wait_spinning` method that adds the node to an executor, spins it up to a configurable timeout, and then may perform further processing of new data.

```

1 transitions_table = [
2     ['START', 'INIT', 'WAIT_FOR'],
3     ['READY', 'WAIT_FOR', 'FIRST_TAKEOFF'],
4     ['AIRBORNE', 'FIRST_TAKEOFF', 'EXPLORE'],
5     # ...
6     ['MISSION_DONE', 'DONE', 'RTH'],
7     ['AT_HOME', 'RTH', 'COMPLETED'],
8     ['ERROR', '*', 'ERROR']
9 ]
10
11 states_table = [
12     {'name': 'INIT', 'routine': init_routine},
13     {'name': 'RTH', 'routine': rth_routine},
14     {'name': 'COMPLETED', 'routine': completed_routine},
15     {'name': 'ERROR', 'routine': error_routine}
16 ]

```

Listing 5.3: Example of transitions and state tables in the `transitions_ros` library [131].

A ROS 2 Python application using this library would have to, in order, define a ROS 2 node class with all the required interfaces, specify states and transitions tables, define routines for each state, then create both the node and machine objects and call the `Machine.run` method in a dedicated thread. The proposed implementation keeps the code concisely organized by design, allowing each of the definitions as mentioned above to be performed in a dedicated source file. To conclude, let us explain how this

FSM implementation would handle the classes of tasks mentioned before:

- **Asynchronous:** their triggering or processing can be encoded within topic subscription callbacks of the node, which may be executed during a call to `wait_spinning` or in a similar busy-wait context and may modify data that influences the automaton flow.
- **Synchronous:** these can be performed entirely within state routines by calling, *e.g.*, service or action clients of the node that invoke specific operations in the underlying software modules, and inspecting their results.
- **Collaborative:** these usually require a mixture of the two previous approaches, since the task execution might be synchronously coordinated by all parties but some of its conditions might arise asynchronously, *e.g.*, the mutual triggering of the task itself among multiple autonomous robots or the human operator.

Finally, note that the design of this active FSM implementation integrates very well with the client wrappers for services and actions presented in Section 5.1.4.

5.2 Developing robotic software with DUA

With the introduction of the `dua-utils` software collection in Section 5.1, the description of the Distributed Unified Architecture implementation and features is complete. To finalize also its overview, before discussing some results that led to its development, it is instrumental to see this framework in action. This section presents two software modules that are some of the most representative of the issues that DUA aims to solve, as well as of the functionalities that it provides to the developer to tackle them. Their choice is also motivated by the fact that they have been successfully employed

in real-world scenarios during the course of this work, some of which are described in the following sections. In both cases, the aim is not that of providing a complete description of the software modules which would be all but brief and outside the scope of this work, but rather to show how the DUA framework has been used to develop them and solve the integration issues that they presented; thus, an introductory description of their purpose and functionalities is given, followed by an explanation of how they have been configured as DUA units.

5.2.1 Autonomous flight: `flight_stack`

This project is a collection of software modules for autonomous drone flight and low-level control interfacing. By *autonomous drone*, in this context, we mean essentially a flying robot: its structure can either be that of a multicopter or a fixed-wing aircraft, or anything in between that flies, but it has no human operator for most of its operating time. Instead, the flight control and stabilization as well as planning tasks are entirely handled by onboard intelligence, in the form of both software modules and hardware components, *i.e.*, integrated computers and microcontrollers. The drone is supposed to be equipped with sensors of various kinds, such as inertial measurement units (IMUs), global navigation satellite systems (GNSS), LiDARs, and cameras, and actuators such as servos and motors.

Among the many kinds of autonomous robots, drones are one of the most complex to develop and operate. This is due to their nature, which requires them to operate in three-dimensional space, and to the fact that they are usually used in scenarios where the environment is not entirely known and can change rapidly. The greatest complexity is due to the fact that drones are inherently unstable systems, which means that they require continuous control to remain in the air and to perform the tasks they

are assigned. Such control is usually achieved by means of a control loop that reads sensor data, computes the necessary corrections to the drone's attitude and position, and sends commands to the actuators in the form of torques and thrusts. The control loop must run at a high frequency, usually in the order of hundreds of Hertz, to ensure that the drone remains stable and responsive to external disturbances; for this to work, the most important source of feedback for an autonomous drone is the IMU, which provides the drone's attitude and angular velocity at a frequency that can get as high as 1 kHz. This tight timing requirement implies that basic flight stabilization and control must be handled by a dedicated microcontroller, on which it runs as a task of some real-time operating system (RTOS) that ensures that the control loop runs at the required frequency.

One of the most famous projects that implements both low-level hardware and software for autonomous flight is the Pixhawk project [133, 134]. The Pixhawk project represents a significant open-source initiative aimed at establishing robust hardware and software standards for autopilot systems used in unmanned aerial vehicles (UAVs) and other robotic platforms. Initiated in 2011, Pixhawk emerged from a collaborative effort involving researchers, industry stakeholders, and hobbyists to create a highly versatile platform that can cater to both commercial and research applications. The central focus of the project is the development of a modular, adaptable standard for flight controllers that accommodate a diverse array of vehicles, ranging from small UAVs to advanced aerial robotics. By adhering to open hardware standards, Pixhawk ensures interoperability across various autopilot systems, thereby enabling manufacturers and developers to create devices that maintain consistency, reliability, and compatibility throughout the ecosystem. These standards are integral to ensuring that UAVs can operate safely and efficiently, supporting a wide range of sensors, communication modules, and flight peripherals critical for autonomous navigation and control.

Pixhawk hardware is characterized by its modular architecture, allowing for the seamless integration of different sensors, power systems, and actuators. The hardware supports multiple communication protocols, such as CAN bus, I2C, SPI, and UART, facilitating comprehensive data collection essential for autonomous decision-making and precise navigation. This flexibility allows developers to select the most appropriate hardware components for their specific use cases, thus promoting innovative and customized solutions across a broad spectrum of applications.

Integral to the Pixhawk ecosystem is the PX4 firmware [135], an open-source flight control software designed to operate on Pixhawk-compatible hardware. Initially developed by the Computer Vision and Geometry Lab at ETH Zurich, and later expanded under the Dronecode project [136], PX4 delivers a customizable and scalable low-level control architecture that can accommodate a broad range of platforms. The PX4 firmware framework is modular, supporting autonomous navigation, sensor fusion, control loops, and failsafe mechanisms, thus enabling precise and stable control over UAVs and making the system adaptable for applications ranging from hobbyist drones to industrial-grade systems. The PX4 firmware is built on a highly modular architecture, where different functional components are implemented as separate modules that can be enabled or disabled based on mission-specific requirements. This modularity allows the firmware to be optimized for diverse use cases, reducing computational overhead and supporting efficient operation on hardware with limited resources. Key features of PX4 include sophisticated attitude estimation, position control, mission planning, and collision avoidance, all of which are enhanced by advanced sensor fusion algorithms that synthesize data from multiple sensors to ensure reliable state estimation, even under challenging environmental conditions. Owing to strong community support, frequent updates, and extensive documentation, PX4 has become one of the most widely adopted flight control solutions in both research and

industry. Its application spans diverse domains, including agricultural monitoring, environmental data collection, search and rescue missions, and more advanced industrial operations. The synergy between PX4's firmware capabilities and Pixhawk's hardware standards has resulted in a thriving ecosystem that supports a variety of use cases, providing developers with a flexible, reliable, and cutting-edge solution for implementing autonomous systems which are recently spanning beyond the aerial domain.

Even if PX4 has evolved into a very powerful and flexible platform, there are some tasks that are still beyond its capabilities. Keep in mind that, to correctly perform its main task of basic flight stabilization, it must remain a real-time software that runs on dedicated hardware. This means that it cannot be used to perform high-level tasks that require more computational power, such as computer vision, path planning, or decision-making. This is why the most recent control architectures for autonomous drones usually include a *companion computer*, *i.e.*, a SBC that runs a full operating system and that is connected to the flight controller through a serial or Ethernet link. The companion computer is responsible for all the high-level tasks that the drone must perform, and it communicates with the flight controller to send it the necessary commands to perform the tasks. This way, the drone can perform complex operations such as autonomous navigation, obstacle avoidance, or target tracking, while still being able to fly and stabilize itself thanks to the flight controller. This situation is depicted in Figure 5.1.

The flight management unit (FMU) is usually capable of passing all the necessary data to the companion computer, such as the drone's attitude, position, and velocity, as well as the status of the sensors and the actuators. The companion computer can then use this data to perform its tasks, and send back to the FMU the commands that the

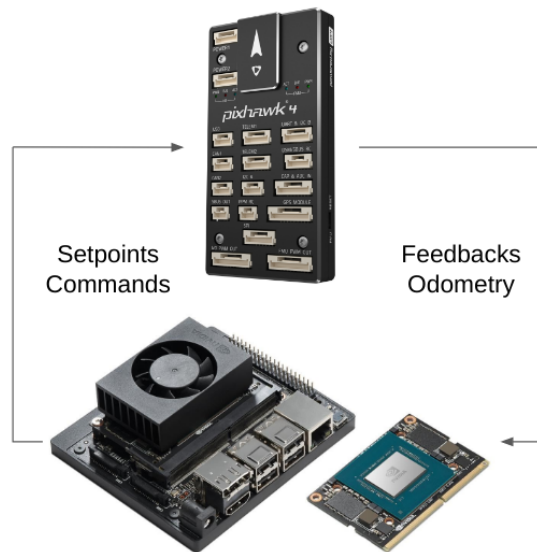


Figure 5.1: Functional diagram of a hardware architecture for autonomous flight: a Pixhawk 4 flight management unit (FMU) (top), and an Nvidia Jetson Xavier NX SBC (bottom).

drone must follow. From the point of view of the companion computer, the drone is essentially the FMU: there is no distinction, since all the actuation and sensing technicalities are addressed by the FMU, with which the software running on the companion computer must interface. Recalling the introductory discussion made in Section 2.1, since an autonomous drone can be viewed as an autonomous robot, it is reasonable to suppose that its software architecture still follows a hierarchical organization. The FMU offers just a set of communication channels over which data can be sent, but all of the technicalities involved in parsing this data, or sending the right commands and correctly parsing their feedback to request even a simple operation to it remain unsolved. Demanding this task to every single software module in the higher levels of the drone's control architecture would lead to a seriously flawed design, even just for a simple issue: if every module can talk to the FMU at the same time, who is actually controlling the drone at any given time? It appears clear at this point that, in such a complex system, an entire layer of the architecture must be reserved to modules that are in charge of interfacing with the FMU, abstracting the

complexities of the communication protocols and the data formats, and providing a clear and consistent interface to the higher-level modules while handling all the necessary synchronization and arbitration tasks. This is the role of the `flight_stack` [137] DUA unit, which offers these functionalities thanks to the ROS 2 middleware and the PX4 firmware. Currently, it supports only multicopters but support for different types of aircraft is an ongoing development.

The `flight_stack` repository contains five ROS 2 packages, the main three of which are called `px4_msgs`, `micrortps_agent`, and `flight_control`. Its current design is based on the PX4 firmware version v1.13.3, with which it is compatible, and has been derived first by forking packages with the same name originally maintained by a part of the PX4 development community. At the time of writing the 1.13.3 is not the most up-to-date version of the PX4 firmware but it is the one that has been employed in the applicative contexts of this work; updates of the supported firmware and subsequently to the `flight_stack` are planned in the near future.

The `px4_msgs` package comes first since it contains a set of message definitions that are used to communicate with the FMU in both directions. These definitions are crucial, since they correspond both in terms of fields and data types with the data packets that the modules of the PX4 firmware exchange internally among themselves. Thanks to this similarity, a module of the PX4 firmware easily serializes and deserializes packets coming from a serial UART link that connects the FMU with the companion computer. This module is called the `micrortps_client`, and it is one of the real-time tasks of PX4; its peculiarity is that it follows the same real-time data transmission protocol that is implemented as part of the DDS standard presented in Section 3.1.1. The other end of this communication is a task that runs on the companion computer, implemented as a ROS 2 node in the `micrortps_agent` package. The operational

task that this node performs is that of managing the serial interface connected to the FMU and interfacing it with the ROS 2 domain. For every different function of the PX4 firmware, there is a dedicated pair of message type and ROS 2 topic that the FMU either publishes or subscribes to. To make this possible, the `micrortps_agent` node publishes or subscribes to the same ROS 2 topics that it offers as communication interfaces towards the FMU to other ROS 2 nodes running on the companion computer, handling FMU data serialization and deserialization as well as managing the serial UART link. To efficiently interface itself with the physical layer, as well as with ROS 2, the `micrortps_agent` makes use of both ROS 2 RMW and bare DDS APIs at the same time, the latter of which are provided by the eProsima Fast DDS implementation [7].

Having available a set of ROS 2 topics that expose all the functionalities of the PX4 firmware solves only the first half of the problem: abstracting the serial communication link and its technicalities, like message queueing and flow control, translating it into a set of publish-subscribe communication primitives. What remains to be addressed is the abstraction of the whole drone, offering to the higher layers of the control architecture only a set of communication primitives which may even be more sophisticated than simple message topics, but still hide a lot of complications concerning the execution of elementary tasks that the drone must perform such as, *e.g.*, takeoff and movement. This is the purpose of the `flight_control` package, which is the core of the `flight_stack` DUA unit.

In the control architecture of an autonomous drone based on the `flight_stack`, the `flight_control` node is the first one that does not directly interface any hardware: to it, thanks to the first abstraction provided by the `micrortps_agent`, the drone appears as another node in the architecture that communicates over a set of topics. The first, basic functionalities that the `flight_control` offers are in fact based on

this communication primitive: other nodes are expected to exchange data directly with the `flight_control` using standard formats, and it will then translate them into formats that PX4 expects and forward the packets to the `micrortps_agent`, which is in charge of sending them to the FMU handling the serial link. Such first basic functionalities, all implemented with topics, include:

- Position control, which is implemented by forwarding local position setpoints to PX4, formatted as (x, y, z, ψ) where ψ is the yaw angle, $\psi \in [-\pi, +\pi]$: nodes can send a setpoint in the form of a ROS 2 message, and the `flight_control` will translate it into a PX4 message and forward it to the FMU; note that this is also how hovering of a multicopter is achieved, since the `flight_control` retains and republishes the most recent position setpoint at a frequency that avoids triggering PX4 failsafes.
- Velocity control, which is implemented by forwarding twist setpoints in body frame to PX4, expressed as $(v_x, v_y, v_z, \omega_z)$ where v_x, v_y, v_z are the linear velocities along the body axes and ω_z is the angular velocity around the body Z-axis.
- Local pose data forwarding, for navigation in GNSS-denied environments or when an additional source of local position and orientation data is available and must be provided to PX4 to be fused with the onboard inertial sensors. This data must first be transformed in a NED (North-East-Down) frame and then forwarded to PX4.
- PX4 local pose estimate republishing.
- Battery monitoring.
- PX4 log message monitoring.

More complex functions are implemented using services and actions, that other nodes in the architecture can call acting as clients to request them and wait for their completion, receiving feedbacks in the meantime. These functions include:

- Arming.
- Disarming.
- Takeoff.
- Landing.
- Basic movement to a specific position.
- Turn to a desired heading at the current position.
- FMU reboot.

For each of these functions, the `flight_control` action and service servers exchange data with specific topics handled by the `micrortps_agent`, gathering feedback and handling all possible outcomes and error conditions. The node also internally keeps track of the current operational state of the drone, *e.g.*, if it is armed or not, to allow every client to request only the operations that are possible in the current state. The `flight_control` node is also in charge of handling the synchronization of the requests that it receives, to avoid that multiple clients request conflicting operations at the same time, and to ensure that the drone is always in a consistent state.

The relevant portion of the `flight_stack` within the context of this work is its configuration as a DUA unit. It currently supports, and has been tested with, multiple targets ranging from SBC ones such as `jetson5`, to development ones such as `x86-cudev`. The unit configuration section of the Dockerfile for the `jetson5` target, intended

for Nvidia Jetson SBCs presented in Section 4.2.2, is shown in Listing 5.4. Since no firmware development must be performed on the SBC that is to be installed on the drone, the only configurations required concern the dependencies that are necessary for the `micrortps_agent` to work. These include some Python packages, as well as an installation of the Fast-DDS-Gen tool by eProsima [138]: this is required to generate IDL files that define messages for both the ROS and the serial side of the communications that the `micrortps_agent` must handle, since IDL definitions used for the serial link hold the same data types and fields but use different padding and serialization techniques to reduce the size of the packets that are sent over the link. Other SBC-oriented targets in the repository are configured in a similar way.

Development targets must add to the aforementioned dependencies additional setup steps that configure what is required to build the PX4 firmware. For reference, Listing B.1 in Appendix B reports the unit configuration of the `x86-cudev` target. Alongside the same steps performed in SBC targets, more dependencies are installed which are necessary to build the NuttX RTOS [139], which is the preferred real-time operating system to run the PX4 firmware on Pixhawk boards. Furthermore, since a cross-compiler is required to build the firmware in an x86 environment, version 9 of GCC [95] for ARM is downloaded and decompressed. Finally, version 2.10.1 of eProsima Fast DDS and its dependencies are downloaded and built from source, to provide support for the compilation of the PX4-side of the communication protocol that the `micrortps_agent` handles.

The `flight_stack` DUA unit is a clear example of how DUA can be used to solve the integration issues that arise when developing software for autonomous systems. Focusing in to the development targets, the initial dependencies are easily installed since they are distributed as packages for the system package manager, and thus may

be made part of the host system installation. The latter tools, however, must be built from source and then installed system-wide, a process that may only be scripted at best. In contrast, DUA automates these integration and setup steps in a way that is transparent to the developer who, to use this unit, only needs to add it configuring their project targets as described in Section 4.3.2. Furthermore, all these setup steps assume that the Linux distribution being employed is Ubuntu [28], which is the case inside DUA containers but may not be so on the host system.

```

1 # flight_stack START #
2 # Install PX4 general dependencies required by microRTPS Bridge
3 RUN apt-get update && apt-get install -y --no-install-recommends \
4     astyle \
5     libxml2-dev \
6     libgstreamer-plugins-base1.0-dev \
7     rsync && \
8     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/* /apt/lists/*
9
10 # Install PX4 Python dependencies required by microRTPS Bridge
11 RUN . /opt/dua-venv/bin/activate && yes | pip3 install -U \
12     future \
13     jinja2>=2.8 \
14     jsonschema \
15     kconfiglib \
16     lxml \
17     psutil \
18     pygments \
19     pymavlink \
20     pyulog
21
22 # Build and install Fast-DDS-Gen (required by microRTPS Bridge)
23 WORKDIR /opt
24 RUN git clone --recursive --single-branch --branch 'v2.4.0' \
25     https://github.com/eProsima/Fast-DDS-Gen.git && \
26     cd Fast-DDS-Gen && \
27     ./gradlew assemble && \
28     ln -s /opt/Fast-DDS-Gen/scripts/fastddsngen scripts/fastrtpsgen && \
29     cd .. && \
30     chgrp -R internal Fast-DDS-Gen && \
31     chmod -R g+rw Fast-DDS-Gen
32 WORKDIR /root
33 ENV PATH=/opt/Fast-DDS-Gen/scripts:${PATH}
34 # flight_stack END #

```

Listing 5.4: Unit configuration of the jetson5 target of the flight_stack DUA unit [137] as of November 26, 2024.

5.2.2 Driving a sensor: zed_drivers

One of the hardest problems that autonomous robots face is that of localization: when a robot is deployed in an environment, which could be either known, partially known or completely unknown, to be able to move it must first know where it is. This information is usually represented as a pose, which is a combination of the robot's position and orientation in the environment. Robots are equipped with sensors, whose variety ranges from inertial measurement units to cameras, LiDARs, and GNSS receivers. These devices provide data that can later be used, either directly or by appropriate post-processing algorithms, to obtain an estimate of the current pose of the robot. Providing a summary of the techniques employed in this field lies beyond the scope of this work, but it is worth mentioning that the most common approach is that of sensor fusion, which consists of combining the data from multiple sensors to obtain a more accurate and reliable estimate of the robot's pose.

Among the many classes of sensors, RGB^{IV} cameras are one of the most versatile and widely used in robotics. They can range from cheap and lightweight solutions including, *e.g.*, a single optic and producing only a video stream, to more expensive but sophisticated ones that include more optics and can provide additional information, such as depth maps. One of the most famous series of cameras aimed at robotics applications is the ZED series by Stereolabs [140]. The ZED cameras are stereoscopic cameras, *i.e.*, they mimic the human eye by housing two identical optics that are separated by a fixed, known distance. The sensors on each optic are synchronized, thus the frames they produce can be used to compute a depth map of the scene that the camera is viewing. This depth map can be then be expressed in a variety of formats, from grayscale images to point clouds, and can be employed in a variety of contexts,

^{IV}Red-Green-Blue, *i.e.*, cameras that operate in the visible range of the spectrum and produce color image frames.

from obstacle avoidance to 3D environment mapping. ZED cameras have emerged as a cost-effective solutions also due to their software support offered by Stereolabs: the ZED SDK [141] is a complete and versatile software development kit that enables programmers to easily interface with a ZED camera and get its data from a variety of programming languages and environments. One of the most famous disadvantages of working with images in real-time is the high cost in terms of computational power that is demanded to the host system: processing pictures usually requires a lot of memory and CPU power, and the more complex the processing, the more resources are needed. The ZED SDK, however, has been designed to take advantage of Nvidia GPU devices present on the host, being it either a desktop computer or an SBC: thanks to the CUDA parallel computing platform and its many hardware and software optimizations, the internal algorithms of the ZED SDK can run in real-time and produce high-quality results even on low-power SBCs and similar devices. Thanks to this feature, the ZED SDK integrates a positional tracking module that, if activated, provides a real-time estimate of the pose of the camera in the environment, with respect to a reference frame that is placed at the camera's starting position. This estimate is obtained internally by the SDK algorithms by fusing the data from the camera's optics and the IMU.

In the context of a robotic software architecture based on ROS 2, the integration of a sensor like a ZED camera requires the development of a specific module that abstracts it and publishes its data to the ROS network in standard formats. Such a module is usually called, in the ROS world, a *driver node*: note that it is not a device driver in the sense that this term usually implies in the operating systems world, but rather a software module that interfaces with a device and exposes its functionalities to the rest of the software architecture. It is called *driver* because its task is that of driving the device, *i.e.*, of making it work and of providing a clear and consistent interface

to the rest of the nodes in the architecture, who should not be concerned with the technicalities of the device itself. The code of the driver node would be based on the SDK of the device itself, if any, which is the ZED SDK in the case of the package discussed in this section: the `zed_drivers` package [142].

Like the other software modules discussed so far, `zed_drivers` is a DUA unit that contains multiple ROS 2 software packages. The main package is called `zed_driver`, and it is the subject of the current discussion. To briefly introduce it, let us first consider the task at hand that the node that this package compiles to has to solve: a ZED camera is a hardware device that must be first opened, specifying a set of configuration parameters for its various functions, then started, and finally its data must be read, processed, and published to the ROS 2 network. This is, in essence, a summary of the structure of the `zed_driver` node implemented by the package. At the time of writing, the `zed_drivers` unit supports and has been tested with version 4.1 of the ZED SDK and with the ZED Mini and ZED 2i cameras, but the support for newer ZED X series cameras should be ensured by the ZED SDK itself; testing on these newer platforms is planned in the near future.

At startup, the ROS 2 node parses its own parameters, eventually read from a configuration file: these parameters specify many configuration options of the supported functionalities of the camera, ranging from resolution and framerate, to depth sensing mode and depth map size, to IMU sampling rate; providing a full list here would again be outside the scope of this work. The node then initializes the ZED camera, setting it up according to the options that the parameters specified, and starts it. The node then enters a loop where it reads the data produced by the camera: one convenient feature of the ZED SDK is that when a new frame sample is made available to the program, *i.e.*, a new iteration of the loop begins, all the computations belonging to

the SDK including those running in GPU have been performed. The data, readily available to be read and used, can be divided in four main categories: RGB frames, depth maps, positional tracking data, and IMU samples. Each of these four sets of data must be processed before it can be published over the ROS 2 network, if anything it has to be copied by the program memory to properly-formatted ROS 2 messages that encode standard sensor data types. This process, especially on low-power devices like SBCs, can be computationally expensive. For this reason, the `zed_driver` node runs four parallel threads that handle each kind of data separately at the same time, optimizing the use of system resources to minimize latencies and ensure that the data is published before the camera produces the next frames, so that the node can keep up with the camera's framerate; the IMU thread, in particular, runs at a higher frequency sampling the camera IMU independently of the optics, since it runs at a higher rate. RGB frames must be converted to ROS 2 image messages, depth maps to point cloud messages, positional tracking data to pose messages and transformations, and IMU samples to IMU messages. The node then publishes these messages over appropriate topics.

To summarize the node's capabilities, here are the ZED SDK functionalities that it supports, each with a complete set of configuration options exposed as node parameters:

- RGB video stream.
- Depth maps and point clouds computation.
- IMU sampling.
- Pose estimation with the positional tracking module.
- Raw camera data logging in SVO file.

- Real-time video stream over the network with NVENC encoding [143].

To achieve this, the code of the node makes wide use of the ZED SDK C++ API. Coming back to the scope of this work, this means that a compatible installation of the ZED SDK must be present on the host system to which the camera is connected. According to the official documentation, setting it up is easy since it is distributed as a script that first install dependencies and then the SDK itself; this is true, supposing that the system on which the script is run meets a certain set of requirements: it must be a Ubuntu [28] system with some dependencies already in place, and it must have a Nvidia GPU with a version of the CUDA toolkit [108] installed which is also compatible with the version of the ZED SDK that one is installing. In the case of Jetson SBCs, things get a bit more complex because additional libraries, distributed by Nvidia via the JetPack SDK installation, are required. The `zed_drivers` DUA unit automates all these setup steps, making the installation of the ZED SDK and the configuration of the project repository a matter of minutes.

Before delving into two example unit configuration Dockerfiles, please recall that the system environments offered by DUA base units are stable and configured in a way to offer a good set of system packages that are ensured to be compatible with each other. Still, to install an SDK that is distributed as a self-installing script, one must do some level of work on each target to ensure that the installation runs smoothly in a reproducible way, and that it actually results in a usable SDK. This clarifies once more the necessity of a framework like DUA to cut down setup times while doing research and prototyping, and to ensure that the software that is developed is portable and reliable enough that once a prototype is ready for testing, any subsequent issue that arises can be traced back to the software itself and not to its system environment.

For reasons that should be clear at this point, the `zed_drivers` unit supports only

targets aimed at systems that may have an Nvidia GPU. Listing 5.5 shows the unit configuration for the `jetson5` target. In this case, the setup is pretty straightforward: after an initial set of system dependencies that the SDK installer necessarily expects, the ZED SDK installation script is downloaded and executed. Note that, due to a compatibility issue with an Nvidia library, the version that is downloaded supports a higher JetPack version than the one on which the base unit is based: this is one of the many kinds of integration issues that usually happen in this field and that, to be solved, might require many reinstallation or board reflashes. The DUA framework, on the other hand, grants the developer the possibility to make many quick subsequent attempts without the need to reflash the Jetson board each time, by simply modifying the Dockerfile and rebuilding the image. The only critical configuration step is then the creation of a fake `/etc/nv_tegra_release` file that the SDK installer expects to find to correctly identify the system on which it is running, which appears to be missing in Nvidia JetPack Docker images.

The unit configuration for the `x86-cudev` target is shown in Listing 5.6: surprisingly, this is more intricate than the one for the Jetson boards. First, due to other incompatibilities with the contents of the official Nvidia image, cuDNN [107] and cuBLAS libraries must be upgraded to the latest version, the latter of which even requires a manual installation from the contents of its system package. Then, finally, the ZED SDK can be installed by downloading its installer script and running it, requiring no additional steps.

At the end of this section, after these units have been discussed as examples of how the DUA framework can help solve integration issues and provide replicable and reliable development environments, it is possible to note that thanks to the contents of these units a working prototype of a complex autonomous system such that of

an autonomous drone could be set up in a matter of minutes: first, a DUA project would have to be created, adding the units to it and configuring supported targets as illustrated in Section 4.3.2. Then, the project can be deployed onto a compatible SBC to be installed on the prototype, and what is left to be done is just downloading the base unit image and building the target image on top of it. A development could then also be set up to build and flash the microcontroller firmware. The necessary software modules would then be ready to be built, configured, and run on the SBC, and the prototype could be tested right away.

```

1 # zed_drivers START #
2 # Install ZED SDK dependencies
3 # (Nvidia stuff should already be installed in the base unit)
4 RUN apt-get update && \
5     apt-get install -y --no-install-recommends \
6     apt-transport-https \
7     udev \
8     zstd && \
9     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
10
11 # Install the ZED SDK
12 # We currently support version 4.1
13 # Do some more black magic here to enable streaming features
14 # on Jetson inside a container.
15 # The syntax for the echo is:
16 # R${L4T_MAJOR_VERSION} (release), \
17 #     REVISION: ${L4T_MINOR_VERSION}.${L4T_PATCH_VERSION}
18 # NOTE: We install an SDK version for a newer version of JetPack
19 # than that of this base image, due to an incompatibility with the
20 # deprecated libnvbuf.
21 RUN echo "# R35 (release), REVISION: 2.1" > /etc/nv_tegra_release && \
22     wget --no-check-certificate -nc -O zed_sdk.run \
23     https://download.stereolabs.com/zedsdk/4.1/l4t35.4/jetsons && \
24     chmod +x zed_sdk.run && \
25     ./zed_sdk.run -- silent skip_python && \
26     rm zed_sdk.run && \
27     ln -sf /usr/lib/aarch64-linux-gnu/tegra/libv4l2.so.0 \
28     /usr/lib/aarch64-linux-gnu/libv4l2.so && \
29     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/* && \
30     chgrp -R internal /usr/local/zed && \
31     chmod -R g+rx /usr/local/zed
32 ENV LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/zed/lib
33 ENV LOGNAME=neo
34 # zed_drivers END #

```

Listing 5.5: Unit configuration of the jetson5 target of the zed_drivers DUA unit [142] as of November 26, 2024.

```

1 # zed_drivers START #
2 # Install ZED SDK dependencies
3 # NOTE: Here we require an upgrade to make sure
4 # we have the latest Nvidia libraries.
5 RUN apt-get update && apt-get install -y \
6   --no-install-recommends --allow-change-held-packages \
7   libcudnn8 \
8   libcudnn8-dev \
9   zstd && \
10  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
11
12 # Install libcublas-12-0 libraries manually
13 # to fix some incompatibilities in ZED SDK dependencies
14 WORKDIR /tmp
15 RUN apt-get update && \
16   apt-get download libcublas-12-0 && \
17   mkdir contents && \
18   dpkg-deb -xv libcublas-12-0_12.0.2.224-1_amd64.deb contents/ && \
19   mv contents/usr/local/cuda-12.0/targets/x86_64-linux/lib/* \
20   /usr/local/cuda/lib64/ && \
21   rm -rf contents libcublas-12-0_12.0.2.224-1_amd64.deb && \
22   ldconfig && \
23   rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
24 WORKDIR /root
25
26 # Install the ZED SDK
27 # We currently support version 4.1
28 RUN wget -nc -O zed_sdk.run \
29   https://download.stereolabs.com/zedsdk/4.1/cu118/ubuntu22 && \
30   chmod +x zed_sdk.run && \
31   ./zed_sdk.run -- silent skip_python && \
32   rm zed_sdk.run && \
33   rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/* && \
34   chgrp -R internal /usr/local/zed && \
35   chmod -R g+rxw /usr/local/zed
36 ENV LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/zed/lib
37 # zed_drivers END #

```

Listing 5.6: Unit configuration of the x86-cudev target of the zed_drivers

DUA unit [142] as of November 26, 2024.

5.3 Preliminary results

The DUA framework and the technologies presented so far have led to the development of a few prototypes during the time in which this thesis was drafted. These prototypes were developed either to perform specific tasks in real-world scenarios, or to test novel solutions in fields like robot autonomy and localization and mapping. The common ground of these prototypes is the use of the DUA framework in their development and deployment, which proved the effectiveness of the proposed solutions. This section presents some preliminary published results concerning the application of the technologies on which DUA is based to the field of autonomous robotics. Their success has brought to the formalization of the DUA framework as presented in Chapter 4, as well as to the solutions presented in the previous Section 5.1.

5.3.1 A ROS 2 distributed architecture for unmanned aircraft

In [67], a novel distributed architecture is designed and proposed for an autonomous drone, tasked with the execution of a target reconnaissance and precision landing mission in an indoor, GNSS-denied environment. The mission was part of the 2021 Edition of the Leonardo Drone Contest, a robotics competition hosted by Leonardo S.p.A. in which custom-built UASs have to perform a series of complex tasks autonomously, ranging from localization and flight control to environment mapping and autonomous navigation.

The research team from the Tor Vergata University of Rome participated in the 2021 Leonardo Drone Contest, which was conducted in Turin, Italy during September 2021. This competition featured six Italian universities deploying autonomous drones to execute a sophisticated mission within a structured indoor environment. The

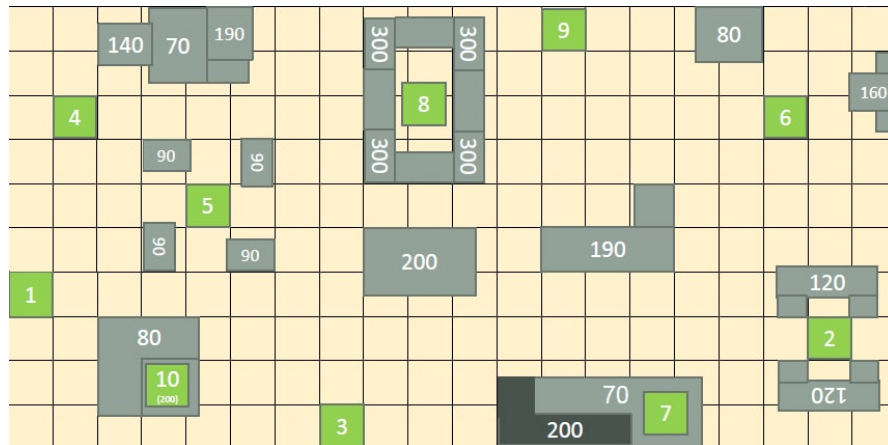


Figure 5.2: The 2021 Leonardo Drone Contest field layout.

operational field was meticulously configured with various elements, including 10 distinct landing pads and 3 mobile ground robots, with all participants receiving comprehensive details about the field layout prior to the competition. The visual markers demarcating the landing pads provided additional navigational guidance. A depiction of the 2021 field layout is shown in Figure 5.2: landing pads are in green and marked with their numerical ID, while known obstacles are in gray and their height in centimeters is reported.

The mission objectives for each drone were structured as follows.

1. Initiate take-off from a designated pad.
2. Conduct environmental reconnaissance to locate one of the three ground-based mobile robots.
3. Capture photographic documentation of the identified robot and transmit the imagery to an operator, who would subsequently communicate a specific landing sequence.
4. Execute precision navigation between pads, performing sequential landings

and take-offs according to the prescribed sequence.

Participants were allowed to use a Ground Control Station (GCS) for mission initialization and continuous drone monitoring. The landing sequence was strategically determined by an operator at the GCS to optimize the cumulative score, which was calculated through the summation of points assigned to each successful landing on a given pad. The point distribution was uniquely encoded on the ground-based robot, so it required initial identification.

The competition imposed rigorous technical constraints on all participating teams. Specifically, the use of GNSS platforms was prohibited, and Light Detection and Ranging (LiDAR) sensors were expressly forbidden. Additionally, teams were precluded from deploying any fixed ground-based sensors or motion capture systems, thereby challenging participants to develop innovative autonomous navigation strategies using the few remaining sensors which were not only allowed, but also capable of being mounted on a small UAS.

To facilitate the comprehensive mission objectives, a sophisticated hardware and software architecture was meticulously designed and implemented on advanced hardware. The autonomous quadcopter developed for the Leonardo Drone Contest 2021 represents a complex system that not only meets the specified requirements but also serves as an optimal test framework for the methodologies introduced in Chapter 3. The architectural design strategically partitions both hardware and software components across multiple hierarchical levels, addressing the distinct critical challenges inherent to each layer.

Nvidia Jetson Xavier NX	
CPU	NVIDIA Carmel ARM v8.2 (6C/6T, up to 1.4 GHz)
RAM	8 GB LPDDR4x
GPU	NVIDIA Volta (384 CUDA Cores, 48 Tensor Cores)
OS	Ubuntu Linux 20.04
Holybro Pixhawk 4	
CPU	STM32F765 (ARM Cortex-M7, up to 216 MHz)
RAM	512 KB
OS	NuttX 10.0.0

Table 5.1: Main technical specifications of the SBCs used in the implementation of [67].

Implementation details

The hardware architecture was composed by three distinct systems: a Holybro Pixhawk 4 flight controller (see Section 5.2.1), an Nvidia Jetson Xavier NX companion computer (see Section 4.2.2), and a PC acting as the Ground Control Station. The two embedded systems were mounted on the UAS prototype and interconnected as in Figure 5.1, which also depicts a functional diagram for the hardware architecture: the Pixhawk 4 flight controller is responsible for low-level flight control tasks, while the Nvidia Jetson Xavier NX companion computer executes high-level algorithms and mission-critical tasks and interfaces with the former to send commands and receive feedbacks and status data. A summary of the technical specifications of these two embedded systems is given in Table 5.1.

Figure 5.3 illustrates the comprehensive organization of software modules across functional layers.

- **Lowest Layer:** Comprised of real-time modules critical for executing fundamental flight operations, as well as device drivers.
- **Middle Layers:** Hosts higher-level, computationally intensive algorithms re-

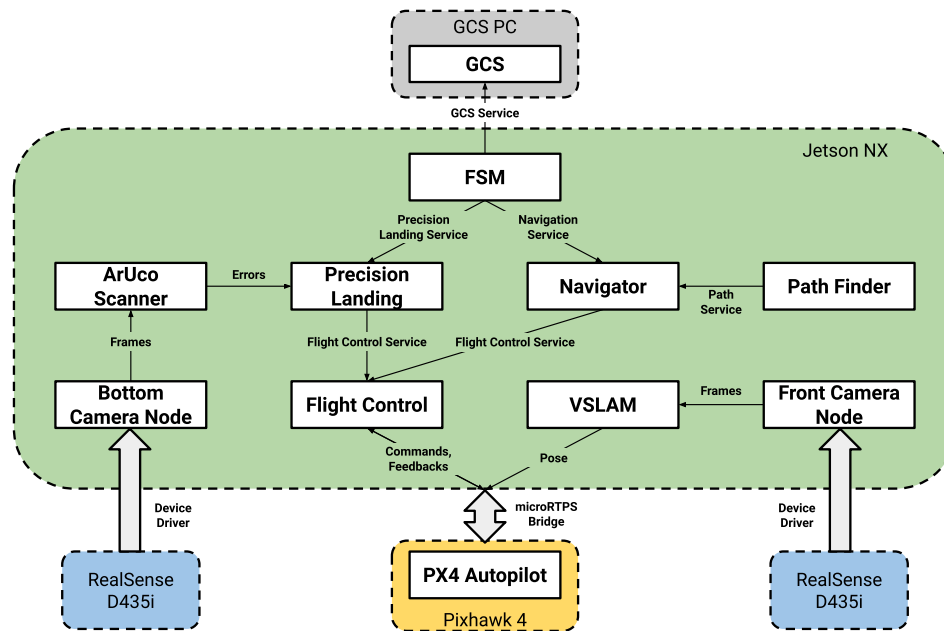


Figure 5.3: Software architecture of the LDC21 autonomous drone: thin arrows indicate communications established with ROS 2 interfaces, thick arrows represent dedicated software interfaces, white boxes represent ROS 2 nodes.

sponsible for executing individual mission tasks.

- **Highest Layer:** Dedicated to supervision logic, implemented as a Finite-State Machine (FSM), and the Ground Control Station (GCS) operator interface.

The architectural objective was to develop a cohesive set of ROS 2 modules that leverage interdependent functionality. This modular approach ensures vertical abstraction, enabling higher-level modules to operate without direct concern for lower-level implementation details such as take-off, landing, and navigational maneuvers. By abstracting these fundamental operations, the architecture promotes a clean, hierarchical system design that enhances modularity and operational flexibility.

Regarding the set of software modules running on the UAS onboard computers, the lowest layer was composed of the early version of what has become the `flight_stack` DUA unit presented in Section 5.2.1. This prototype was in fact running the very first

version of this software, and it has provided a testbed for the development of a bridged DDS-based communication among the two levels of the onboard architecture.

The remaining layers contain modules that concern the following specific tasks: perception, navigation and exploration, and mission supervision. Regarding perception, the prototype was equipped with two Intel RealSense D435i depth cameras: one was pointed forward and used for localization, while the other was pointed downward and used for target detection. Both mobile robots and landing pads were equipped with ArUco markers [144, 145], a particular kind of fiducial markers with limited information aimed at improving detection probability and pose estimation capabilities. The ArUco Scanner module was responsible for detecting these markers and transmitting detection data to the FSM module, that ran a state machine to manage the mission phases and the drone behavior.

To understand the environment and get a consistent estimate of its position and orientation within it, the UAS ran a VSLAM (Visual Simultaneous Localization And Mapping) algorithm. The SLAM problem is notoriously complex and computationally intensive to solve, requiring sophisticated solutions from both the mathematical and implementation perspectives. Nonetheless, it was deemed as the only solution in this case due to the Contest rules: no sensors other than cameras were reliable enough to provide the required information, and due to accumulated error it was not possible to rely on odometry alone. The chosen VSLAM algorithm for this implementation was the ORB-SLAM2 system [146], a state-of-the-art solution for monocular, stereo, and RGB-D cameras that provides real-time performance and high accuracy in both localization and mapping tasks.

The VSLAM module was responsible for running the algorithm and providing the estimated pose of the drone directly to the PX4 firmware. The ORB-SLAM2 system

employs binary feature extraction from input frames to construct a local map while addressing bundle adjustment optimization problems. To maximize computational efficiency and optimize memory utilization, these features are consistently employed throughout the entire system. The algorithm designates certain frames as keyframes, which serve as critical reference points for the local map structure. These keyframes are instrumental in computing the current local pose through the construction of a *covisibility graph*, which facilitates both localization and environmental mapping functions. As the map increases in scale, the system generates an additional structural representation known as the *essential graph*, which enables loop detection capabilities. Upon loop detection, the system executes an automatic loop closure procedure wherein all graphs undergo analysis and updating. During this process, redundant keyframes and nodes are eliminated, and the local map undergoes revision incorporating recent measurements to minimize systematic errors.

The architecture of the ORB-SLAM2 system comprises four distinct operational phases: tracking, local mapping, loop closing, and global adjustment. Each phase executes in a dedicated thread. The tracking phase monitors scene features, while local mapping utilizes these same features to construct the local map in relation to the established keyframes. The loop closing phase identifies potential loops and communicates this information to the global adjustment thread, which performs global bundle adjustment across the entire map to enhance overall accuracy.

ORB-SLAM2 supports multiple input modes: monocular, stereo, and RGB-D. The implementation used in this work introduces an additional mode that uses infrared and depth images, called IRD mode. This modification addresses specific technical limitations of Intel RealSense cameras, particularly the temporal misalignment between RGB and depth frames, while maintaining proper synchronization with in-

frared frames. Significant modifications to the ORB-SLAM2 algorithm have been implemented to expand parameter control capabilities, incorporate ARM processor compatibility, and integrate NVIDIA CUDA GPU optimizations. The adaptation for ARM architecture required the disabling of x86-specific optimizations, particularly affecting performance in feature extraction and computation phases that previously leveraged AVX/SSE vector instruction sets. To address these performance implications, CUDA GPU optimization became essential. Within the distributed architecture shown in Figure 5.3, this algorithm operates as a ROS 2 node, consuming camera topic frames and publishing both the current pose estimation and system state on separate topics.

The next task to address was that of autonomous navigation and exploration of the environment, to find ground targets and landing zones. The Navigator module was implemented for this purpose, with the aim of providing a high-level interface to navigation operations. These operations were exposed as ROS 2 synchronous communication primitives, *i.e.*, in the form of services: their refinement as more complex operations requiring a stateful communication was left as future work and developed for subsequent prototypes. The Navigator module was responsible for sending commands to the lower levels to move the drone from its current position to a given target position, provided that such a position was reachable and that there existed a path to reach it. This additional task of computing and returning paths was left to the Path Finder module: its primary objective was to determine an optimal trajectory between current and target positions that simultaneously minimizes distance while ensuring collision avoidance. The implemented solution approaches path planning through graph construction representing traversable space within the environment. Trajectory optimization utilizes the A* search algorithm [147].

The algorithm initially constructs a graph representation directly from environmental

point cloud data acquired via ORB-SLAM2. This process begins by discretizing the environment into a three-dimensional grid of variable-resolution cubic cells, each identified by its centroid. The interconnection network of traversable cells constitutes the graph structure. Boundary cells are automatically designated as non-traversable to constrain the UAS operational space. Environmental obstacle identification proceeds through point cloud preprocessing, beginning with the removal of out-of-bounds points. The algorithm associates points with their containing cells, designating cells exceeding a predetermined point density threshold as non-traversable. The remaining cells form the graph vertex set, followed by edge construction. For each cell, the algorithm identifies neighboring cells in eight coplanar directions at equal altitude, plus immediate vertical neighbors. Edge insertion between adjacent cells depends on a corridor analysis based on the physical dimensions of the drone: an edge is established if and only if the point density within the corridor falls below a specified threshold. This methodology enables variable-resolution environmental discretization while maintaining consideration for the physical footprint of the drone during transitions.

Valid corridors result in edge insertion with costs equal to the Euclidean distance between cell centroids. This iterative process yields a graph representation of the operational space, computed offline. The designated Path Finder ROS 2 node implements a computationally optimized A* algorithm to determine shortest paths between arbitrary nodes upon request, carried over a service interface. Path finding operations begin with origin and target cube identification and validation. Upon confirming traversability, the algorithm initiates shortest path computation, returning a sequence of waypoints leading to the target position. The selection of A* is motivated by its mathematical properties of admissibility, completeness, and optimality. As an informed search algorithm, A* leverages domain knowledge through heuristic guidance,

distinguishing it from other similar algorithms. In the context of three-dimensional navigation, the implementation employs Euclidean distance as its heuristic function, facilitating efficient minimum-distance path identification.

Supervision

The top level of the software architecture is a finite-state machine that implements high-level supervision logic. This FSM presents the drone as an automaton executing complete missions within the Leonardo Drone Contest parameters while managing contingencies such as hardware failures, battery depletion, and localization system degradation or complete failure. State transitions trigger formulation of requests to subordinate modules, with transitions conditioned upon both current state and intervening events.

The implementation integrates with the distributed architecture through a ROS 2 node that facilitates bidirectional communication with other modules via service calls and alert reception. System initialization comprises a comprehensive status verification of all modules through ROS 2 middleware APIs; mission execution begins only upon confirmation of full operational readiness across all subsystems.

The structure of the FSM mirrors mission dynamics as specified in the Contest rules. The implementation consolidates all abstraction levels into a single routine where state transitions are invoked based on the operational conditions of the drone following each execution step. Error handling leverages the meta-machine concept: primary mission phases execute within states comprising a subordinate FSM, while maintaining guaranteed transition paths to termination states under all circumstances. The high-level and low-level FSMs are shown in Figures 5.4 and 5.5.

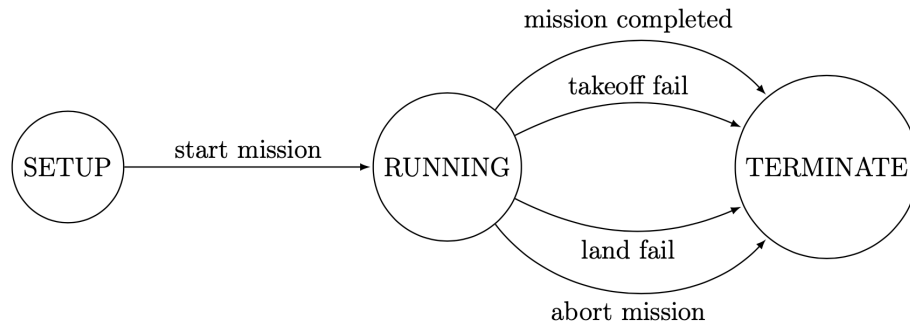


Figure 5.4: High-level finite-state machine for the Leonardo Drone Contest 2021 mission.

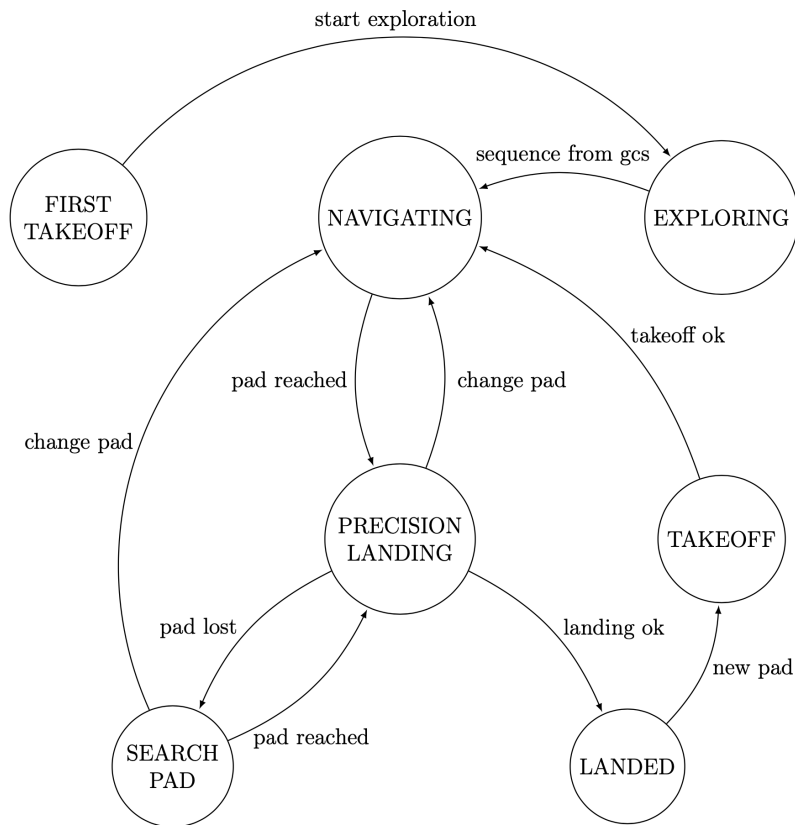


Figure 5.5: Low-level finite-state machine for the Leonardo Drone Contest 2021 mission.

Experimental results

The system hereby described has been tested, in both its single parts and its operational capabilities as a whole, also to evaluate the robustness of the proposed FSM.

Due to limited space available in the University facilities, laboratory tests allowed to perform partial evaluations, especially regarding the possible outcomes of the various phases of the mission. The complete verification of the functioning of all the modules and the FSM was performed during the 2021 Leonardo Drone Contest.

The first tests performed in the laboratory were aimed at assessing the capabilities of the VSLAM algorithm implementation. Performance evaluation of the custom implementation was conducted in an indoor office environment under variable illumination conditions to assess trajectory estimation accuracy. The evaluation methodology compared the estimated trajectory against ground truth measurements. Due to limitations in acquiring continuous ground truth data, the assessment utilized five discrete reference points measured using a laser sensor with a precision of ± 2 mm. The experimental hardware matched that of the UAS prototype as detailed previously. Figure 5.6 presents the experiment, depicting the ORB-SLAM2 estimated trajectory alongside the reference trajectory and ground truth waypoints where Root Mean Square Error (RMSE) calculations were performed. Table 5.2 quantifies the RMSE between ORB-SLAM2 estimated positions and corresponding ground truth measurements at the five reference waypoints. The analysis demonstrates that ORB-SLAM2 maintains functional performance on this novel platform architecture, despite its intrinsic differences from the original target platform, while indicating potential areas for further optimization.

Next, laboratory validation testing was conducted to evaluate the performance of the full system. The test sequence began with an exploration and perception phase, during which the system constructed an environmental map while localizing itself within it. An ArUco marker fixed to the ground, serving as a simulated robot target, was successfully identified during this phase. Upon completion of exploration, the mission

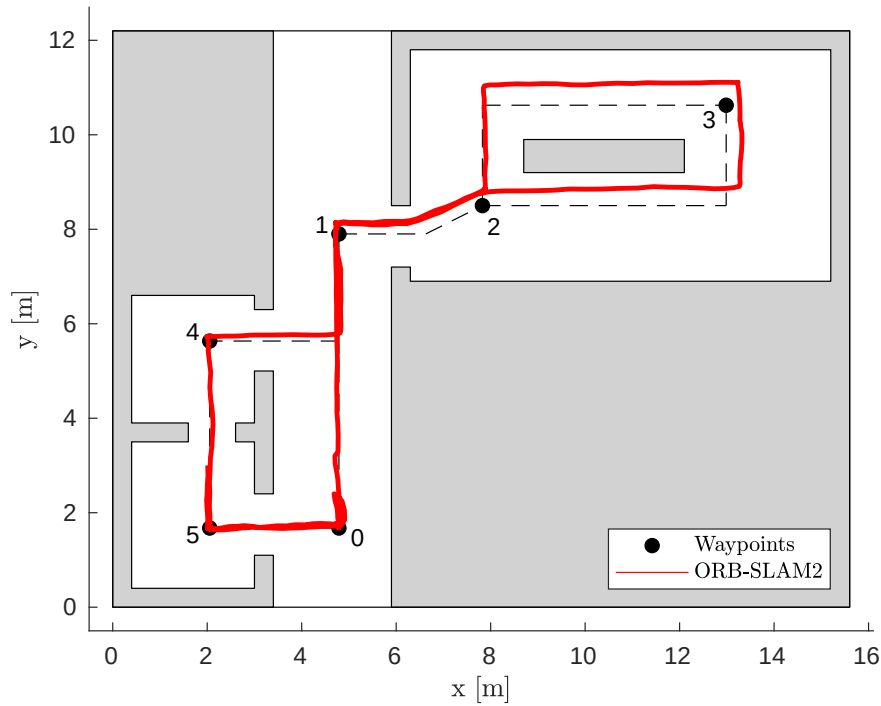


Figure 5.6: VSLAM algorithm performance evaluation test.

phase commenced, comprising navigation and landing operations targeting two designated pads. To validate the FSM error handling capabilities, the first target pad was intentionally removed, which triggered the execution by the FSM of a proximity search routine at the expected location. Upon failing to locate the first pad, the system proceeded to the second landing position, where the pad was successfully identified and a precision landing was performed on it. Figure 5.7 illustrates the complete flight trajectory. Despite spatial constraints of the laboratory environment, the facility proved adequate for comprehensive algorithm validation prior to Contest deployment. The test execution spanned 7 minutes and 33 seconds, covering 52 meters at a maximum velocity of 1.7 km/h. The trajectory demonstrates high precision with one notable exception: a significant oscillation near the conclusion of the initial exploration loop, attributed to computational resource saturation on the Xavier NX companion computer – an issue subsequently resolved. Quantitative performance

Waypoint position (x, y) [m]	ORB-SLAM2 position (x, y) [m]
(4.788, 7.901)	(4.718, 8.158)
(7.826, 8.501)	(7.925, 8.871)
(12.986, 10.626)	(13.329, 11.146)
(2.053, 5.634)	(1.980, 5.729)
(2.053, 1.676)	(2.034, 1.665)
RMSE	0.352 m

Table 5.2: RMSE between the ORB-SLAM2 estimated trajectory and the ground truth waypoints.

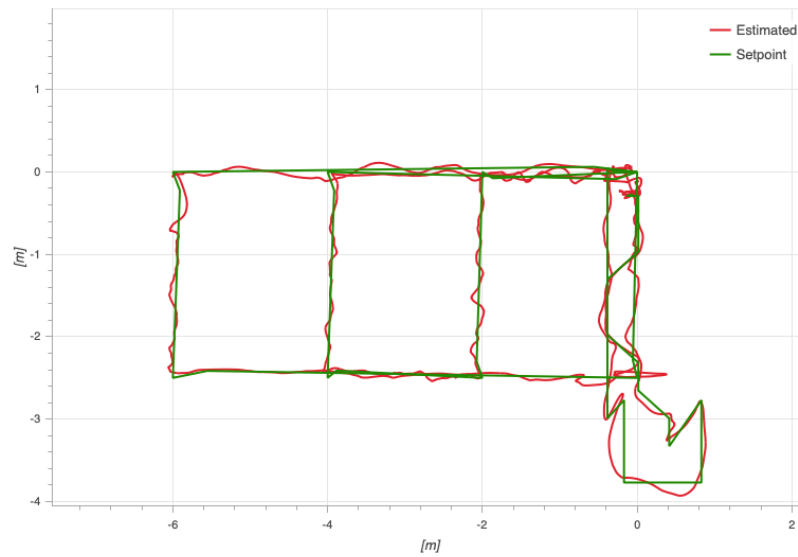


Figure 5.7: Laboratory test of the 2021 Leonardo Drone Contest mission; the green path represents the setpoints while the red one represents the estimated trajectory.

evaluation of the navigation subsystems was conducted through Root Mean Squared Error (RMSE) analysis along each spatial axis, with results presented in Table 5.3.

The Contest environment required extended flight durations to complete full mission profiles, with the quadcopter demonstrating 14-minute endurance capabilities while traversing 118 meters. The presented data corresponds to the fourth of the four Contest runs. This particular execution involved an abbreviated exploration phase, transitioning to mission operations immediately upon robot detection to commence the search for landing pads. Figure 5.8 illustrates the flight trajectory executed dur-

RMSE_x [m]	RMSE_y [m]	RMSE_z [m]
0.37	0.47	0.29

Table 5.3: Laboratory test RMSE (m) along each axis.

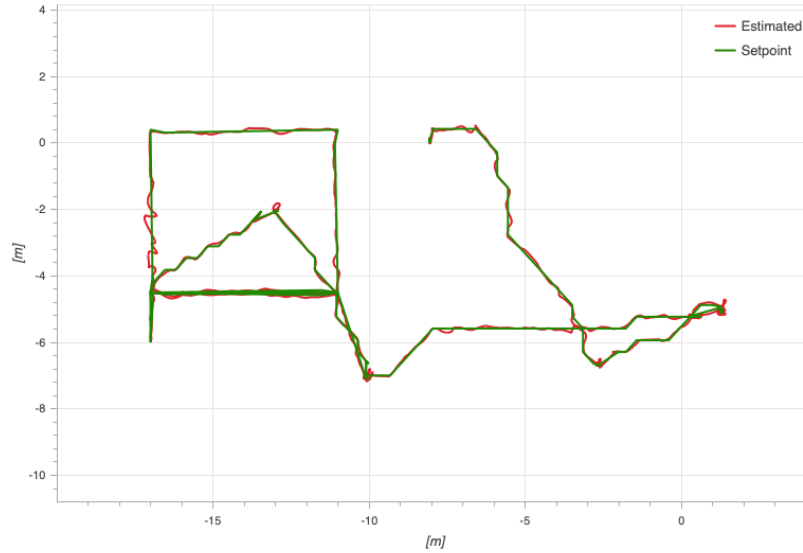


Figure 5.8: 2021 Leonardo Drone Contest mission run; the green path represents the setpoints while the red one represents the estimated trajectory.

ing the Contest run, during which five successful precision landings were achieved. Quantitative performance analysis yielded RMSE measurements across the trajectory as detailed in Table 5.4. The (x, y) planar error demonstrates superior performance compared to laboratory testing, attributed to immediate execution of the initial loop enabling ORB-SLAM2 to construct a reliable map with reduced temporal overhead. Conversely, the increased RMSE along the z axis reflects the increased magnitude of vertical maneuvers required during Contest operations.

Regarding the overall performance of the UAS prototype also in terms of its full software architecture, the implementation has been a success. The FSM has been able to manage the mission phases and the drone behavior, correctly transmitting data back to the GCS, which was an essential requirement for the Contest. Despite some technical issues that compromised the first two out of four runs, the team was able

RMSE_x [m]	RMSE_y [m]	RMSE_z [m]
0.21	0.24	0.38

Table 5.4: 2021 Leonardo Drone Contest run RMSE (m) along each axis.

to sort those out and the UAS prototype completed its mission in the last two runs. Thanks to extensive press coverage offered by Leonardo in 2021, Figure 5.9 shows the moments right before and after some of the most successful and exciting precision landings performed by the UAS prototype during the Contest. It is interesting to note how, once the procedure was started, the prototype always brought it to completion landing right within the central black portion of each pad, correcting localization errors by aligning its center with the ArUco marker on the ground. The Tor Vergata team has been awarded the second prize in 2021, paving the way for a successful streak in the following years: the 2022 edition has seen the team winning the second prize again, while the third prize has been won in 2023 together with the jury prize. In 2024, the team has finally won the first prize demonstrating also the effectiveness of the proposed DUA architecture, which in the meantime has been developed and adopted in this project.

5.3.2 Efficient sensor fusion for autonomous agents

In Section 5.2.2, the problem of pose estimation for an autonomous robot has been briefly presented. Its complexity has been outlined, together with the variety of sensors that might be used to achieve a reliable and accurate estimate of the position and orientation of a robot. The concept of *sensor fusion* has also been mentioned, as a family of techniques consisting in merging data coming from multiple, even heterogeneous sensors, to produce a single final pose estimate. The advantage of sensor fusion, over its increased complexity both in terms of mathematical formulation and

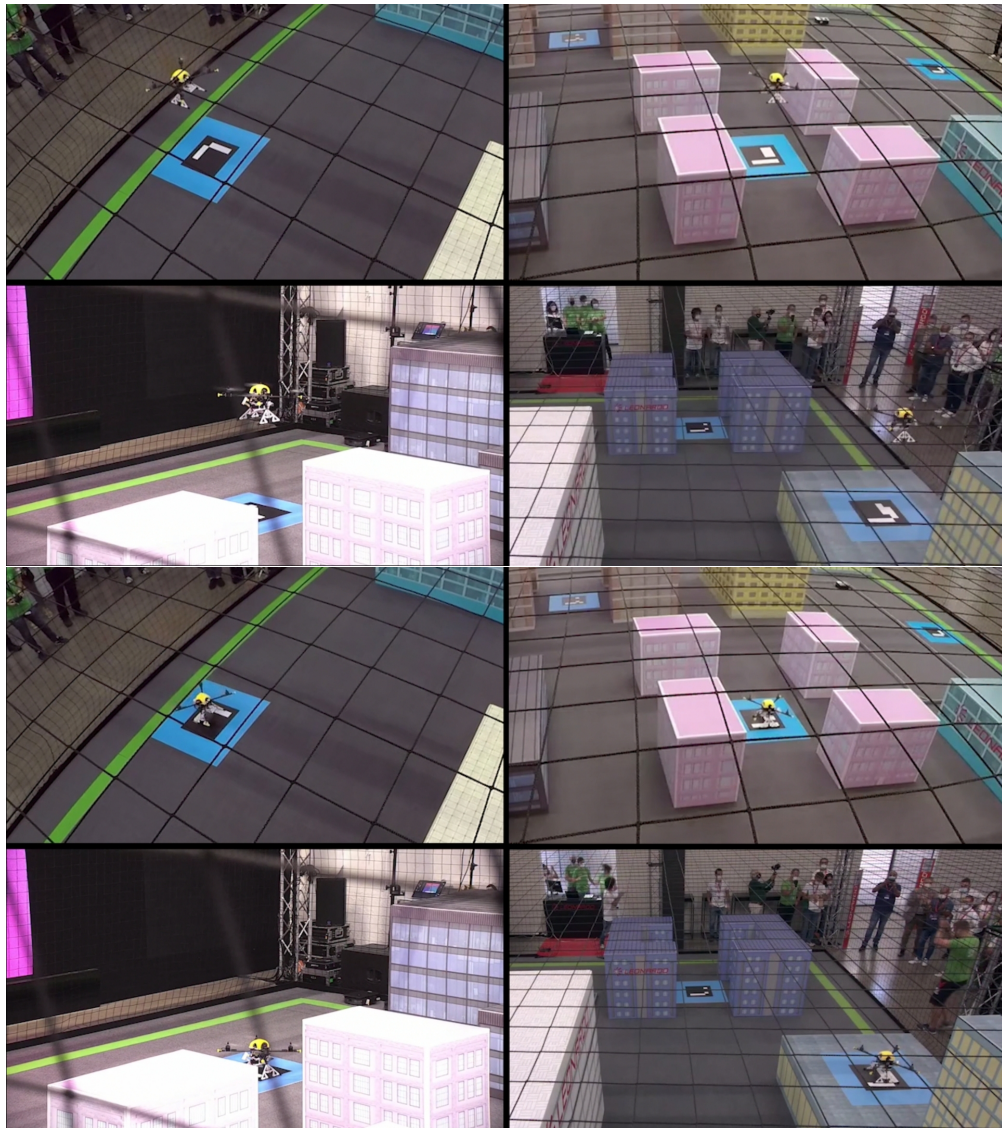


Figure 5.9: Successful precision landings performed on the ArUco landing pads during the 2021 Leonardo Drone Contest.

implementation, is the possibility to overcome the issues of one sensor by leveraging the other ones. Over the years, many sensor fusion algorithms have been proposed, each one with its own strengths and weaknesses, and each one suitable for a specific kind of sensor or application. Explaining the details of each one of them goes out of the scope of this work, but a good summary can be found in [68] together with a novel, synthetic approach to the problem.

In [68], the challenge of efficient position measurement fusion in a horizontal plane (x, y) is addressed for autonomous systems equipped with multiple visual sensors. The proposed methodology diverges from existing approaches by prioritizing reduced computational complexity and minimal sensor-specific information requirements. This design philosophy facilitates both rapid deployment on physical platforms and seamless integration of heterogeneous visual sensors within the system architecture, as validated through experimental evaluation. The fundamental research objective is to develop a computationally efficient sensor fusion framework that maintains high accuracy while minimizing implementation complexity and sensor-specific dependencies. This stands in contrast to conventional approaches that often require extensive computational resources and detailed sensor characterization. Describing this work is instrumental to this thesis since it has been carried out using a preliminary version of the DUA framework presented in Chapter 4, so the following discussion briefly summarizes the main results and then moves to the description of the implementation details.

The blending algorithm

Sensor fusion methodologies enhance measurement accuracy and reliability by combining data from multiple sources, achieving superior performance compared to individual sensor utilization. A fundamental challenge in sensor fusion implementation arises from the inherent diversity of sensor characteristics, with each device exhibiting distinct advantages and limitations that vary based on operational requirements and environmental conditions. Effective fusion systems must therefore incorporate mechanisms for dynamic sensor trustworthiness evaluation while maintaining operational capability during partial sensor failures.

Established fusion methodologies predominantly rely on precise characterization of data sources through covariance matrices, noise parameters, and mathematical models. However, accurate determination of these parameters presents significant challenges in practical implementations, particularly for real-time estimation from operational data streams. The accuracy of these parameters directly impacts the performance of estimation algorithms and, consequently, the robustness of the complete localization system. This limitation serves as the primary motivation for the proposed fusion architecture: to achieve enhanced robustness with respect to sensor-specific parameters while maintaining architectural simplicity. The framework proposed in [68] minimizes dependence on precise sensor characterization, thereby reducing implementation complexity while maintaining system reliability. This approach represents a departure from conventional methodologies that require extensive sensor modeling and parameter estimation. By reducing reliance on detailed sensor models, the system achieves improved robustness to parameter uncertainty while facilitating practical deployment in operational environments.

A detailed description of the mathematical foundation of the proposed method is out of the scope of this work, the interested reader can see [68] for a complete discussion. The aspects that concern this work, completely instrumental to the characterization of the implementation that achieved the experimental results presented in the paper, are limited to the structure of the algorithm. The algorithm, called *blending*, can be summarized as follows: first, new samples are gathered from every sensor, and their values are subtracted the previous sample to obtain an incremental measurement rejecting bias. Secondly, the incremental measurements are filtered using an appropriate method, to further reduce noise and reject outliers. Finally, the blending stage happens, where all incremental samples are combined to obtain a final increment to add to the previous one. The result of this last addition is the final position estimate,

which is then used to control the robot.

Formally, let one suppose that $n \in \mathbb{N}$ sensors are available, each one producing data at a same rate $f_s \in \mathbb{R}_{>0}$. The incremental, filtered sample at time $k \in \mathbb{Z}_{\geq 0}$ corresponding to each sensor may then be denoted by $\Delta_{i,f}[k]$ for $i = 1, \dots, n$. Each sensor also provides information about its functioning state, considered either on or off for simplicity and denoted by $s_i[k] \in \{0, 1\}$ for $i = 1, \dots, n$. Let $\bar{\Delta}_f[k] = [\Delta_{1,f}[k]', \dots, \Delta_{n,f}[k]']'$, $\bar{s}[k] = [s_1[k], \dots, s_n[k]]'$, and $\alpha_1, \dots, \alpha_n \in \mathbb{R}$ such that $\sum_i \alpha_i = 1$. Let $\mathfrak{B} : \mathbb{R}^{2n} \times \mathbb{R}^n \rightarrow \mathbb{R}^2$ be the *blending map* that produces the final increment sample. The blending stage of the algorithm is summarized by the following equations:

$$\mathfrak{B}(\bar{\Delta}_f[k], \bar{s}[k]) = \sum_{i=1}^n \bar{\alpha}_i[k] \Delta_{i,f}[k], \quad (5.1a)$$

$$\bar{\alpha}_i[k] = s_i[k] \alpha_i \left(1 + \frac{\sum_{j=1}^n (1 - s_j[k]) \alpha_j}{\sum_{j=1}^n s_j[k] \alpha_j} \right) \quad i = 1, \dots, n. \quad (5.1b)$$

The formulation presented in (5.1) exhibits simplicity while incorporating several advantageous characteristics. The parameters $(\alpha_1, \dots, \alpha_n)$ enable fine-tuned calibration to accommodate varying levels of sensor accuracy and reliability. The responsiveness of the method to sensor operational states facilitates resilient operation during both transient and permanent sensor failures, as demonstrated in the experimental validation discussed in [68]. From a computational perspective, (5.1) demonstrates high efficiency, as it reduces to a series of scalar products that can be executed with minimal computational overhead on standard microprocessor architectures.

The distributed implementation

The experimental platform developed in [68] has been designed to integrate only visual sensors, for both simplicity and because they are among the most subject to the

issues mentioned in the previous section. The chosen sensors were a ZED Mini camera (see Section 5.2.2) and an Intel RealSense D435i depth camera. The first, through an early version of the `zed_driver` package presented in Section 5.2.2, produced samples of its position estimate, while the latter was used to run the ORB-SLAM2 algorithm in the same IRD mode as described in Section 5.3.1.

The hardware architecture was again distributed, including a Nvidia Jetson Xavier NX SBC and an Up Squared x86 SBC; the former was responsible for driving the ZED camera, while the latter had the task of interfacing with the D435i and running the ORB-SLAM2 system. The choice of a x86 SBC to run ORB-SLAM2, in contrast to what has been shown in Section 5.3.1, was motivated by the possibility to leverage the vector instruction sets of the x86 architecture to speed up the computations of the algorithm in a way that is closer to its original implementation. The two SBCs were interconnected by an Ethernet connection, and all of the software modules were implemented as ROS 2 nodes. Finally, to better resemble the situation of an autonomous robot, a microcontroller was added, in the form of a Pixhawk 4 flight controller running PX4 (see Section 5.2.1). This FMU acted only as a microcontroller, receiving fused position estimates and interpolating them with data coming from its internal IMU. A summary of the technical characteristics of the hardware modules is given in Table 5.5.

The software framework used to manage the experimental implementation in [68] was a first, embryonic version of what has subsequently become the DUA framework presented in Chapter 4. There were, in fact, a variety of software tools involved, ranging from the ROS 2 middleware to the implementation of the ORB-SLAM2 algorithm and the ZED SDK (see Section 5.2.2). All of these software required their own dependencies, as well as specific setup steps that were slightly different on each of the two boards.

Up Squared	
CPU	Intel Pentium N4200 (4C/4T, up to 2.5 GHz)
RAM	8 GB LPDDR4
OS	Ubuntu Linux 20.04
Nvidia Jetson Xavier NX	
CPU	NVIDIA Carmel ARM v8.2 (6C/6T, up to 1.4 GHz)
RAM	8 GB LPDDR4x
GPU	NVIDIA Volta (384 CUDA Cores, 48 Tensor Cores)
OS	Ubuntu Linux 20.04
Holybro Pixhawk 4	
CPU	STM32F765 (ARM Cortex-M7, up to 216 MHz)
RAM	512 KB
OS	NuttX 10.0.0

Table 5.5: Main technical specifications of the SBCs used in the implementation of [68].

For this reason, Docker was introduced as a tool to manage the software dependencies and setup steps, as well as to ensure a consistent and reproducible behavior across all the boards and the development PCs. This allowed the developers to focus on the solution and implementation of the blending algorithm, streamlining the experimental validation presented in the paper. Different Docker images were created for each of the different platforms, including development PCs, which integrated all the necessary dependencies and software together. The techniques applied to the creation of these Docker images have led to the development of the base units of *dua-foundation*, introduced in Section 4.2. Then, the rest of the framework followed.

6 The case of tokamak devices

The previous chapter motivated the importance of the proposed DUA framework and of the technologies on which it is based in the context of autonomous robots. It clarified the relevance of the proposed solution in that context and provided some preliminary results that suggest its potential, as well as some practical application that benefited of the DUA structure. This chapter focuses on a different, yet related context: that of tokamak devices for nuclear fusion with magnetic plasma confinement.

From the point of view of a system architecture developer, these two scenarios are not so different: both involve complex systems with many interacting components and both require a high degree of autonomy. However, autonomy is the first item that differentiates these contexts: an autonomous robot, as discussed in Chapters 2 and 5, is a single entity that may have to operate independently for long periods of time, while a tokamak device may even surpass a robot in terms of complexity but it is an experimental device; this implies that it is not expected to operate without human operators involved and that its operational times might be shorter than those of a robot. Nonetheless, the various subsystems of a tokamak device are usually expected to operate autonomously during an experiment, thus the technologies and design paradigms discussed previously are still relevant. Among those, as the following clarifies, the distributed paradigm remains among the most suitable since the control system of a tokamak may be made of a variety of components that need to

communicate and coordinate with each other.

Tokamak device are another application scenario for the work developed for this thesis. This chapter presents the concepts of nuclear fusion with magnetic confinement and plasma shape control, then introduces the Tokamak à Configuration Variable (TCV) device that has been the subject of this work. Subsequently, two innovations are presented: an application of the distributed DDS middleware (see Section 3.1.1) to the TCV control architecture, and the dynamic optimization of the coil currents for plasma shape control, both made possible by the versatility of TCV's distributed control system.

6.1 Magnetic confinement fusion on TCV

Nuclear fusion occurs when light atomic nuclei combine to form heavier elements. This process powers stellar bodies and represents a promising avenue for clean, sustainable energy generation. These reactions, such as the fusion of Deuterium and Tritium nuclei to produce an α -particle and a high-energy neutron, release substantial energy with the potential for efficient power generation.

The development of fusion energy has acquired heightened significance in the context of contemporary global challenges. As nations worldwide confront climate change and seek to reduce greenhouse gas emissions, fusion technology offers a carbon-neutral alternative to fossil fuels. Furthermore, fusion power could address the growing energy demands of emerging economies while promoting sustainable development, as it requires minimal fuel resources and produces no long-lived radioactive waste like fission does. The abundance of fusion fuel sources in nature also promises enhanced energy security and reduced geopolitical tensions associated with conventional energy

resources. Moreover, nuclear power plants can be scaled to meet the demand of the territory in which they are located, without the need to identify suitable locations to harvest renewable but intrinsically intermitted sources of energy such as solar or wind.

To facilitate fusion reactions, the participating elements must be heated to temperatures sufficient for charged nuclei to overcome their repulsive Coulomb forces. At these elevated temperatures, the matter exists as fully ionized gas, termed *plasma* [148], which must be confined for durations adequate to achieve practical reaction probabilities.

Tokamaks [149] are experimental devices that have originated in the 1950s in the USSR; Figure 6.1 depicts the TCV machine, which has been the subject of this work. They employ toroidal vacuum chambers for plasma confinement using magnetic field configurations [150, 151, 152]. Dedicated coils encircling the chamber generate a *toroidal* magnetic field, while *poloidal* field coils produce an *axisymmetric* poloidal field. This poloidal field serves multiple functions: controlling plasma position and shape while inducing *plasma current*; the plasma must, in fact, carry a current that is instrumental to keep the fusion reaction going, as well as to maintain a reference temperature. Over the decades, tokamaks have contributed to study the behavior of plasma among different types of reactions, as well as to develop the technologies necessary to improve and maintain magnetic confinement. More recent devices, such as the ITER and the JT-60SA machines, incorporate the generation of energy into their project goals.

Plasma configurations, characterized by distinct cross-sectional geometries in the poloidal plane, exhibit varying properties regarding stability, performance, and interaction with vessel components. The plasma shape significantly influences the stability

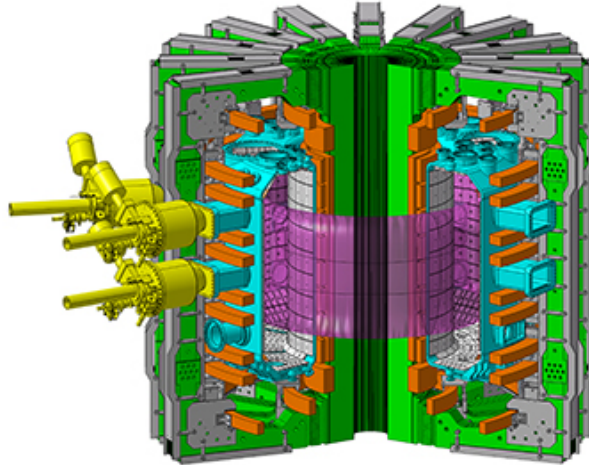


Figure 6.1: Schematic of the TCV device.

characteristics of global *magnetohydrodynamic* (MHD) modes and regulates particle transport phenomena. Furthermore, precise plasma shaping enables optimization of chamber utilization and plasma volume while facilitating controlled management of power deposition from plasma-wall interactions in regions where energy escapes the confined plasma core. To define a plasma shape and its position, two key elements are considered: the *centroid* and the *last closed flux surface* (LCFS).

The centroid is the center of the plasma cross-section along the poloidal plane, and can be either *magnetic* or *geometric*; the type of centroid to consider depends on the specific application at hand. The magnetic centroid is usually involved when it is convenient to approximate the plasma as a filament carrying the plasma current; in that case, the magnetic centroid corresponds to this ideal filament revolving around the vessel. The geometric centroid, on the other hand, can be considered when no such approximation is relevant and the entirety of the plasma cross-section within the vessel must be taken into account. When dealing with magnetic control, which is also the case of this work, the magnetic centroid is usually considered.

Poloidal field coils, running along the torus, generate a poloidal field that induces a

poloidal flux in the vessel. Along the plasma volume, there are points in which this flux has the same value. Examining the plasma from a poloidal cross-section, one can group these points by flux values, thus identifying *isoflux surfaces*. These run outward starting from the plasma centroid, initially as closed surfaces that enclose one another. The last closed one of these surfaces is identified as the LCFS.

Thus, controlling the plasma shape usually involves tracking reference setpoints for the centroid and the LCFS, expressed as a set of control points [153, 154]. The shape control capabilities directly impact operational parameters and plasma-material interactions, allowing for advanced scenarios that maximize performance while preserving machine integrity. This type of control represents a critical tool for achieving optimal plasma conditions while maintaining operational safety margins. To achieve different plasma configurations in terms of shape, the machine must be equipped with both a vessel large enough to contain them and with a set of coils that can generate the necessary magnetic fields.

The control infrastructure of a tokamak fusion device represents a sophisticated integration of diverse technological components operating across multiple temporal and spatial scales. The hardware architecture encompasses an extensive array of elements: high-power electrical supplies for magnetic field generation, Programmable Logic Controllers (PLCs) for real-time control and safety systems, analog control and diagnostic devices, and server-grade computing systems for data acquisition and processing. This hardware foundation must support both high-bandwidth real-time operations and complex experimental procedures while maintaining strict safety protocols.

The operational complexity extends beyond hardware considerations to include intricate control schemes for plasma magnetic confinement, comprehensive diagnostic

systems for plasma state monitoring, and experimental execution frameworks. These systems must operate in concert to maintain plasma stability while facilitating scientific investigation. The human interface component adds another layer of complexity, requiring control rooms staffed by teams of skilled operators managing different aspects of machine operation. The supervision infrastructure must provide interfaces ranging from detailed module-level control to high-level Supervisory Control and Data Acquisition (SCADA) systems, enabling both fine-grained control and system-wide monitoring.

Such operational sophistication demands a robust and well-structured architectural foundation. The control system design must accommodate real-time requirements, ensure fault tolerance, and facilitate system maintenance and upgrades. A distributed systems paradigm offers particularly compelling advantages in this context, providing natural solutions for parallel operation, redundancy, and scalability. This approach enables effective management of the inherent complexity of the system while maintaining the flexibility necessary for experimental physics operations.

6.1.1 The Tokamak à Configuration Variable

There are various dimensional parameters that can be used to evaluate the size of a tokamak device; one of these is the *major radius* R_0 , which is the distance from the center of the torus to the outer wall of the vessel. With a major radius $R_0 = 0.88\text{ m}$, TCV is classified as a medium-scale fusion research device. It is operated at the Swiss Plasma Center (SPC) of the École Polytechnique Fédérale de Lausanne (EPFL), Switzerland [155]. The device designation reflects its exceptional plasma shaping capabilities, enabled by an extensive array of independent Poloidal Field (PF) coils in conjunction with a distinctive rectangular vacuum vessel geometry, achieving toroidal

magnetic fields up to 1.54 T.

The PF coil system, illustrated in Figure 6.2, comprises four distinct subsystems.

- The **OH coils**, consisting of a central solenoid (OH1) and an up-down symmetric arrangement of 6 series-connected coils (OH2), provide plasma current drive functionality.
- The **E coils**, comprising 8 independently controlled circuits positioned on the high-field side.
- The **F coils**, featuring 8 independent circuits located on the low-field side.
- The **G coils**, consisting of two up-down symmetric in-vessel coils in anti-series configuration for vertical stabilization of the plasma.

The device architecture accommodates removable *baffles* [156] that enhance plasma core-divertor separation. The configuration depicted in Figure 6.2 shows the Short-Inner-Long-Outer (SI-LO) baffles arrangement.

The exceptional plasma shaping capabilities of TCV, combined with its continuously enhanced control architecture, have established this device as a significant contributor to international fusion research [157]. Through successive upgrades to both hardware and control systems spanning its three decades of operation, TCV has maintained its position at the forefront of experimental fusion science. Its contributions span multiple international research campaigns [158], including investigations of advanced plasma configurations and real-time control developments.

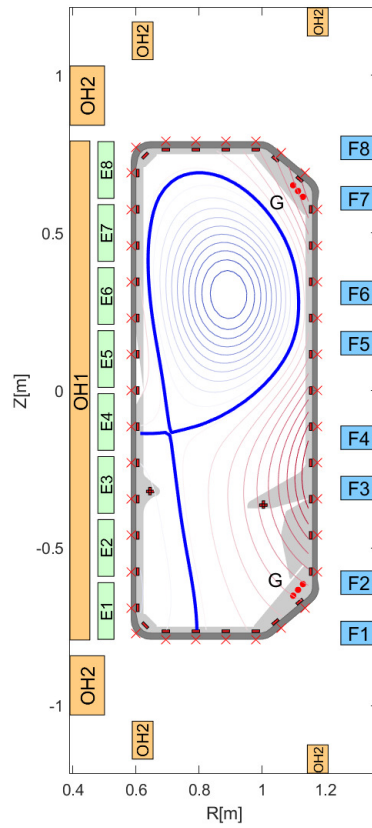


Figure 6.2: TCV poloidal cross-section with an example plasma configuration (lower single-null). The OH, E, F, and G coils are highlighted in different yellow, green, blue, and red, respectively. Removable baffles appear in the cross-section in gray protruding inward in the bottom part of the vessel.

6.1.2 The Système de Contrôle Distribué

Real-time control is a fundamental requirement in modern tokamak operation. The Système de Contrôle Distribué (SCD) [159, 160, 161] represents a distributed real-time control architecture designed for the TCV tokamak. The system emerged from the necessity to handle a considerable number of actuators and diagnostics while maintaining precise control over plasma behavior. Its distributed nature, of particular interest in this work, stems from the need to minimize analog signal paths across the

facility, employing real-time-capable networks to interconnect various subsystems.

The SCD must interface with hundreds of diagnostic channels while controlling multiple actuator systems. These include independently controllable coil power supplies, steerable real-time Electron Cyclotron launchers with continuously variable power, multiple neutral beam injectors, and an array of gas injection valves. Such an extensive set of controllable elements demands a sophisticated control infrastructure capable of coordinating multiple operations simultaneously.

A distinctive characteristic of TCV operation lies in the fast plasma dynamics, which imposes stringent timing requirements on the control system. Despite its relatively small size compared to future reactor designs, TCV requires primary magnetic control loops executing within 100 μ s. This temporal constraint represents a defining factor in the system architecture, influencing both hardware and software designs.

The SCD design philosophy emphasizes two seemingly contrasting requirements: operational flexibility and system reliability. The control infrastructure must facilitate rapid implementation and testing of novel control schemes while maintaining robust operation during regular experimental campaigns. This duality led to the adoption of MATLAB/Simulink as the primary development environment, enabling control algorithm prototyping through graphical programming and subsequent automated code generation.

The initial implementation of the SCD, now more than a decade old, required complete control code recompilation before each plasma discharge when modifying control parameters. While this approach supported the exploratory phase of control algorithm development, it proved increasingly difficult to maintain as the system expanded and the number of integrated modules and algorithms increased.

The transition to the MARTe2 framework (see Section 2.3.6) marked a significant improvement, introducing a modular, maintainable architecture with enhanced standardization capabilities. MARTe2, specifically designed for fusion applications, provides a comprehensive environment for real-time software applications. Its data-driven runtime workflow enables dynamic parameter and waveform loading during the initialization of a discharge, eliminating the need for frequent recompilations. This framework has also improved the integration between real-time computers and the TCV IT infrastructure, streamlining the deployment of new control algorithms from development to experimental validation.

The SCD hardware architecture comprises multiple computational nodes interconnected through dedicated networks. The main processing units consist of industrial PCs operating under real-time Linux, specifically customized with the PREEMPT_RT patch, CPU isolation, and specialized kernel parameters. These modifications enable deterministic operation while maintaining a conventional operating system environment, avoiding the complexity of custom-compiled kernels. A summary schematic of the system is given in Figure 6.3.

Input and output operations rely on high-density ADC/DAC modules. A dedicated reflective memory network (RFM) provides communication at 170 MB/s with sub-microsecond latency between computational nodes. This infrastructure is complemented by an EtherCAT real-time industrial network deployed to manage distributed low-count I/O subsystems, each handling up to 10 analog channels, with a total capacity of approximately 500 channels. The computational nodes, each performing specific functions in the control architecture, are organized as follows.

- **Node 01** operates the Far Infra Red interferometer (FIR) and related X ray diagnostics.

- **Node 02** and **Node 03** constitute the main control nodes of the system. Node 02 serves as the primary controller while Node 03 acts as its twin for debugging and backup purposes, *i.e.*, it runs the same code on the same hardware reading the same measurements but it is not connected to any actuator. These nodes manage all coils, gas valves, and auxiliary heating systems.
- **Nodes 05** and **Node 06** provide computational capabilities through 6-core Intel Core i7 processors. One unit incorporates a real-time network analyzer to monitor the performance of the reflective memory network.
- **Node 07** executes real-time MHD analysis, monitoring rotating tearing modes and locked modes.
- **Node 08** handles fast acquisition and real-time processing of Electron Cyclotron Emission data.
- **Node 09** manages real-time acquisition and processing of the Thomson scattering diagnostic, providing calibrated electron temperature and density profiles.
- **Node 10**, designated as MANTIS, operates a sophisticated 10-camera multi-spectral imaging system. It captures spectrally filtered tangential images of the plasma at frequencies between 200 Hz and 800 Hz, enabling real-time processing for applications such as radiation front position estimation along the divertor.

The primary control functionality, implemented on Node 02 with Node 03 as backup, represents a critical component of the system. This subsystem interfaces with a 192-channel ADC module, sending data streams through two high-speed fiber optic links to both the industrial PCs that implement the nodes. This redundant configuration

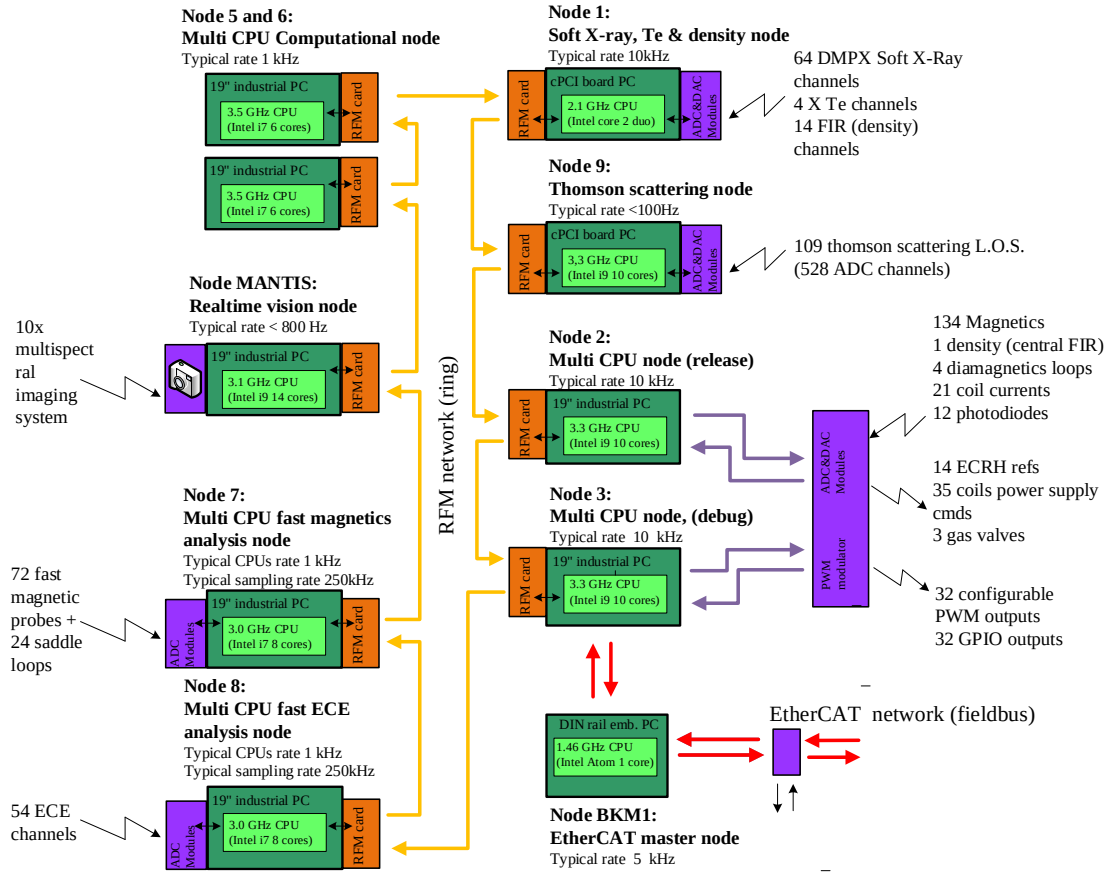


Figure 6.3: Schematic of the Système de Contrôle Distribué (SCD) architecture.

ensures synchronized data delivery to both nodes, enabling parallel operation during plasma discharges.

A central server orchestrates the entire control system, managing a main shot state machine that coordinates all distributed computers. This server maintains an MDSplus database [162] containing control system data and real-time algorithm configurations, hosts the development environment for control algorithm implementation, and provides a secure platform for authorized users to integrate new controllers into the TCV control framework.

The software development workflow employs an object-oriented simulation framework, designated as SCDDS (Simulink-based Control Design and Deployment Suite),

which manages parameter and waveform access consistently between simulation and real-time environments. After new algorithms have been developed with the graphical programming tools offered by MATLAB/Simulink, the SCDDS framework handles the complexity of parameter management, enabling developers to focus on control logic implementation. This approach proves particularly valuable when validating new algorithms using historical plasma discharge data. Once validated in simulation, control algorithms undergo automated code generation, producing C code that replicates the behavior of the original Simulink design. The SCDDS framework ensures a streamlined integration process within the MARTe2 environment, in which the entirety of the real-time control algorithms run to ensure that the system satisfies the stringent timing requirements in a deterministic fashion.

6.2 Improving the SCD network with the DDS

The strict real-time requirements of a sophisticated control system such as the SCD, described in Section 6.1.2, ask for an implementation that is designed and configured for the task at hand in any of its parts. Operating system kernels must be compiled with patches that enable fine-grained real-time task scheduling policies. The software must be written with frameworks that guarantee real-time execution, such as MARTe2. The hardware must be powerful enough to handle the computational load of the control algorithms, reserving to them all the system resources they require (*e.g.*, CPU, RAM, PCI bandwidth) in a deterministic fashion, cutting out every other task including system ones. When sampling times get to the microsecond precision declared in Section 6.1.2, every component of the system must be carefully considered.

In this scenario, things are a bit complicated by the fact that SCD is *distributed* in the sense that has been described in Chapter 1. It is not a single system but an intercon-

nection of multiple computational nodes each executing a predefined set of tasks encoded in specific software subsystem, also interfacing with dedicated hardware components when necessary. Each node cannot perform its tasks alone: for example, control nodes 02 and 03 cannot actuate the coil power supplies properly without the real-time data provided by the plasma equilibrium reconstruction, which runs on nodes 05 and 06, which in turn need the data from the plasma diagnostics running on other nodes. By this description, it is clear how the communication infrastructure among the nodes must be carefully designed and implemented to guarantee the real-time performance of the whole system. It must employ a technology that makes it fast, deterministic, and reliable.

Currently (see [161]), the SCD nodes are interconnected by a real-time fiber optic network that implements a *reflective memory* (RFM). In essence, reflective memories are shared memory systems that allow multiple machines to share a data area. The implementation of the RFM usually consists of a high-speed connection medium, in this case an optic fiber connection, and dedicated network cards to install on the machines forming the network. These cards contain the actual data area, and their device drivers map that area within the main memory of the host machine. The RFM implementation is such that each local copy of the shared data area is kept coherent with the others, so that every write operation is propagated to all the other copies. The timings that these operations might take can be very tight but, most importantly, they are deterministic. This makes the RFM a suitable technology for this kind of application scenario.

The RFM technology has been commercially available since the 1980s and SCD adopts Abaco Systems network cards [163]. As stated in Section 6.1.2, these cards and their fiber optic interconnection allow SCD to maintain sub-microsecond latencies when

transferring data among the nodes with a throughput up to 170 MB/s. This implementation has allowed SCD to perform well in the last decade but it is now showing some concerning issues, especially in terms of scalability and maintainability. First, as stated in [163], the Abaco cards are reaching their End Of Life (EOL) status with a limited production phase started in 2022. This, together with their high price (around 8.000€/card), is creating procurement concerns at SPC for such a critical component of the SCD. Secondly, the throughput of the RFM network is not enough to handle the increasing amount of data that the SCD nodes must exchange. New diagnostics are being evaluated and added each year, with some of them capable of generating considerable amounts of data, such as the MANTIS camera system on Node 10. The problem becomes clear when one notes that the data area is shared among *all* the nodes, so they have to use that space to exchange data coming from every device and subsystem all at the same time. Despite some access policies that have been implemented to deal with this without increasing latencies (see, again, [161]), a saturation point has been reached and new diagnostics such as bolometry cannot currently be used in real-time control because their data cannot be shared among the nodes.

To allow further upgrades to the SCD increasing TCV's experimental capabilities, as well as to avoid ending up relying on an EOL component, a replacement solution for the RFM network has become necessary. Various criteria have been applied to its selection, including the cost-effectiveness of relying again on dedicated hardware components when common network hardware such as Ethernet has evolved so much in the meantime, guaranteeing bandwidths that were simply unthinkable at the time in which the reflective memory was invented. Ideally, a properly configured, reserved Ethernet network is enough if it is driven by a software protocol that is able to guarantee the same determinism and low latencies of the RFM. The Data Distribution Service, introduced in Section 3.1.1, is a candidate technology that can provide these

features and has been selected as a replacement for the RFM network in the SCD.

This section describes the design of a software component for the SCD architecture that integrates the DDS middleware for communication purposes. Then, it presents some preliminary results of its application on SCD nodes, showing promising performance that motivated the decision to proceed with the integration of the DDS in the SCD. These results prove how the distributed technologies that constitute the base of the DUA framework presented in Chapter 4 can be applied successfully even to a different context such as the control system of a tokamak device.

6.2.1 Design of the DDSDataSource for the MARTe2 framework

As every hardware component of the SCD architecture, the reflective memory is integrated with it by the means that the MARTe2 framework provides. Recalling the overview of MARTe2 given in Section 2.3.6, the software components dedicated to the integration of specific hardware are called DataSources. The reflective memory makes no exception to this structure, since it is interfaced with MARTe2 by a DataSource called RFM2g, that can either read from or write to the shared data area. Thus, the end goal of the DDS integration process is the development of a new DataSource that replaces the RFM2g one, called DDSDataSource. Similarly to the RFM2g, the DDSDataSource may act as either a publisher or a subscriber on a given topic. The following description of the design of the DDSDataSource is largely based on the characteristics of a DDS implementation, which have been illustrated in Section 3.1.1, and has been presented in [164]. The DDSDataSource class is a C++ class that inherits from the DataSourceI interface of MARTe2, which is the base class for all the DataSources in the framework. Thus, all the DDSDataSource features hereby described are implemented within the methods of this class, that the MARTe2 framework calls at

specific times during the execution of the real-time loop.

Before the DDSDataSource features can be presented, it is instrumental to address the three main problems that had to be solved to finalize its design. Two of them are mainly related to the DDS implementation, while the third directly concerns the SCD use case. As the following discussion clarifies, the DDS middleware offers a vast set of configuration options and capabilities but the SCD use case requires a little subset of them which can ensure efficient and reliable operation. The chosen DDS implementation for this work is eProsima Fast DDS [7].

How to discover other entities?

One can recall (see Section 3.1.1) that the DDS standard specifies the Discovery Protocol to automate the discovery and configuration of all DDS entities in the network. Such entities include both DDS participants and the DataReaders and DataWriters that share the actual data being transmitted. By default, this protocol operates with multicast IP transmissions requiring no manual configuration.

In a distributed, large scenario such as a swarm of autonomous robots, being able to automate the discovery phase is a great advantage. However, in the SCD case it might not be necessary and even counterproductive. First, the SCD network is much smaller than that of a swarm of robots, with currently 10 nodes that are using the DDS over an isolated, dedicated 1 Gbps Ethernet network where all the hosts are known and preconfigured. Consequently, the number of topics and thus that of expected communication endpoints is also relatively small, especially compared to that of the complete software architecture of a sophisticated autonomous robot equipped with multiple sensors, actuators, and software modules.

For this reason, the Discovery Protocol feature of the DDS standard is not required

in the implementation of the `DDSDataSource`. Instead, it is preferred to expose manual configuration options of the DDS network settings, to avoid the overhead of the discovery phase and let the SCD developers have full control over the network configuration. In the DDS implementation, the network settings are handled by the `DomainParticipant` object, which has to be unique for every application that uses the DDS. The `DDSDataSource` is a software component implemented as a C++ class, thus multiple instances of it may be present inside a single MARTe2 application. For this reason, the `DataSource` implementation automatically instantiates a single `DomainParticipant` for the entire MARTe2 application, in a way such that new `DataSource` instances retrieve a reference to the already existing `DomainParticipant`. This feature is completely transparent to the developer using the `DataSource`, who only has to specify the network settings that are then applied process-wide through the `DomainParticipant`.

What kind of messages to exchange?

A great advantage of the DDS middleware over the RFM is that while the latter offers a single data area shared among all the nodes with a fixed size, the former allows the definition of multiple topics, each with its own dedicated data channel that can be configured to have a specific format. Nodes can subscribe and publish to the topics they require, even concurrently: this implies that the data that can flow over the network is much more than that shared by the RFM. This approach is more flexible and allows the nodes to use up to the whole bandwidth of the Ethernet network dedicated to DDS communications. As illustrated in Section 3.1.1, the middleware allows to define the data structure of the messages that are exchanged over the network. In the case of SCD, considering all the different diagnostics and actuators, there would be many different structured messages to define. Due to time constraints (a replacement

for the RFM had to be quickly developed), such a sophistication is not required at this stage. Currently, the `DDSDataSource` is designed to exchange a single message type, which is a byte array that can contain any kind of signal. In MARTe2, outgoing signal data passes through memory exchange components, called IOGAMs, before it reaches the `DataSource`; there, multiple signal samples can be packed into a single byte array which then becomes a `DataSource` message. The same holds for data coming from the network: a byte array coming from the DDS layer can be unpacked by an IOGAM. The specific instructions to pack and unpack the signals are given in the `cfg` of the MARTe2 application.

How to exchange data with the SCD framework?

The last critical issue to address concerns the expected behavior of a data exchange involving the `DDSDataSource` within a MARTe2 application running on SCD. This is due to how the RFM2g `DataSource` to be replaced currently behaves. Data exchanged over the reflective memory network can be grouped in two distinct classes, differentiated by their purpose:

- data that is required for offline, slow reconstruction of the plasma equilibrium, sampled at lower rates and with the possibility of higher latencies;
- data that is required for *real-time fast control*, sampled at the highest rates allowed by the SCD.

The perspective that better highlights this problem is that of a RFM2g `DataSource` that reads data from the shared memory area. Since the reflective memory has an internal clock, and because MARTe2 threads have to execute a real-time loop at a fixed rate, the `DataSource` component can synchronize itself with the hardware and ensure

that the thread rate is the same as the RFM clock. Ideally, since the RFM satisfies the timing requirements of magnetic control, this is feasible. In practice, it has been found that this is not the case: there are situations in which the RFM network jitter occasionally becomes non-negligible, causing real-time threads that run at its rate to miss deadlines. This may have critical and potentially destructive repercussions on the plasma equilibrium, as the control algorithms may not be able to actuate the coils in time to stabilize the plasma. The solution to this problem, currently implemented in the RFM2g DataSource, is to handle the two situations described above in two different ways, corresponding to two different operation modes of the RFM2g DataSource acting as a reader.

Synchronized The reader DataSource is synchronized with the RFM clock and reads data at the highest rate allowed by the network. This mode is used for data that is not involved in fast control, since the RFM jitter may cause the DataSource to miss deadlines. This also implies that, if new data is produced at each RFM clock cycle, the DataSource returns a new data sample at each iteration of the real-time thread loop.

NotSynchronized The reader DataSource is not synchronized with the RFM clock and reads data at a rate specified by some other component running in the real-time thread (*e.g.*, a LinuxTimer DataSource). This rate can be set equal to that of the RFM clock but the DataSource is not bound to it, allowing it to deterministically respect fast control deadlines. A direct consequence of this behavior is that, if new data is produced at each RFM clock cycle, the DataSource returns a new data sample only when the RFM clock cycle is completed, else it returns again the last valid sample. Clearly this makes the DataSource act as a sort of sample-and-hold device, but repeating a sample occasionally does not

cause problems in the SCD control loop in practical scenarios and is preferred to missing deadlines.

To streamline the integration of the new `DDSDataSource` in the SCD, it has to provide this same behavior.

6.2.2 Features of the `DDSDataSource`

As any `DataSource`, the `DDSDataSource` can handle data in two ways: it can either publish data to the network or subscribe to data coming from it. Thus, it can be configured to act either as a `Publisher` or as a `Subscriber`; sample MARTe2 configurations for each are shown in Listings 6.1 and 6.2 respectively. This first configuration option corresponds to the `Mode` parameter. The rest of the configuration options are described below, and constitute the features of the `DDSDataSource`.

Each `DDSDataSource` instance has a specific `Name`, which is a character string set by the developer in the configuration file that uniquely identifies that `DataSource` in the network. This parameter is not required by the DDS implementation but by a functionality of the `DDSDataSource` itself, termed the *handshake protocol*. During the shot initialization phase, the SCD state machine initializes all the nodes and their software components and awaits for their readiness. The `DDSDataSource` instances, when started, provide a similar acknowledgement to the state machine so that the shot routine proceeds only when all communication channels and related DDS entities have been correctly initialized. To do so, the `DDSDataSources` synchronize with one another by transparently exchanging messages containing their names with a `TRANSIENT_LOCAL` durability policy (see Section 3.1.1). This is a feature that is not present in the `RFM2g DataSource` and that is entirely made possible by the DDS middleware. The `NamesToMatch` parameter is a string containing a semicolon-separated list

of names of other `DDSDataSource`s with which the `DDSDataSource` must synchronize, awaiting for their presence message before completing its own initialization.

The `DDSDataSource` exchanges data over a specific network interface, which is specified by the `NetworkInterface` parameter: a string containing either the character representation of the NIC IP address or its system name. The `TopicName` parameter is a string that identifies the topic to which the `DDSDataSource` is associated. Transmissions belonging to that topic must observe a specific Quality of Service policy on all the nodes that interact with it; such a policy can be specified by setting the `ReliabilityKind`, `HistoryKind`, and `HistoryDepth` parameters. These parameters specify the reliability of the data transmission, the kind of buffering that the DDS middleware must apply to the data, and the depth of the buffer, respectively (see, again, Section 3.1.1). Note that they have to match among configurations concerning `DataSources` operating on a same topic, else the corresponding DDS entities will not match and the communication will not be established. The `MaxSamples` parameter specifies the maximum number of samples that the DDS middleware implementation can handle: in a real-time framework such as MARTe2, this setting is used to preallocate the memory that will be used to store the samples, thus avoiding dynamic memory allocations during the execution of real-time loops.

The `ExecutionMode` parameter concerns the behavior of the `DataSource` with respect to the previous RFM2g implementation, as discussed in Section 6.2.1 where the effect of this setting on the `Subscriber DataSource` has been described. Regarding the implementation of this mechanism, the `Subscriber DataSource` always stores the last valid sample received from the DDS layer, returning either it or a new one depending on its `ExecutionMode` setting and the availability of new data. However, this setting also has a meaning for `Publisher DataSources`: in this case, the DDS implementation

can spawn a dedicated thread to handle data writes to the network layer (*i.e.*, system calls that are part of the transport protocol OS API). Thus, the two possible behaviors for a Publisher are as follows.

Synchronized When writing data to the network layer, the DDS API blocks the calling thread until the data has been effectively passed to the transport layer. This is the preferred behavior in the MARTe2 framework, since it guarantees that the data is effectively transmitted to the network before the real-time loop proceeds to the next cycle.

NotSynchronized When writing data to the network layer, the DDS API does not block the calling thread and returns immediately. The data is just copied to a buffer that is then processed by a dedicated DDS implementation thread. This is the default behavior of the DDS implementation since it reduces application latencies, but it may not be suitable for fast real-time loops.

The last configuration parameters of a `DDSDataSource` concern real-time scheduling of internal threads within the MARTe2 application. For each `DomainParticipant`, the eProsima Fast DDS implementation spawns the following three threads.

- A thread handling asynchronous events.
- A thread handling the reception of data from the transport layer.
- A thread handling the transmission of data to the transport layer.

These threads are always necessary, since they handle both DDS control transmissions and actual data packets. This also clarifies why it is preferable to have a single `DomainParticipant` for every application on a single DDS domain, as mentioned

in Section 3.1.1. In the MARTe2 framework, every thread must have a specific real-time scheduling policy. Usually, this task is handled by the MARTe2 scheduler in a way that is completely transparent to the developer, so there is no way to let the MARTe2 scheduler handle these three threads. In compliance to what is done in the core MARTe2 library, these threads are assigned a `SCHED_FIFO` scheduling policy with a priority level of 75. The `SenderThreadCPUMask`, `EventsThreadCPUMask`, and `ReceiverThreadCPUMask` parameters are hexadecimal numbers that specify the CPU cores on which the threads are allowed to run. These parameters allow the developer to specify the CPU affinity of the DDS threads in a similar way to what is done for the MARTe2 threads.

Note that on every SCD node runs a single MARTe2 application, which can contain an arbitrary number of `DDSDataSources` that share the same `DomainParticipant`, thus the same three internal threads. This means that the price to pay to replace an expensive, EOL, obsolete hardware component with a software component that is more flexible and scalable is three CPU cores per node plus a few Ethernet cables.

The `Signals` section of the `DDSDataSource` configuration specifies the MARTe2 signals that the `DataSource` handles, in either the send or receive direction. To transmit a variable-length byte array, providing at the same time some basic diagnostic information, the following two signals are defined.

- `MsgCounter` is a signal of type `uint32` that contains the number of messages that have been exchanged by the `DataSource` since its initialization.
- `DataBuffer` is a signal of type `uint8` that contains the actual data to be exchanged over the network. It is a one-dimensional array whose dimension must be set by the developer in accordance to the resulting size of the packed array of

signals that is produced, or expected, by the IOGAMs.

```
1 +DDSPublisher = {
2     Class = DDSDataSource
3     Mode = Publisher
4     Name = "Pub05"
5     ExecutionMode = Synchronized
6     NetworkInterface = "eno1"
7     TopicName = "rfm_1"
8     HistoryKind = KeepLast
9     HistoryDepth = 10
10    ReliabilityKind = Reliable
11    MaxSamples = 1000
12    SenderThreadCPUMask = 0x4
13    EventsThreadCPUMask = 0x8
14    ReceiverThreadCPUMask = 0x10
15    NamesToMatch = "Sub06"
16    Signals = {
17        MsgCounter = {
18            Type = uint32
19            NumberOfDimensions = 1
20            NumberOfElements = 1
21        }
22        DataBuffer = {
23            Type = uint8
24            NumberOfDimensions = 1
25            NumberOfElements = 1024
26        }
27    }
28 }
```

Listing 6.1: Sample configuration settings of a Publisher DDSDataSource.

```

1 +DDSSubscriber = {
2     Class = DDSDataSource
3     Mode = Subscriber
4     Name = "Sub06"
5     ExecutionMode = NotSynchronized
6     NetworkInterface = "eno1"
7     TopicName = rfm_1
8     HistoryKind = KeepLast
9     HistoryDepth = 10
10    ReliabilityKind = Reliable
11    MaxSamples = 1000
12    SenderThreadCPUMask = 0x4
13    EventsThreadCPUMask = 0x8
14    ReceiverThreadCPUMask = 0x10
15    NamesToMatch = "Pub05"
16    Signals = {
17        MsgCounter = {
18            Type = uint32
19            NumberOfDimensions = 1
20            NumberOfElements = 1
21        }
22        DataBuffer = {
23            Type = uint8
24            NumberOfDimensions = 1
25            NumberOfElements = 1024
26        }
27    }
28 }

```

Listing 6.2: Sample configuration settings of a Subscriber DDSDataSource.

6.2.3 Preliminary benchmarking results

As illustrated in [161], the machines currently implementing SCD nodes form a heterogeneous set, spanning from recent servers to hardware that is even a decade old. This is due to the fact that the SCD has been developed over the years and, during each

improvement, continuous operation of the TCV machine had to be ensured, deferring the upgrade of some of the most critical systems.

The eProsima Fast DDS implementation version used in this work is 2.14, which requires a Linux kernel 5 or later, and a GCC 11.4 toolchain supporting the C++17 language standard. For this reason, at the time of writing, the `DDSDataSource` has not yet been tested during a real experiment on the TCV machine. For this to happen, the nodes have to be fully upgraded replacing the oldest machines, a process that is currently ongoing. However, the `DDSDataSource` has been tested in a laboratory environment, on the newer machines that are shortly going to replace the older ones. The results of these tests are promising and show that the DDS middleware can be successfully integrated into the SCD architecture.

Two tests have been performed, one for each of the two operation modes of the `DDSDataSource` described in Section 6.2.1. The first test has been performed on a Publisher `DDSDataSource` that sends data to a Subscriber `DDSDataSource` running in `Synchronized` mode, while the second one involved a Subscriber running in `NotSynchronized` mode. In both situations, data has been sent from one machine to another over a dedicated 1 Gbps Ethernet connection, with a payload of 4 kB corresponding to a page of main memory; this payload was always passed to proper IOGAMs, so the cycle times also involve significant memory copy operations. The `DDSDataSource` data exchange was the only task that the real-time threads had to perform, so that cycle time data gathered during the tests could be used to fully evaluate the latencies introduced by the DDS middleware.

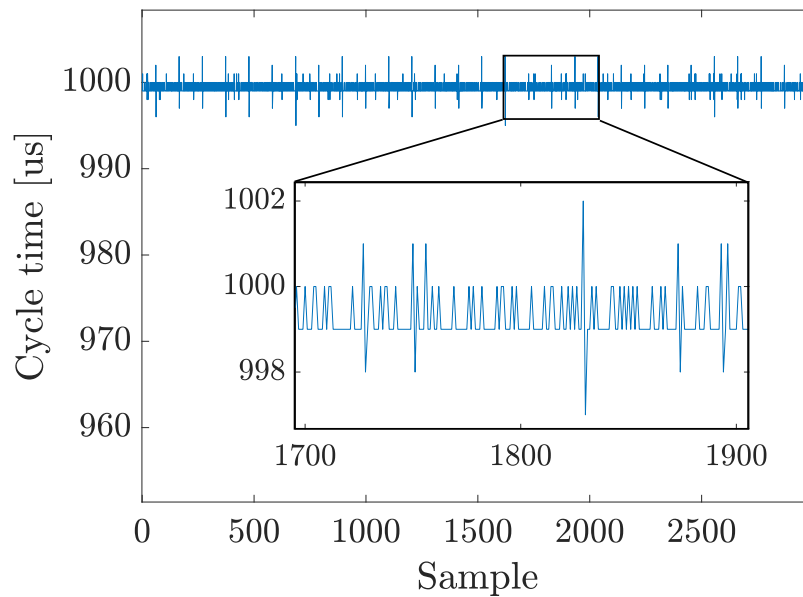


Figure 6.4: Steady-state cycle time of the real-time thread containing a Publisher DDSDataSource running in Synchronized mode.

Publisher side

The Publisher ran in Synchronized mode during both tests because, even if this feature is also available to publishers, it is not desirable in this context to let an internal DDS thread buffer write operations to the transport layer. The steady-state cycle time of the real-time thread running the Publisher DataSource is shown in Figure 6.4. The thread runs at a rate enforced by a LinuxTimer DataSource, which is configured to run at 1 kHz. As Figure 6.4 shows, there is some overhead due to the DDS write operations towards the transport layer, but its maximum entity of 3 μ s is negligible compared to the overall cycle time. Thus, the publisher side of the DDSDataSource is suitable for fast control applications.

Subscriber side in Synchronized mode

The steady-state cycle time of a real-time MARTe2 thread running the subscriber DDSDataSource in Synchronized mode is shown in Figure 6.5. Recall that in this mode the DataSource is synchronized with the transport layer and blocks the running thread, waiting until a new data packet has been received. Thus, the expected cycle time of the real-time thread in ideal conditions would be that of the publisher thread, equal to 1000 μ s. Figure 6.5 shows that the actual cycle time averages to that value, but the jitter reaches considerable peaks of over 100 μ s. This is due to the fact that each time the thread must wait for a new data packet, it hands over control to the operating system: the thread is first removed from the run queue of its CPU, then it is put back in when a new data packet is received. This operation and all the context switches in between introduce non-deterministic latencies that, no matter how efficient and deterministic the kernel scheduler is, become noticeable when the timings have this order of magnitude. Thus, this behavior is expected and compliant with the RFM2g implementation but it is unavoidable and confirms that this execution mode suits data collection and offline diagnostics use cases.

Subscriber side in NotSynchronized mode

Figure 6.6 depicts the steady-state cycle time of a real-time MARTe2 thread running a subscriber DDSDataSource in NotSynchronized mode. Recall that, in this mode, the DataSource does not enforce a rate for the real-time thread loop, thus a LinuxTimer DataSource has been added to the tasks performed by the thread. The rate of the LinuxTimer, to test the capabilities of the implementation, has been pushed at 5 kHz: five times higher than that of the publisher, resulting in an expected cycle time of 200 μ s. Note that, by configuring the real-time thread in this way and with this execu-

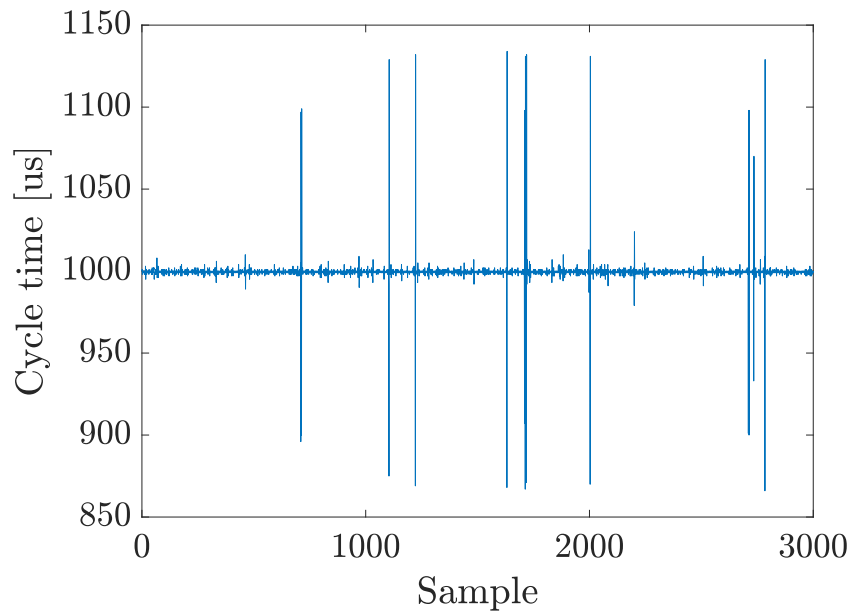


Figure 6.5: Steady-state cycle time of the real-time thread containing a Subscriber DDSDataSource running in Synchronized mode.

tion mode, the DDSDataSource copies and returns the same 4 kB sample five times for every publisher cycle. Figure 6.6 shows that the actual cycle time is, in practice, compatible with the expected value: the maximum jitter observed at steady-state is equal to 5 μ s, which is negligible compared to the overall cycle time. Moreover, the jitter is so small and periodic that it is fully compatible with the maximum time resolution achievable by the LinuxTimer, which is providing the clock to the entire real-time thread. This final test confirmed that the DDSDataSource implementation is also suitable for fast feedback control applications when configured in this way, and confirmed it as a replacement for the RFM2g DataSource in the SCD architecture.

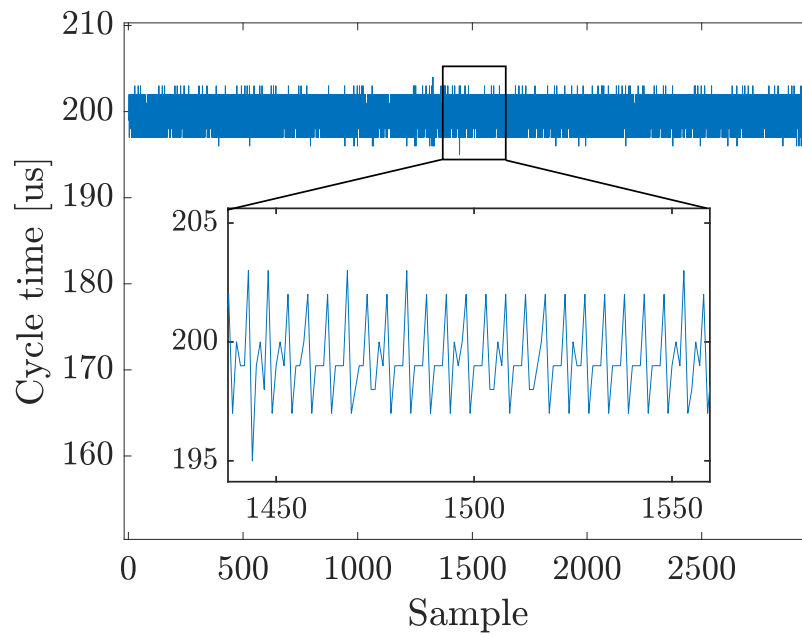


Figure 6.6: Steady-state cycle time of the real-time thread containing a Subscriber DDSDataSource running in NotSynchronized mode.

A The dua-template update workflow

```
1 # Workflow to sync local repository with latest version of DUA template.
2 #
3 # Roberto Masocco <r.masocco@dotxautomation.com>
4 #
5 # June 13, 2024
6
7 # Copyright 2024 dotX Automation s.r.l.
8 #
9 # Licensed under the Apache License, Version 2.0 (the "License");
10 # you may not use this file except in compliance with the License.
11 # You may obtain a copy of the License at
12 #
13 #     http://www.apache.org/licenses/LICENSE-2.0
14 #
15 # Unless required by applicable law or agreed to in writing, software
16 # distributed under the License is distributed on an "AS IS" BASIS,
17 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
18 # See the License for the specific language governing permissions and
19 # limitations under the License.
20
21 name: Sync DUA template
22
23 on:
24   # Run every day at 00:01
25   schedule:
26     - cron: "1 0 * * *"
27
28   # Run when base branch is updated
29   push:
30     branches:
```

```
31     - master
32
33     # Run when manually triggered
34     workflow_dispatch:
35
36     env:
37         BASE_BRANCH: master
38         HEAD_BRANCH: chore/sync-dua-template
39         GIT_AUTHOR_NAME: ${ github.repository_owner }
40         GIT_AUTHOR_EMAIL: ${ github.repository_owner }@users.noreply.github.com
41         REPO_TEMPLATE: dotX-Automation/dua-template
42         THIS_REPO: ${ github.repository }
43
44     jobs:
45         sync-dua-template:
46             # Do not run on the template repository itself
47             if: github.repository != 'dotX-Automation/dua-template'
48
49             name: Sync DUA template
50             runs-on: ubuntu-latest
51             continue-on-error: false
52
53             steps:
54                 # Configure Git to reduce warnings
55                 - name: Configure Git to reduce warnings
56                   run: git config --global init.defaultBranch master
57
58                 # Clone the template repository (we need the full history of this)
59                 - name: Check out template repository
60                   uses: actions/checkout@v4
61                   with:
62                       repository: ${ env.REPO_TEMPLATE }
63                       ref: 'master'
64                       token: ${ github.token }
65                       path: ${ env.REPO_TEMPLATE }
66                       fetch-depth: 0
67
68                 # Clone the target repository
69                 - name: Check out ${ github.repository }
70                   uses: actions/checkout@v4
```

```

71     with:
72         repository: ${GITHUB_REPOSITORY}
73         ref: ${GITHUB_BASE_BRANCH}
74         token: ${GITHUB_TOKEN}
75         path: ${GITHUB_REPOSITORY}
76
77     # Checkout a branch
78     - name: Create branch in ${GITHUB_REPOSITORY}
79       run: |
80         git -C "${GITHUB_REPOSITORY}" fetch origin "${GITHUB_HEAD_BRANCH}" || true
81         git -C "${GITHUB_REPOSITORY}" branch -a
82         git -C "${GITHUB_REPOSITORY}" checkout -B "${GITHUB_HEAD_BRANCH}" \
83             "remotes/origin/${GITHUB_HEAD_BRANCH}" || \
84         git -C "${GITHUB_REPOSITORY}" checkout -b "${GITHUB_HEAD_BRANCH}"
85
86     # Parse template contents and synchronize changes
87     - name: Parse template contents and synchronize changes
88       run: |
89         _files="$(find ${REPO_TEMPLATE} \
90             ! -path "*/.git/*" \
91             ! -path "*/.github/workflows/*" \
92             ! -path "*/.vscode/*" \
93             ! -name ".gitignore" \
94             ! -name "LICENSE" \
95             ! -name "README.md" \
96             -type f \
97             -print)"
98         _deleted_files="$(git -C ${REPO_TEMPLATE} log \
99             --diff-filter=D \
100             --name-only \
101             --pretty=format: | grep -v '^$')"
102         for _deleted_file in ${_deleted_files}; do
103             _file_path="${GITHUB_REPOSITORY}/${_deleted_file}${REPO_TEMPLATE}/"
104             if [[ -f "${_file_path}" ]]; then
105                 echo "INFO: Deleting ${_file_path}"
106                 rm -f "${_file_path}"
107             fi
108         done
109         for _file in ${_files}; do
110             _src="${_file}"

```



```

111     _dst="${THIS_REPO}/${_file}${REPO_TEMPLATE}/"
112     _dst="${_dst%/*}/"
113     mkdir -p "${_dst}"
114     echo "INFO: Copying '${_src}' to '${_dst}'"
115     cp "${_src}" "${_dst}"
116     done
117     git -C "${THIS_REPO}" diff
118
119     # Commit changes, push, and create a pull request
120     - name: Push changes and create a pull request
121     env:
122         GH_TOKEN: ${GITHUB_TOKEN}
123     run: |
124         if [[ -n $(git -C "${THIS_REPO}" status --porcelain) ]]; then
125             git -C "${THIS_REPO}" config user.name "${GIT_AUTHOR_NAME}"
126             git -C "${THIS_REPO}" config user.email "${GIT_AUTHOR_EMAIL}"
127             git -C "${THIS_REPO}" add .
128             git -C "${THIS_REPO}" commit -m \
129                 "chore: sync to dua-template@${GITHUB_SHA}"
130             if [[ -n $(git -C "${THIS_REPO}" branch -r --list \
131                 "origin/${HEAD_BRANCH}") ]]; then
132                 git -C "${THIS_REPO}" push
133                 echo "INFO: Pushed changes to branch ${HEAD_BRANCH}"
134             else
135                 git -C "${THIS_REPO}" push -u origin "${HEAD_BRANCH}"
136                 pushd "${THIS_REPO}"
137                 gh pr create \
138                     --base "${BASE_BRANCH}" \
139                     --head "${HEAD_BRANCH}" \
140                     --title "Sync to dua-template@${GITHUB_SHA}" \
141                     --body "Sync to dua-template@${GITHUB_SHA}."
142                 popd
143                 echo "INFO: Created new pull request from \
144                     ${HEAD_BRANCH} to ${BASE_BRANCH}"
145             fi
146         else
147             echo "INFO: No changes to commit"
148         fi

```

Listing A.1: dua-template update GitHub workflow as of November 22, 2024 [115].

B Unit configurations

```
1 # flight_stack START #
2 # Install PX4 general dependencies
3 RUN apt-get update && apt-get install -y --no-install-recommends \
4     astyle \
5     libxml2-dev \
6     libgstreamer-plugins-base1.0-dev \
7     rsync && \
8     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
9
10 # Install PX4 Python general dependencies
11 RUN . /opt/dua-venv/bin/activate && yes | pip3 install -U \
12     future \
13     jinja2>=2.8 \
14     jsonschema \
15     kconfiglib \
16     lxml \
17     psutil \
18     pygments \
19     pymavlink \
20     pyulog
21
22 # Install NuttX toolchain dependencies
23 # Get these from: PX4-Autopilot/Tools/setup/ubuntu.sh
24 # and merge with your base unit's contents
25 RUN apt-get update && apt-get install -y --no-install-recommends \
26     automake \
27     binutils-dev \
28     bison \
29     flex \
30     g++-multilib \
```

```

31 gcc-multilib \
32 gdb-multiarch \
33 genromfs \
34 gettext \
35 gperf \
36 libelf-dev \
37 libexpat-dev \
38 libgmp-dev \
39 libisl-dev \
40 libmpc-dev \
41 libmpfr-dev \
42 libncurses5 \
43 libncurses5-dev \
44 libncursesw5-dev \
45 libtool \
46 texinfo \
47 u-boot-tools \
48 util-linux \
49 vim-common && \
50 rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
51
52 # Download and install NuttX toolchain (pray that this remains online)
53 WORKDIR /opt
54 RUN wget -O /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 \
55     https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-rm/9-2020q2/\
56     gcc-arm-none-eabi-9-2020-q2-update-x86_64-linux.tar.bz2 && \
57     tar -jxf /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 -C /opt/ && \
58     rm /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 && \
59     chgrp -R internal gcc-arm-none-eabi-9-2020-q2-update && \
60     chmod -R g+rw gcc-arm-none-eabi-9-2020-q2-update
61 ENV PATH=/opt/gcc-arm-none-eabi-9-2020-q2-update/bin:${PATH}
62 WORKDIR /root
63
64 # Build and install Foonathan memory allocator
65 WORKDIR /opt
66 RUN git clone --single-branch --branch 'v1.3.0' \
67     https://github.com/eProsima/foonathan_memory_vendor.git && \
68     cd foonathan_memory_vendor && \
69     mkdir build && \
70     cd build && \

```

```
71  cmake .. && \
72  cmake --build . --target install && \
73  cd ../../ && \
74  chgrp -R internal foonathan_memory_vendor && \
75  chmod -R g+rw foonathan_memory_vendor
76 WORKDIR /root
77
78 # Build and install Fast-DDS-Gen (required by microRTPS Bridge)
79 WORKDIR /opt
80 RUN git clone --recursive --single-branch --branch 'v2.4.0' \
81  https://github.com/eProsima/Fast-DDS-Gen.git && \
82  cd Fast-DDS-Gen && \
83  ./gradlew assemble && \
84  ln -s /opt/Fast-DDS-Gen/scripts/fastddsgen scripts/fastrtpsgen && \
85  cd .. && \
86  chgrp -R internal Fast-DDS-Gen && \
87  chmod -R g+rw Fast-DDS-Gen
88 WORKDIR /root
89 ENV PATH=/opt/Fast-DDS-Gen/scripts:${PATH}
90
91 # Build and install Fast-CDR
92 WORKDIR /opt
93 RUN git clone --single-branch --branch 'v1.1.1' \
94  https://github.com/eProsima/Fast-CDR.git && \
95  cd Fast-CDR && \
96  mkdir build && \
97  cd build && \
98  cmake .. && \
99  make -j$(nproc --all) && \
100  make install -j$(nproc --all) && \
101  cd ../../ && \
102  chgrp -R internal Fast-CDR && \
103  chmod -R g+rw Fast-CDR
104 WORKDIR /root
105
106 # Build and install Fast-DDS
107 WORKDIR /opt
108 RUN git clone --single-branch --branch 'v2.10.1' \
109  https://github.com/eProsima/Fast-DDS.git && \
110  cd Fast-DDS && \
```

```
111  mkdir build && \  
112  cd build && \  
113  cmake -DTHIRDPARTY=ON .. && \  
114  make -j$(nproc --all) && \  
115  make install -j$(nproc --all) && \  
116  cd ../../ && \  
117  chgrp -R internal Fast-DDS && \  
118  chmod -R g+rw Fast-DDS  
119  WORKDIR /root  
120 # flight_stack END #
```

Listing B.1: Unit configuration of the x86-cudev target of the `flight_stack` DUA unit [137] as of November 26, 2024.

Bibliography

- [1] E. S. Raymond. *How To Become A Hacker*. 2001. URL: <http://www.catb.org/esr/faqs/hacker-howto.html> (visited on 11/04/2024).
- [2] MathWorks. *MATLAB*. URL: <https://www.mathworks.com/products/matlab.html> (visited on 11/13/2024).
- [3] MathWorks. *Simulink*. URL: <https://www.mathworks.com/products/simulink.html> (visited on 11/13/2024).
- [4] T. Fischer et al. “A RoboStack Tutorial: Using the Robot Operating System Alongside the Conda and Jupyter Data Science Ecosystems”. In: *IEEE Robotics & Automation Magazine* 29.2 (June 2022), pp. 65–74.
- [5] Anaconda Inc. *Conda*. URL: <https://docs.conda.io/projects/conda/en/stable/> (visited on 11/06/2024).
- [6] eProsima. *Vulcanexus, the All-in-One ROS 2 tool set*. URL: <https://vulcanexus.org/> (visited on 11/06/2024).
- [7] eProsima. *Fast DDS*. URL: <https://fast-dds.docs.eprosima.com/en/latest/> (visited on 11/20/2024).
- [8] L. R. Dalesio, A. J. Kozubal, and M. R. Kraimer. “EPICS architecture”. English. In: *Proc. of the International Conference on Accelerator and Large Experimental Physics Control Systems*. Los Alamos National Lab., NM (United States), 1991.
- [9] EPICS Controls. *EPICS - Experimental Physics and Industrial Control System*. URL: <https://epics-controls.org/> (visited on 11/23/2024).
- [10] L. Calacci, D. Carnevale, and D. Abruzzese. “BlockNet: a new flexible control architecture”. In: *2020 28th Mediterranean Conference on Control and Automation (MED)*. Sept. 2020, pp. 1039–1044.

Bibliography

- [11] R. L. Bocchino et al. “F Prime: An Open-Source Framework for Small-Scale Flight Software Systems”. In: *Proc. of the 2018 Small Satellite Conference*. Aug. 2018.
- [12] Fusion for Energy. *MARTe2*. URL: <https://vcis.f4e.europa.eu/marte2-docs/master/html/> (visited on 11/16/2024).
- [13] A. C. Neto et al. “MARTe: A Multiplatform Real-Time Framework”. In: *IEEE Transactions on Nuclear Science* 57.2 (Apr. 2010), pp. 479–486.
- [14] P. Naur and B. Randell. *Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th-11th October 1968*. Tech. rep. NATO Scientific Affairs Division, 1969.
- [15] G. Pardo-Castellote. “OMG Data-Distribution Service: architectural overview”. In: *Proc. of the 23rd International Conference on Distributed Computing Systems*. May 2003, pp. 200–206.
- [16] J. M. Schlesselman, G. Pardo-Castellote, and B. Farabaugh. “OMG data-distribution service (DDS): architectural update”. In: *Proc. of the 2004 IEEE Military Communications Conference (MILCOM)*. Vol. 2. Oct. 2004, pp. 961–967.
- [17] M. S. Essers and T. H. J. Vaneker. “Evaluating a Data Distribution Service System for Dynamic Manufacturing Environments: A Case Study”. In: *Procedia Technology*. 2nd International Conference on System-Integrated Intelligence: Challenges for Product and Production Engineering 15 (Jan. 2014), pp. 621–630.
- [18] S. Saxena, H. E. Z. Farag, and N. El-Taweel. “A distributed communication framework for smart Grid control applications based on data distribution service”. In: *Electric Power Systems Research* 201 (Dec. 2021), p. 107547.
- [19] A. Corsaro et al. “Quality of service in publish/subscribe middleware”. In: *Global Data Management* 19.20 (2006), pp. 1–22.
- [20] eProxima. 1.1. *What is DDS? — Fast DDS 3.1.0 documentation*. URL: https://fast-dds.docs.eprosima.com/en/latest/fastdds/getting_started/definitions.html (visited on 11/13/2024).
- [21] A. Corsaro et al. “Zenoh: Unifying Communication, Storage and Computation from the Cloud to the Microcontroller”. In: *2023 26th Euromicro Conference on Digital System Design (DSD)*. Sept. 2023, pp. 422–428.

Bibliography

- [22] G. Baldoni et al. “Facilitating distributed data-flow programming with Eclipse Zenoh: the ERDOS case”. In: *Proc. of the 1st Workshop on Serverless mobile networking for 6G Communications*. Association for Computing Machinery, July 2021, pp. 13–18.
- [23] S. Greising and A. Reichert. “Zenoh Whitepaper”. FLECS Technologies GmbH, Jan. 2023.
- [24] S. Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (May 2022), eabm6074.
- [25] Open Robotics. *Robot Operating System*. URL: <https://www.ros.org/> (visited on 11/16/2024).
- [26] M. Quigley et al. “ROS: an open-source Robot Operating System”. In: *ICRA workshop on open source software*. Vol. 3. May 2009.
- [27] Open Robotics. *Installation — ROS 2 Documentation: Rolling documentation*. URL: <https://docs.ros.org/en/rolling/Installation.html> (visited on 11/12/2024).
- [28] Canonical. *Ubuntu Linux*. URL: <https://ubuntu.com/> (visited on 11/12/2024).
- [29] Open Robotics. *Open Robotics*. URL: <https://www.openrobotics.org> (visited on 11/12/2024).
- [30] B. Gerkey. *Why ROS 2?* June 2014. URL: https://design.ros2.org/articles/why_ros2.html (visited on 11/13/2024).
- [31] D. Thomas. *Changes between ROS 1 and ROS 2*. Sept. 2015. URL: <https://design.ros2.org/articles/changes.html> (visited on 11/13/2024).
- [32] W. Woodall. *ROS on DDS*. June 2014. URL: https://design.ros2.org/articles/ros_on_dds.html (visited on 11/13/2024).
- [33] Open Robotics. *ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/index.html> (visited on 11/13/2024).
- [34] Open Robotics. *Tutorials — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials.html> (visited on 11/13/2024).
- [35] D. Thomas. *A universal build tool*. Mar. 2017. URL: https://design.ros2.org/articles/build_tool.html (visited on 11/13/2024).
- [36] D. Thomas. *colcon - collective construction*. URL: <https://colcon.readthedocs.io/en/released/> (visited on 11/13/2024).

Bibliography

- [37] kitware. *CMake*. URL: <https://cmake.org/> (visited on 11/13/2024).
- [38] P. Eby et al. *Setuptools*. URL: <https://setuptools.pypa.io/en/latest/userguide/> (visited on 11/13/2024).
- [39] D. Thomas. *The build system “ament_cmake” and the meta build tool “ament_tools”*. July 2015. URL: <https://design.ros2.org/articles/ament.html> (visited on 11/13/2024).
- [40] D. Thomas. *Interface definition using .msg / .srv / .action files*. Mar. 2019. URL: https://design.ros2.org/articles/legacy_interface_definition.html (visited on 11/13/2024).
- [41] Open Robotics. *Understanding topics — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html> (visited on 11/13/2024).
- [42] Open Robotics. *Understanding services — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html> (visited on 11/13/2024).
- [43] Open Robotics. *Understanding actions — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html> (visited on 11/13/2024).
- [44] G. Biggs, J. Perron, and S. Loretz. *Actions*. URL: <https://design.ros2.org/articles/actions.html> (visited on 11/16/2024).
- [45] S. Macenski et al. “Impact of ROS 2 Node Composition in Robotic Systems”. In: *IEEE Robotics and Automation Letters* 8.7 (July 2023), pp. 3996–4003.
- [46] C. Bédard et al. “Message flow analysis with complex causal links for distributed ROS 2 systems”. In: *Robotics and Autonomous Systems* 161 (Mar. 2023), p. 104361.
- [47] Yuya Maruyama, S. Kato, and T. Azumi. “Exploring the performance of ROS2”. In: *2016 International Conference on Embedded Software (EMSOFT)*. Oct. 2016, pp. 1–10.
- [48] L. Puck et al. “Distributed and Synchronized Setup towards Real-Time Robotic Control using ROS2 on Linux”. In: *2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*. Aug. 2020, pp. 1287–1293.

Bibliography

- [49] T. Kronauer et al. “Latency Analysis of ROS2 Multi-Node Systems”. In: *2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. Sept. 2021, pp. 1–7.
- [50] J. Staschulat, I. Lutkebohle, and R. Lange. “The rclcpp Executor: Domain-specific deterministic scheduling mechanisms for ROS applications on microcontrollers: work-in-progress”. In: *Proc. of the 2020 International Conference on Embedded Software (EMSOFT)*. IEEE, Sept. 2020, pp. 18–19.
- [51] Open Robotics. *ROS 2 Alternative middleware report*. Tech. rep. Sept. 2023. URL: <https://discourse.ros.org/t/ros-2-alternative-middleware-report/33771> (visited on 11/13/2024).
- [52] Open Robotics. *Launch — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Launch-Main.html> (visited on 11/13/2024).
- [53] Open Robotics. *Recording and playing back data — ROS 2 Documentation*. URL: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html> (visited on 11/13/2024).
- [54] National Instruments. *LabVIEW*. URL: <https://www.ni.com/en/shop/labview.html> (visited on 11/13/2024).
- [55] Open Robotics. *Gazebo*. URL: <https://gazebo.org/home> (visited on 11/13/2024).
- [56] Open Robotics. *RViz*. URL: <https://github.com/ros2/rviz> (visited on 11/13/2024).
- [57] Open Navigation. *Nav2*. URL: <https://docs.nav2.org/> (visited on 11/13/2024).
- [58] S. Macenski et al. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* 168 (Oct. 2023), p. 104493.
- [59] S. Macenski et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2020, pp. 2718–2725.
- [60] S. Macenski et al. “Regulated pure pursuit for robot path tracking”. In: *Autonomous Robots* 47.6 (Aug. 2023), pp. 685–694.

Bibliography

- [61] A. Merzlyakov and S. Macenski. “A Comparison of Modern General-Purpose Visual SLAM Approaches”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sept. 2021, pp. 9190–9197.
- [62] M. Iovino et al. “A survey of Behavior Trees in robotics and AI”. In: *Robotics and Autonomous Systems* 154 (2022), p. 104096.
- [63] PickNik Robotics. *MoveIt 2*. URL: <https://moveit.picknik.ai/main/index.html> (visited on 11/13/2024).
- [64] Michael Görner et al. “MoveIt! Task Constructor for Task-Level Motion Planning”. In: *2019 International Conference on Robotics and Automation (ICRA)*. ISSN: 2577-087X. May 2019, pp. 190–196. DOI: 10.1109/ICRA.2019.8793898. URL: <https://ieeexplore.ieee.org/abstract/document/8793898> (visited on 11/12/2024).
- [65] D. T. Coleman et al. “Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study”. In: *Journal of Software Engineering for Robotics* 5.1 (May 2014), pp. 3–16.
- [66] A. Bonci et al. “Robot Operating System 2 (ROS2)-Based Frameworks for Increasing Robot Autonomy: A Survey”. In: *Applied Sciences* 13.23 (Jan. 2023), p. 12796.
- [67] L. Bianchi et al. “A novel distributed architecture for unmanned aircraft systems based on Robot Operating System 2”. In: *IET Cyber-Systems and Robotics* 5.1 (2023), e12083.
- [68] L. Bianchi et al. “Efficient visual sensor fusion for autonomous agents”. In: *Proc. of the 7th International Conference on Control, Automation and Diagnosis (ICCAD)*. IEEE, May 2023, pp. 01–06.
- [69] eProsima. *micro-ROS*. URL: <https://micro.ros.org/> (visited on 11/13/2024).
- [70] eProsima. *Micro XRCE-DDS*. URL: <https://micro-xrce-dds.docs.eprosima.com/en/latest/> (visited on 11/13/2024).
- [71] STMicroelectronics. *STM32 Microcontrollers*. URL: <https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html> (visited on 11/13/2024).
- [72] Espressif Systems. *ESP32 Wi-Fi & Bluetooth SoC*. URL: <https://www.espressif.com/en/products/socs/esp32> (visited on 11/13/2024).

Bibliography

- [73] C. Pahl et al. "Cloud Container Technologies: A State-of-the-Art Review". In: *IEEE Transactions on Cloud Computing* 7.3 (July 2019), pp. 677–692.
- [74] R. Dua, A R. Raja, and D. Kakadia. "Virtualization vs Containerization to Support PaaS". In: *2014 IEEE International Conference on Cloud Engineering*. Mar. 2014, pp. 610–614.
- [75] Docker Inc. *Docker*. URL: <https://www.docker.com/> (visited on 11/14/2024).
- [76] D. Merkel. "Docker: lightweight Linux containers for consistent development and deployment". In: *Linux Journal* 2014.239 (May 2014), 2:2. URL: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment> (visited on 11/14/2024).
- [77] J. Turnbull. *The Docker Book: Containerization Is the New Virtualization*. James Turnbull, July 2014.
- [78] Docker Inc. *Docker Hub Container Image Library*. URL: <https://hub.docker.com> (visited on 11/14/2024).
- [79] Docker Inc. *Docker Desktop*. URL: <https://www.docker.com/products/docker-desktop/> (visited on 11/14/2024).
- [80] C. Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Operating Systems Review* 49.1 (Jan. 2015), pp. 71–79.
- [81] R. Dua et al. "Performance analysis of Union and CoW File Systems with Docker". In: *2016 International Conference on Computing, Analytics and Security Trends (CAST)*. Dec. 2016, pp. 550–555.
- [82] N. Mizusawa, K. Nakazima, and S. Yamaguchi. "Performance Evaluation of File Operations on OverlayFS". In: *2017 Fifth International Symposium on Computing and Networking (CANDAR)*. Nov. 2017, pp. 597–599.
- [83] Docker Inc. *Docker Compose*. URL: <https://github.com/docker/compose> (visited on 11/14/2024).
- [84] N. N. Zolkifli, A. Ngah, and A. Deraman. "Version Control System: A Review". In: *Procedia Computer Science* 135 (Jan. 2018), pp. 408–415.
- [85] L. Torvalds et al. *Git*. URL: <https://git-scm.com/> (visited on 11/14/2024).
- [86] J. Loeliger and M. McCullough. *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*. "O'Reilly Media, Inc.", Aug. 2012.

Bibliography

- [87] C. Bird et al. "The promises and perils of mining git". In: *2009 6th IEEE International Working Conference on Mining Software Repositories*. May 2009, pp. 1–10.
- [88] L. Torvalds et al. *Linux kernel*. URL: <https://kernel.org/> (visited on 11/14/2024).
- [89] C. Rodríguez-Bustos and J. Aponte. "How Distributed Version Control Systems impact open source software projects". In: *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*. June 2012, pp. 36–39.
- [90] Microsoft. *Windows Subsystem for Linux*. URL: <https://learn.microsoft.com/en-us/windows/wsl/about> (visited on 11/19/2024).
- [91] F. Bellard. *QEMU*. URL: <https://www.qemu.org/> (visited on 11/20/2024).
- [92] R. Masocco. *dua-foundation*. URL: <https://github.com/dotX-Automation/dua-foundation> (visited on 11/16/2024).
- [93] R. Masocco. *dotxautomation/dua-foundation - Docker Hub*. URL: <https://hub.docker.com/r/dotxautomation/dua-foundation> (visited on 11/20/2024).
- [94] Canonical. *ubuntu - Official Image | Docker Hub*. URL: https://hub.docker.com/_/ubuntu (visited on 11/21/2024).
- [95] Free Software Foundation. *GCC, the GNU Compiler Collection*. URL: <https://gcc.gnu.org/> (visited on 11/20/2024).
- [96] G. Van Rossum. *Python*. URL: <https://www.python.org/> (visited on 11/21/2024).
- [97] Oracle. *Java*. URL: <https://dev.java/> (visited on 11/20/2024).
- [98] Eclipse Foundation. *Cyclone DDS*. URL: <https://cyclonedds.io/> (visited on 11/20/2024).
- [99] OpenCV team. *OpenCV*. URL: <https://opencv.org/> (visited on 11/20/2024).
- [100] I. Culjak et al. "A brief introduction to OpenCV". In: *2012 Proceedings of the 35th International Convention MIPRO*. May 2012, pp. 1725–1730.
- [101] B. Jacob and G. Guennebaud. *Eigen*. URL: https://eigen.tuxfamily.org/index.php?title=Main_Page (visited on 11/20/2024).
- [102] Raspberry Pi Ltd. *Raspberry Pi*. URL: <https://www.raspberrypi.com/> (visited on 11/21/2024).
- [103] AAEON Technology. *UP Bridge the Gap - Edge Computing Devices*. URL: <https://up-board.org/> (visited on 11/21/2024).

Bibliography

- [104] Nvidia. *Jetson Modules*. URL: <https://developer.nvidia.com/embedded/jetson-modules> (visited on 11/21/2024).
- [105] C. Song et al. "Hardware for Deep Learning Acceleration". In: *Advanced Intelligent Systems* 6.10 (2024), p. 2300762.
- [106] Nvidia. *JetPack SDK*. URL: <https://developer.nvidia.com/embedded/jetpack> (visited on 11/21/2024).
- [107] Nvidia. *CUDA Deep Neural Network*. URL: <https://developer.nvidia.com/cudnn> (visited on 11/21/2024).
- [108] Nvidia. *CUDA Toolkit*. URL: <https://developer.nvidia.com/cuda-toolkit> (visited on 11/21/2024).
- [109] PyTorch Foundation. *PyTorch*. URL: <https://pytorch.org/> (visited on 11/21/2024).
- [110] F. Chollet and ONEIROS. *Keras*. URL: <https://keras.io/> (visited on 11/21/2024).
- [111] Linux Foundation. *ONNX*. URL: <https://onnx.ai/> (visited on 11/21/2024).
- [112] Google Brain Team. *TensorFlow*. URL: <https://www.tensorflow.org/> (visited on 11/21/2024).
- [113] Ultralytics Inc. *Ultralytics YOLO*. URL: <https://www.ultralytics.com/yolo> (visited on 11/21/2024).
- [114] Nvidia. *NVIDIA L4T ML*. URL: <https://catalog.ngc.nvidia.com/orgs/nvidia/containers/l4t-ml> (visited on 11/21/2024).
- [115] R. Masocco. *dua-template*. URL: <https://github.com/dotX-Automation/dua-template> (visited on 11/16/2024).
- [116] GitHub. *GitHub*. URL: <https://github.com> (visited on 11/21/2024).
- [117] Microsoft. *Visual Studio Code*. URL: <https://code.visualstudio.com/> (visited on 11/21/2024).
- [118] R. Masocco. *dua-utils*. URL: <https://github.com/dotX-Automation/dua-utils> (visited on 11/16/2024).
- [119] E. Freeman et al. *Head First Design Patterns: A Brain-Friendly Guide*. 2nd ed. O'Reilly Media, Inc., Dec. 2020.
- [120] R. Masocco. *dua_qos*. URL: https://github.com/dotX-Automation/dua_qos (visited on 11/16/2024).

Bibliography

- [121] R. Masocco and L. Bianchi. *params_manager*. URL: https://github.com/dotX-Automation/params_manager (visited on 11/16/2024).
- [122] L. J. Guibas and R. Sedgewick. “A dichromatic framework for balanced trees”. In: *19th Annual Symposium on Foundations of Computer Science (SFCS 1978)*. Oct. 1978, pp. 8–21.
- [123] R. Masocco. *dua_node*. URL: https://github.com/dotX-Automation/dua_node (visited on 11/16/2024).
- [124] R. Masocco. *simple_serviceclient*. URL: https://github.com/dotX-Automation/simple_serviceclient (visited on 11/16/2024).
- [125] R. Masocco. *simple_actionclient*. URL: https://github.com/dotX-Automation/simple_actionclient (visited on 11/16/2024).
- [126] F. Wagner et al. *Modeling Software with Finite State Machines: A Practical Approach*. New York: Auerbach Publications, 2006.
- [127] D. Brand and P. Zafiropulo. “On Communicating Finite-State Machines”. In: *Journal of the ACM* 30.2 (1983), pp. 323–342.
- [128] V. Sklyarov. “Hierarchical finite-state machines and their use for digital control”. In: *IEEE Transactions on Very Large Scale Integration Systems* 7.2 (1999), pp. 222–228.
- [129] G. Pola et al. “Symbolic Models for Networks of Control Systems”. In: *IEEE Transactions on Automatic Control* 61.11 (2016), pp. 3663–3668.
- [130] F. B. Schneider. “Implementing fault-tolerant services using the state machine approach: a tutorial”. In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.
- [131] R. Masocco et al. *transitions_ros*. 2022. URL: https://github.com/dotX-Automation/transitions_ros (visited on 11/25/2024).
- [132] T. Yarkoni and A. Neumann. *transitions*. 2017. URL: <https://github.com/pytransitions/transitions> (visited on 11/25/2024).
- [133] L. Meier et al. “PIXHAWK: A system for autonomous flight using onboard computer vision”. In: *2011 IEEE International Conference on Robotics and Automation*. May 2011, pp. 2992–2997.
- [134] L. Meier et al. “PIXHAWK: A micro aerial vehicle design for autonomous flight using onboard computer vision”. In: *Autonomous Robots* 33.1 (Aug. 2012), pp. 21–39.

Bibliography

- [135] L. Meier, D. Honegger, and M. Pollefeys. “PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms”. In: *Proc. of the 2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2015, pp. 6235–6240.
- [136] Dronecode Foundation. *The Dronecode Foundation*. URL: <https://dronecode.org/> (visited on 11/25/2024).
- [137] R. Masocco. *flight_stack*. URL: https://github.com/IntelligentSystemsLabUTV/flight_stack (visited on 11/25/2024).
- [138] eProsima. *Fast-DDS-Gen*. URL: <https://github.com/eProsima/Fast-DDS-Gen> (visited on 11/26/2024).
- [139] Apache Software Foundation. *Apache NuttX*. URL: <https://nuttx.apache.org/> (visited on 11/26/2024).
- [140] Stereolabs. *Stereolabs | AI perception for automation*. URL: <https://www.stereolabs.com> (visited on 11/26/2024).
- [141] Stereolabs. *ZED SDK*. URL: <https://www.stereolabs.com/developers/release> (visited on 11/26/2024).
- [142] R. Masocco. *zed_drivers*. URL: https://github.com/IntelligentSystemsLabUTV/zed_drivers (visited on 11/25/2024).
- [143] Nvidia. *Nvidia Video Codec SDK*. URL: <https://developer.nvidia.com/video-codec-sdk> (visited on 11/26/2024).
- [144] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer. “Speeded up detection of squared fiducial markers”. In: *Image and Vision Computing* 76 (Aug. 2018), pp. 38–47.
- [145] S. Garrido-Jurado et al. “Generation of fiducial marker dictionaries using Mixed Integer Linear Programming”. In: *Pattern Recognition* 51 (Mar. 2016), pp. 481–491.
- [146] R. Mur-Artal and Juan D. Tardós. “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras”. In: *IEEE Transactions on Robotics* 33.5 (Oct. 2017), pp. 1255–1262.
- [147] A. K. Guruji, H. Agarwal, and D. K. Parsediya. “Time-efficient A* Algorithm for Robot Path Planning”. In: *Procedia Technology* 23 (Jan. 2016), pp. 144–149.

Bibliography

- [148] F. F. Chen. *Introduction to plasma physics*. Springer Science & Business Media, 2012.
- [149] J. Wesson and D. J. Campbell. *Tokamaks*. Vol. 149. Oxford university press, 2011.
- [150] M. Ariola and A. Pironti. *Magnetic control of tokamak plasmas*. Vol. 187. Springer, 2008.
- [151] C. M. Braams and P. E. Stott. *Nuclear Fusion: Half a Century of Magnetic Confinement Fusion Research*. CRC Press, June 2002.
- [152] B. Esposito et al. “Runaway electron generation and control”. In: *Plasma Physics and Controlled Fusion* 59.1 (Nov. 2016), p. 014044.
- [153] A. Mele et al. “Design of a novel plasma shape controller for the TCV tokamak”. In: *2024 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. July 2024, pp. 284–289.
- [154] A. Tenaglia et al. “A unified approach to plasma shape control with coil current allocation in magnetic confinement fusion”. In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. Vol. P2.085. 2024.
- [155] “The control of tokamak configuration variable plasmas”. In: *Fusion Technology* (1997). Ed. by J. B. Lister et al.
- [156] H. Reimerdes et al. “Initial TCV operation with a baffled divertor”. In: *Nuclear Fusion* 61.2 (Jan. 2021), p. 024002.
- [157] B. P. Duval et al. “Experimental research on the TCV tokamak”. In: *Nuclear Fusion* 64.11 (Oct. 2024). Publisher: IOP Publishing, p. 112023.
- [158] E. Joffrin et al. “Overview of the EUROfusion Tokamak Exploitation programme in support of ITER and DEMO”. In: *Nuclear Fusion* 64.11 (Aug. 2024), p. 112019.
- [159] H. B. Le et al. “Distributed digital real-time control system for TCV tokamak”. In: *Fusion Engineering and Design* 89.3 (Mar. 2014), pp. 155–164.
- [160] H. Anand, C. Galperti, S. Coda, et al. “Distributed digital real-time control system for the TCV tokamak and its applications”. In: *Nuclear Fusion* 57.5 (2017).
- [161] C. Galperti et al. “Overview of the TCV digital real-time plasma control system and its applications”. In: *Fusion Engineering and Design* 208 (Nov. 2024), p. 114640.

Bibliography

- [162] F. Santoro et al. “Exploring MDSplus data-acquisition software and custom devices”. In: *Fusion Engineering and Design* 155 (June 2020), p. 111534.
- [163] Abaco Systems. *PCIE-5565RC Interface Card* | Abaco Systems. URL: <https://www.abaco.com/products/pcie-5565rc-interface-card> (visited on 12/03/2024).
- [164] R. Masocco et al. “A distributed real-time diagnostic and control network for the TCV tokamak based on the Data Distribution Service”. In: *33rd Symposium on Fusion Technology (SOFT 2024)*. Sept. 2024. DOI: 10.13140/RG.2.2.18403.16166.